

UNIVERSIDAD MAYOR DE SAN ANDRÉS
FACULTAD DE INGENIERÍA
CARRERA INGENIERÍA ELECTRÓNICA



PROYECTO DE GRADO
DISEÑO Y SIMULACIÓN DE UNA ARQUITECTURA DE RED
DEFINIDA POR SOFTWARE EN BASE AL PROTOCOLO
OPENFLOW

Presentado por:

Gustavo Coronel Flores

Tutor:

Ing. Roberto Zambrana Flores

La Paz - Bolivia

2019



**UNIVERSIDAD MAYOR DE SAN ANDRÉS
FACULTAD DE INGENIERIA**



LA FACULTAD DE INGENIERIA DE LA UNIVERSIDAD MAYOR DE SAN ANDRÉS AUTORIZA EL USO DE LA INFORMACIÓN CONTENIDA EN ESTE DOCUMENTO SI LOS PROPÓSITOS SON Estrictamente Académicos.

LICENCIA DE USO

El usuario está autorizado a:

- a) Visualizar el documento mediante el uso de un ordenador o dispositivo móvil.
- b) Copiar, almacenar o imprimir si ha de ser de uso exclusivamente personal y privado.
- c) Copiar textualmente parte(s) de su contenido mencionando la fuente y/o haciendo la cita o referencia correspondiente en apego a las normas de redacción e investigación.

El usuario no puede publicar, distribuir o realizar emisión o exhibición alguna de este material, sin la autorización correspondiente.

TODOS LOS DERECHOS RESERVADOS. EL USO NO AUTORIZADO DE LOS CONTENIDOS PUBLICADOS EN ESTE SITIO DERIVARA EN EL INICIO DE ACCIONES LEGALES CONTEMPLADAS EN LA LEY DE DERECHOS DE AUTOR.

RESUMEN

Las redes de telecomunicaciones fueron evolucionando a lo largo de la historia con el objetivo de satisfacer las necesidades tecnológicas de instituciones gubernamentales, instituciones académicas, empresas y sobre todo de los usuarios finales, pero en la actualidad se percibe que las redes tradicionales no son lo suficientemente flexibles para reconfigurarse y adaptarse rápidamente de forma automática a la gran variedad de servicios demandados.

En el presente proyecto se presenta una alternativa en la administración de las redes de datos tradicionales con la implementación de una arquitectura de red que permita de manera eficaz una mayor flexibilidad, automatización, gestión y control centralizado, con la filosofía enfocada en la programación de las redes o más conocida como redes definidas por software (SDN - Software Defined Networking), que tienen como aspecto fundamental eliminar la inteligencia de los dispositivos de interconexión desacoplando el plano de datos del plano de control, delegando las capacidades de este último a un elemento llamado controlador SDN.

La simulación de la arquitectura de red definida por software se la realizó en un escenario virtual utilizando exclusivamente software libre, haciendo énfasis en la caracterización del protocolo OpenFlow y en el desarrollo de las aplicaciones que se realizaron tanto para el emulador Mininet como también para el controlador SDN Ryu.

Para verificar su funcionamiento, se desarrolló y se implementó el protocolo de árbol de expansión para gestionar los caminos redundantes en la topología de la red virtualizada, se comprobó el cumplimiento de los protocolos de comunicación entre los dispositivos de la red con el programa Wireshark, se analizó a detalle lo que acontece en el interior de los hosts y switches al momento de generar tráfico de datos a través de la red, y por último se analizaron a detalle los tiempos de respuesta en la comunicación entre los hosts de la red mediante tres pruebas diferentes.

DEDICATORIA

*A la memoria de mi querida madre Sara Flores de Coronel
que se encuentra en el cielo junto a Dios,
y a mi hija Dulce Sarahi por ser la inspiración de mi vida.*

AGRADECIMIENTO

*A Dios que me dio la fortaleza necesaria para alcanzar el objetivo,
a mi familia por el apoyo constante e incondicional,
gracias de todo corazón madrina Rosa, padrino Laureano, Abraham y Daniela.*

TABLA DE CONTENIDO

	Pag.
RESUMEN	I
DEDICATORIA	II
AGRADECIMIENTO	III
TABLA DE CONTENIDO	IV
ÍNDICE DE ABREVIACIONES	VII
CAPÍTULO 1.- INTRODUCCIÓN	1
1.1. Antecedentes.....	1
1.2. Problemática	3
1.3. Objetivos del Proyecto	4
1.3.1. Objetivo General	4
1.3.2. Objetivos Específicos.....	4
1.4. Justificación	4
1.5. Alcances del Proyecto	5
1.6. Limitaciones del Proyecto.....	6
CAPÍTULO 2.- MARCO TEÓRICO	7
2.1. Comparativa de las Redes Definidas por Software con las Redes Tradicionales	7
2.2. Lógica de Funcionamiento de las Redes Definidas por Software	9
2.3. Arquitectura y Elementos que Conforman las Redes Definidas por Software.....	10
2.3.1. Capa de Aplicación	10
2.3.2. Capa de Control	11
2.3.3. Capa de Infraestructura	11
2.4. El Controlador SDN	11
2.4.1. Funcionamiento del Controlador SDN	12
2.4.2. Interfaz de Programación de Aplicaciones (API)	14
2.4.3. Tipos de Controladores SDN	15
2.4.4. Elección del Controlador SDN.....	16
2.5. OpenFlow	18
2.5.1. Dispositivo OpenFlow	18
2.5.1.1. Tabla de Flujos.....	19
2.5.1.1.1. Proceso Pipeline OpenFlow	21
2.5.1.2. Tabla de Grupo.....	22
2.5.1.3. Tabla Miss.....	23
2.5.1.4. Canal Seguro	23
2.5.2. Protocolo OpenFlow	23
2.5.2.1. Funcionamiento del Protocolo OpenFlow	24
2.5.2.2. Tipos de Mensajes OpenFlow	25
2.5.2.2.1. Controlador a Switch.....	25

2.5.2.2.2. Asíncronos	25
2.5.2.2.3. Simétricos.....	25
2.5.2.3. Versiones del Protocolo OpenFlow	25
2.5.2.4. Dispositivos Compatibles con OpenFlow	25
2.6. Mininet.....	26
2.7. Ventajas y Desventajas de las Redes Definidas por Software	28
CAPÍTULO 3.- DISEÑO Y DESARROLLO DE LA ARQUITECTURA DE RED DEFINIDA POR SOFTWARE	30
3.1. Escenario de Implementación	30
3.1.1. Descripción de la Topología de Red	31
3.2. Diseño de la Arquitectura de Red Definida por Software.....	32
3.2.1. Instalación del Software y Herramientas Necesarias para la Simulación	33
3.2.1.1. Instalación del Sistema Operativo Ubuntu	34
3.2.1.2. Instalación del Emulador Mininet	39
3.2.1.3. Instalación del Controlador Ryu.....	41
3.2.2. Diseño y Desarrollo de los Módulos Utilizados por el Emulador Mininet y el Controlador Ryu	43
3.2.2.1. Diseño y Desarrollo del Script que Emula la Red Virtual	43
3.2.2.2. Diseño y Desarrollo de la API para el Controlador Ryu.....	48
3.3. Costo del Proyecto.....	65
CAPÍTULO 4.- SIMULACIÓN Y ANÁLISIS DE FUNCIONAMIENTO DE LA SDN.....	67
4.1. Simulación de la Arquitectura de Red Definida por Software	67
4.1.1. Verificación y Análisis del Establecimiento del Protocolo OpenFlow	70
4.1.2. Verificación y Análisis del Establecimiento del STP	74
4.2. Pruebas Básicas de Funcionamiento de la Red Definida por Software.....	79
4.3. Prueba 1: Red Definida por Software con Valores Predeterminados	84
4.4. Prueba 2: Red Definida por Software con la Inhabilitación Programada de Dos Enlaces.....	87
4.5. Prueba 3: Red Definida por Software con la Asignación de Prioridades a los Switches	91
CAPÍTULO 5.- CONCLUSIONES Y RECOMENDACIONES	96
5.1. Conclusiones.....	96
5.2. Recomendaciones	97
5.3. Futuras Líneas de Trabajo.....	98
REFERENCIAS BIBLIOGRÁFICAS	99
ANEXOS	101
ANEXO A: Programas necesarios previos a la instalación del emulador y controlador.....	101
ANEXO B: Comandos básicos del emulador Mininet	102
ANEXO C: Captura de los tiempos de respuesta de las pruebas realizadas.....	107
ANEXO D: Analizador de protocolos Wireshark	109

ÍNDICE DE ABREVIACIONES

ACL	- Access Control List (Lista de Control de Acceso)
API	- Application Programming Interface (Interfaz de Programación de Aplicaciones)
ARP	- Address Resolution Protocol (Protocolo de Resolución de Direcciones)
BGP	- Border Gateway Protocol (Protocolo de Compuerta de Frontera)
BIOS	- Basic Input Output System (Sistema Básico de Entrada y Salida)
BPDU	- Bridge Protocol Data Units (Unidad de Datos de Protocolo Puente)
CAPEX	- Capital Expenditures (Inversiones de Capital)
CCNA	- Cisco Certified Network Associate (Asociado en Redes con Certificación Cisco)
IANA	- Internet Assigned Numbers Authority (Autoridad de Asignación de Números en Internet)
ICMP	- Internet Control Message Protocol (Protocolo de Mensajes de Control de Internet)
IEEE	- Institute of Electrical and Electronics Engineers (Instituto de Ingeniería Eléctrica y Electrónica)
IP	- Internet Protocol (Protocolo de Internet)
LLDP	- Link Layer Discovery Protocol (Protocolo de Descubrimiento de Capa de Enlace)
MAC	- Media Access Control (Control de Acceso al Medio)
NAT	- Network Address Translation (Traducción de Direcciones de Red)
NOC	- Network Operations Center (Centro de Operaciones de Red)
ONF	- Open Networking Foundation (Fundación de Interconexión Abierta)
OPEX	- Operating Expense (Gastos de Operación)
OSI	- Open System Interconnection (Interconexión de Sistemas Abiertos)
QoS	- Quality of Service (Calidad de Servicio)
SDN	- Software Defined Networking (Redes Definidas por Software)
SSH	- Secure Shell (Intérprete de Órdenes Seguro)
TCP	- Transmission Control Protocol (Protocolo de Control de Transmisión)
TLS	- Transport Layer Security (Seguridad de la Capa de Transporte)
UDP	- User Datagram Protocol (Protocolo de Datagrama de Usuario)
USB	- Universal Serial Bus (Bus Universal en Serie)
VLAN	- Virtual Local Area Network (Red de Área Local Virtual)

Capítulo 1.- INTRODUCCIÓN

En este capítulo se expondrá una breve introducción sobre la tecnología de redes definidas por software que se presenta como una alternativa en la administración de las redes de datos tradicionales, también se describirán las problemáticas, los objetivos, la justificación, alcances y limitaciones del proyecto.

1.1. Antecedentes

El desarrollo tecnológico de los dispositivos electrónicos que se utilizan día a día parece no tener límites, y se puede notar que en la mayoría de los casos están siendo diseñados para estar conectados a una red de datos para obtener su máxima utilidad, lo cual genera una significativa demanda de ancho de banda que repercute en la administración de las redes de datos, impulsándolos a proporcionar mejoras en cuanto a la velocidad, confiabilidad, disponibilidad, integridad y seguridad, además de disponer de dispositivos de interconexión que puedan direccionar de manera eficiente este tráfico de datos que va creciendo de forma exponencial.

La forma tradicional con la cual se manejan estos dispositivos de interconexión es a través de la configuración previa que se le proporciona individualmente a cada dispositivo para que opere según las reglas de los administradores de la red, y si en algún momento se requiere realizar algún cambio o agregar un nuevo servicio se lo debe hacer configurando nuevamente cada dispositivo de interconexión, empleando un tiempo considerable y más aún si la red es grande.

Con el interés de facilitar la administración en las redes de datos tradicionales nace la idea de implementar la tecnología conocida como redes definidas por software, que hace referencia a una arquitectura de red que se virtualiza, centralizando la administración de la infraestructura física de la red y agilizando las tareas de gestión mediante el desarrollo personalizado de las aplicaciones y servicios dentro de la empresa.

Basándose en el trabajo previo de los investigadores de las Universidades de Stanford y Berkeley, el año 2011 las empresas: Deutsche Telekom, Facebook, Google, Microsoft, Verizon y Yahoo! crearon la Fundación de Interconexión Abierta del inglés ONF - Open Networking Foundation, una organización sin ánimo de lucro dedicada a la promoción y adopción de las redes definidas por software a través del desarrollo de estándares abiertos, la cual define a una red definida por software formalmente como:

“Una red definida por software es una arquitectura emergente que es dinámica, administrable, rentable, y adaptable, lo cual la hace ideal para naturaleza dinámica y de alto consumo de ancho de banda de las aplicaciones de hoy en día. Esta arquitectura separa las funciones de control y de envío de paquetes de la red, permitiendo que el control de la red sea directamente programable y que la infraestructura subyacente pueda ser abstraída para las aplicaciones y los servicios de red ” (Open Networking Foundation, 2018).

Entonces, la idea fundamental de una arquitectura de red definida por software consiste en separar el plano de control del plano de datos en los dispositivos de interconexión (routers y switches), unificando los últimos mediante una interfaz de comunicación y delegando las capacidades de control a un elemento externo de la topología de red llamado controlador SDN, el cual interactúa con las aplicaciones desarrolladas para realizar tareas específicas en toda la red.

En consecuencia, la implementación de una red definida por software llega a ser un aporte y una alternativa en la administración de las redes de datos, facilitando su gestión mediante la programación de aplicaciones y servicios específicos de forma centralizada, que son desarrolladas y desplegadas en tiempos muy cortos con relación al software rígido y privativo que proporcionan los fabricantes de dispositivos de interconexión de manera genérica.

Cabe mencionar que esta tecnología está siendo implementada a nivel mundial por grandes empresas, desarrollada por universidades e instituciones tecnológicas obteniendo resultados muy favorables y convincentes en distintas áreas.

En nuestra región, existen varios proyectos universitarios a nivel de licenciatura y maestría, con la implementación de esta tecnología de los cuales se citarán dos en especial, ya que fueron un aporte para la realización del presente trabajo: “Modelado y validación formal de una topología SDN/OpenFlow” de Colombia y “Balanceo de carga en redes IPv4, un enfoque de Redes Definidas por Software” de Uruguay.

Luego de realizar una búsqueda en varias bibliotecas y en los repositorios de varias universidades del país, no se encontró ningún trabajo relacionado con esta tecnología ni en la U.M.S.A. ni tampoco a nivel Bolivia, por lo que engrandece el interés de implementar esta tecnología y demostrar su funcionalidad por lo menos en un ambiente virtual.

1.2. Problemática

Debido al incremento exponencial del tráfico de datos que se tiene actualmente con la conexión de todo tipo de dispositivos a internet, se genera una saturación en la red sobre todo en horas pico o cuando existen eventos en los que se reúne mucha gente, en consecuencia, mientras la cantidad de usuarios que se conectan a una red va en aumento dentro de un área determinada, los enrutamientos del tráfico de datos en los dispositivos de interconexión se vuelven esenciales y se necesita un despliegue de servicios dinámico y flexible, pero lamentablemente se tiene el problema de que en las redes tradicionales, no se permite programar, implementar y ejecutar de forma inmediata nuevas tareas o servicios que ayuden en la administración de la red sin interrumpir los servicios que se están ejecutando en ese momento.

La gestión actual de la red se convierte en una tarea compleja para los administradores, teniendo que lidiar con varios inconvenientes, tales como:

- Lento aprovisionamiento de nuevos usuarios.
- Aumento en la complejidad de la red.
- Saturación en las conexiones de la red.
- Adopción lenta de nuevas tecnologías.
- Retraso en el despliegue de nuevos servicios.
- Alto costo de inversión en equipamiento o capital (Capex).
- Alto costo de operación y mantenimiento de la red (Opex).
- Bajos niveles de calidad de servicio de la red.
- Bajos niveles de seguridad en la red.
- Incompatibilidad tecnológica de dispositivos.

Estos problemas que se presentan en la administración de las redes tradicionales, pueden ser mejoradas mediante la implementación de las redes definida por software, ya que los servicios son personalizados y desarrollados de acuerdo a cada lugar, optimizando considerablemente la administración de la red, además de que la implementación y mantenimiento puede ser completamente automatizadas mediante software libre, aplicado a equipos de interconexión de diferentes fabricantes dejando a un lado la incompatibilidad tecnológica, reduciendo de esta forma el Opex y Capex respectivamente.

1.3. Objetivos del Proyecto

1.3.1. Objetivo General

Diseñar y simular una arquitectura de red definida por software, en base al protocolo OpenFlow, para comprender y verificar su funcionamiento.

1.3.2. Objetivos Específicos

- Describir los conceptos fundamentales sobre redes definidas por software.
- Caracterizar los elementos principales de la arquitectura de red definida por software.
- Determinar las herramientas de hardware y software apropiadas, para la implementación de la arquitectura de red definida por software.
- Realizar el diseño y desarrollo del script que emula la red virtual, y de la aplicación que se ejecuta en el controlador de la red definida por software.
- Verificar el funcionamiento de la arquitectura de red definida por software mediante la experimentación de diferentes pruebas.

1.4. Justificación

Ya que esta nueva tecnología, conocida como redes definidas por software está siendo estudiada y adoptada de a poco por grandes empresas e instituciones a nivel internacional, lo que se realizará en el presente trabajo es dar a conocer los conceptos fundamentales para comprender su funcionamiento y verificar los beneficios que presenta, para que se pueda implementar en la administración de servicios en una red de datos, llevándolo a cabo en un ambiente de experimentación virtual creado específicamente para este tipo de redes.

Con la implementación de una arquitectura de una red definida por software, se verifica que existe una mejora significativa en la administración de las redes, resolviendo de manera eficiente y eficaz las problemáticas que se presentan en las redes de datos tradicionales descritas anteriormente. De manera que se pueden alcanzar los siguientes beneficios:

- Una visión unificada y administración centralizada de la infraestructura de red.
- Aprovisionamiento acelerado de nuevos usuarios y servicios.
- Una red ágil y flexible que puede adaptarse automáticamente a los servicios demandados por los usuarios.

- Mayor rapidez para la innovación e implementación de reglas, debido a que los cambios son realizados mediante software y no en el hardware.
- Mayor eficiencia y altos niveles de utilización de los recursos de la red, a través del uso de algoritmos definidos por los programadores de la red.
- Independencia de la marca del fabricante en los dispositivos de red, dado que la inteligencia de la red está centralizada en el controlador y los dispositivos de red son solamente manejadores de los flujos de datos.
- Un entorno de prueba de alta fidelidad con una emulación que puede recrearse completamente mediante software.
- Reducción en los costos de inversión en equipamiento y operación de la red.
- Fortalecimiento de la calidad de servicio y seguridad de la red.

1.5. Alcances del Proyecto

En el presente trabajo se describirá la teoría fundamental sobre el funcionamiento de la arquitectura de red definida por software que se implementará.

Se caracterizará de forma puntual el protocolo OpenFlow, para la comunicación entre los dispositivos de interconexión emulados y el controlador de la red.

Para la implementación de la arquitectura de red definida por software, solo se utilizará el controlador Ryu, ya que cumple con las necesidades y requerimientos del proyecto.

La topología de red que se implementará en la arquitectura de red definida por software, será desarrollada como un script para el emulador Mininet.

Con el fin de automatizar el funcionamiento de la red, se desarrollará y se implementará el protocolo de árbol de expansión, como una aplicación para el controlador SDN, con el cual se gestionarán los caminos redundantes en la topología de la red.

Se comprobará el cumplimiento de los protocolos de comunicación entre los dispositivos de interconexión virtuales y el controlador SDN, mediante el analizador de protocolos Wireshark.

Se analizará detalladamente el estado y comportamiento de los dispositivos virtuales en cada etapa de la simulación.

Se realizará el registro de los tiempos de respuesta en la comunicación entre los hosts de la red con tres pruebas diferentes, con el fin de analizar y comparar los resultados obtenidos en la topología de red virtual.

1.6. Limitaciones del Proyecto

La arquitectura de red definida por software, será implementada utilizando software libre en su totalidad.

En cuanto a la comunicación entre los dispositivos de interconexión virtuales y el controlador SDN, solo se utilizará el protocolo OpenFlow, por ser el más eficaz en este tipo de redes, además de disponer de suficiente documentación para desarrollar el presente trabajo.

La arquitectura de red definida por software será controlada solamente por el controlador Ryu, ya que cumple con las necesidades requeridas del proyecto.

Toda la implementación virtual de la arquitectura de red definida por software, se realizará en una notebook que cuenta con un hardware limitado, por lo que puede presentar una pérdida de fidelidad en los resultados y en el rendimiento al momento de emular muchos dispositivos virtuales y grandes cargas.

Al momento del estudio, se evidenció que la mayor parte de los proveedores de dispositivos de red en nuestro medio, tienen poco conocimiento sobre esta tecnología y la adquisición de dispositivos compatibles solo se lo realiza ha pedido con costos muy variables, por tal razón dentro del presente trabajo no se realizará ningún estudio financiero para la implementación con dispositivos de interconexión reales.

Capítulo 2.- MARCO TEÓRICO

En este capítulo se realiza la comparación de las redes definidas por software con las redes tradicionales, luego se describe el funcionamiento de las redes definidas por software y los elementos básicos que lo componen, haciendo énfasis en la caracterización del protocolo OpenFlow y del controlador Ryu dentro de la arquitectura SDN.

2.1. Comparativa de las Redes Definidas por Software con las Redes Tradicionales

Las redes tradicionales están compuestas por dispositivos con sistemas operativos cerrados y desarrollados por el propio fabricante lo que obliga a los administradores de red a solo depender de las funcionalidades ofrecidas por dichos dispositivos, mientras que las SDN's pueden ser programadas de acuerdo a normativas, requerimientos y necesidades de la empresa.

Cuando un paquete llega a un switch en una red convencional, las reglas integradas en el firmware propietario del switch le dicen al dispositivo a donde transferir el paquete, el switch envía cada paquete al mismo destino por la misma trayectoria y trata a todos los paquetes de la misma manera, mientras que en una red definida por software, un administrador de red puede darle forma al tráfico desde una consola de control centralizada o automatizarlo mediante un programa, puede cambiar cualquier regla en los dispositivos de la red cuando sea necesario, asignando o quitando prioridades, incluso bloqueando tipos específicos de paquetes direccionándolos a un análisis riguroso, teniendo así un alto nivel de control, algo que no es posible en el entorno de una red tradicional como el de la Figura 1.

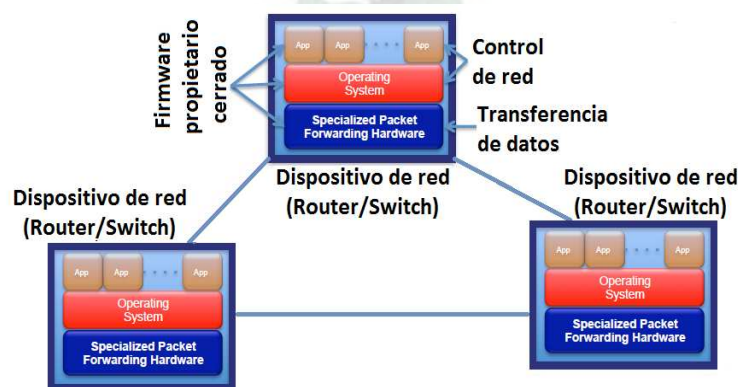


Figura 1. Arquitectura de una red tradicional.

Fuente: Dan Savu, Stefan Stancu. (2014). *Software Defined Networking, technology details and openlab research overview*.

En las redes tradicionales el control es descentralizado, el envío de paquetes y las decisiones de enrutamiento de alto nivel suceden dentro del mismo dispositivo de interconexión individualmente

como se puede ver en la Figura 1, mientras que en las redes definidas por software estas dos funcionalidades (envío de paquetes y decisiones de enrutamiento) se separan permitiendo tener un control centralizado como se muestra en la Figura 2.

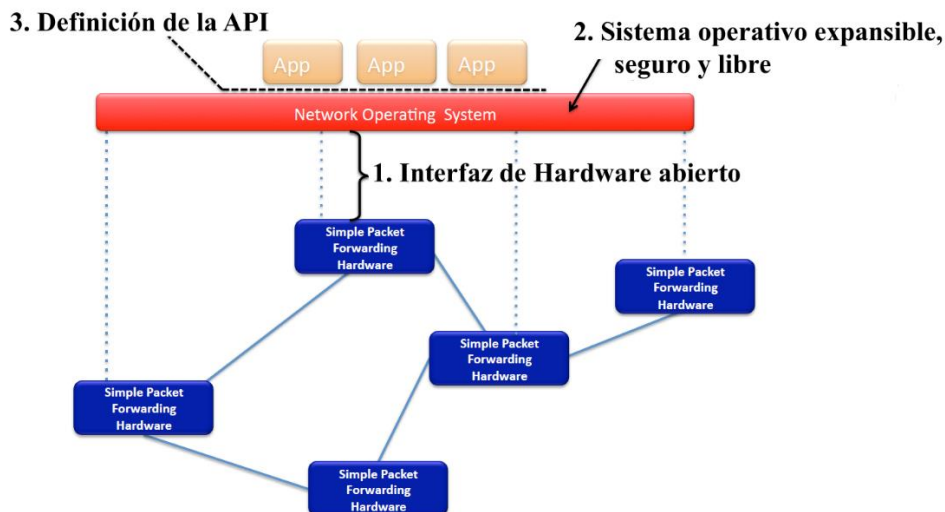


Figura 2. Arquitectura de una SDN.

Fuente: Dan Savu, Stefan Stancu. (2014). *Software Defined Networking, technology details and openlab research overview*.

Con las redes tradicionales las tareas de configuración y mantenimiento son complejas ya que el administrador de la red accede mediante una interfaz de línea de comandos a los dispositivos de red con un consumo de tiempo y verificación considerable, y más aún en el caso de trabajar con dispositivos antiguos, incompatibles o de distintos fabricantes. De lo dicho anteriormente, en la Figura 3, se hace evidente que el control centralizado programable es el aspecto más importante de una red definida por software y que es aprovechado al máximo por los administradores.

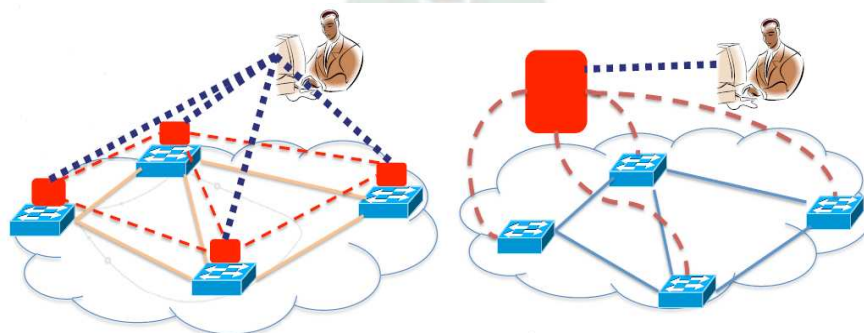


Figura 3. Comparación en la administración de una red tradicional y una SDN.

Fuente: Dan Savu, Stefan Stancu. (2014). *Software Defined Networking, technology details and openlab research overview*.

En la Tabla 1, se presenta una comparación de las características principales de las redes definidas por software y las redes tradicionales.

Tabla 1. Comparación de una SDN y una red tradicional.

SDN	RED TRADICIONAL
Es dinámica.	Relativamente estática y difícil de aplicar nuevas políticas.
Es flexible	No es flexible.
Las tablas de flujos están abiertas.	Las tablas de flujos están cerradas.
Nuevas aplicaciones en menor tiempo.	Nuevas aplicaciones en mayor tiempo.
Configuración centralizada mediante programación en el software del controlador.	Configuración descentralizada.
API's definidas por el programador.	Interfaces propietarias.
Formato y acciones de las tablas de flujos claramente especificadas.	Envío de paquetes basados en coincidencias de las tablas.
Automatización según la configuración en el software del controlador.	Automatización posible pero tediosa.
API's abiertas para el acceso y manipulación del plano de datos de uno o de todos los dispositivos.	Configuración individual de los dispositivos.
Facilita el desarrollo y la innovación.	Ralentiza la investigación y desarrollo.
Reducción de costos operativos y de inversión.	Altos costos operativos y de inversión.

Nota: Elaboración propia recopilada de diversas fuentes.

2.2. Lógica de Funcionamiento de las Redes Definidas por Software

Las redes tradicionales no son lo suficientemente ágiles como para reprogramarse y reajustarse al despliegue de nuevos servicios, en consecuencia, lo que se persigue con la arquitectura de una red definida por software es acercarse a las necesidades actuales de flexibilidad, escalabilidad, automatización y gestión de la red.

La idea principal de una red definida por software reside en el hecho de gestionar las redes de datos separando el plano de control (software de control y protocolos de enrutamiento) del plano de datos (hardware que se encarga de la conmutación de paquetes) en los dispositivos de interconexión (routers y switches).

Una vez obtenido el plano de datos de cada dispositivo, estos se conectan a través de un protocolo de comunicación hacia el plano de control, que generalmente es un dispositivo exterior que contiene un software llamado controlador SDN, el cuál es el encargado de gestionar toda la red de manera centralizada.

Esta centralización de los dispositivos de interconexión mediante el plano de control, permite a los administradores de red aplicar políticas dinámicas desde una perspectiva más alejada y global,

y sin la complicación de tener que configurar cada dispositivo de red de forma individual, sino que es el propio controlador quien se encarga de instalar las reglas necesarias en cada uno de éstos de forma automática y dinámica.

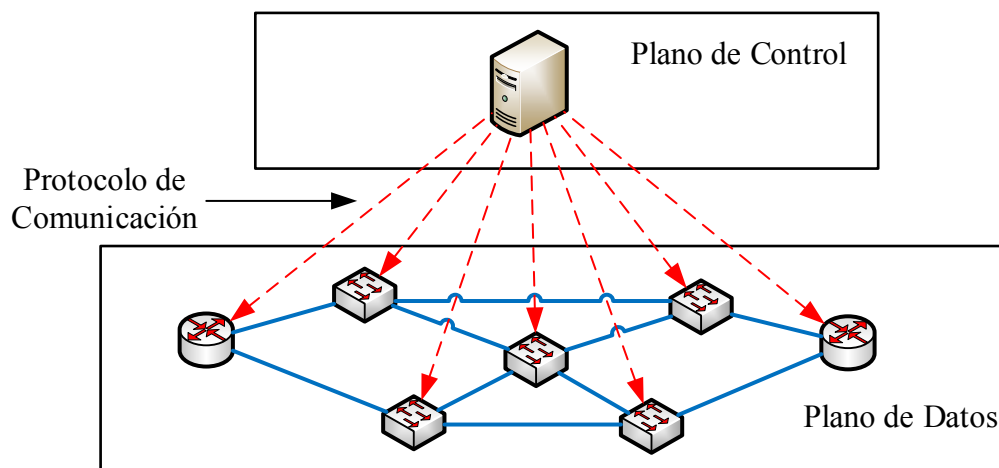


Figura 4. Lógica de funcionamiento de una SDN.
Fuente: Elaboración propia recopilada de diversas fuentes.

2.3. Arquitectura y Elementos que Conforman las Redes Definidas por Software

Según la ONF una arquitectura de red definida por software está formada por tres capas que son accesibles desde las interfaces de programación de aplicaciones o API's.

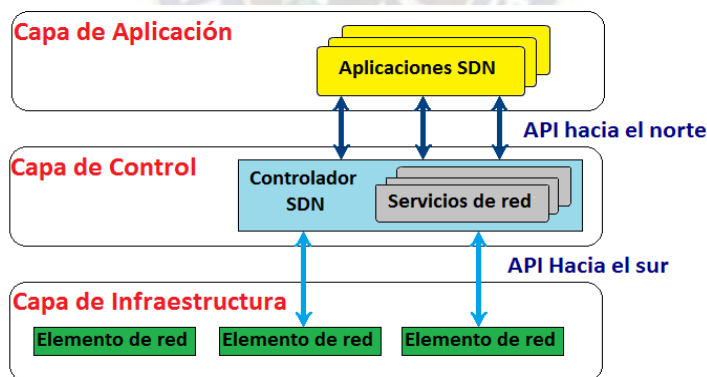


Figura 5. Arquitectura de una Red Definida por Software.
Fuente: ONF TR-521. (2016). *SDN architecture - Open Networking Foundation*.

A continuación, se describirán los elementos que conforman la arquitectura SDN.

2.3.1. Capa de Aplicación

Es el nivel más alto desde donde se programa y automatiza el comportamiento de la red, consiste en las aplicaciones destinadas a satisfacer las necesidades de los usuarios finales que utilizan los servicios de comunicaciones SDN a través de la API hacia el norte.

En cuanto a los lenguajes de programación utilizados para desarrollar las aplicaciones se tienen varias opciones como ser: Python, C++, Java, Flowbased Management Language (FML), Nettle, NetCore, Pyretic, Frenetic, entre otros.

También se cuenta con librerías en un Conjunto de Desarrollo de Software (SDK - Software Development Kit) como: OnePK (Cisco One Platform Kit), Junos Space SDK, HP SDN SDK y OpenNSL (Open Network Switch Layer) entre los más conocidos.

2.3.2. Capa de Control

Dentro de la capa de control se tiene el elemento llamado controlador SDN que es el más importante dentro de una arquitectura SDN, y es desde aquí donde se proporciona una función de control y gestión global de la red, configurando todos los nodos activos para dirigir correctamente el tráfico de paquetes, gracias a la comunicación que se tiene con la capa de aplicación y con la capa de infraestructura mediante sus respectivas API's. La arquitectura permite que el controlador SDN gestione un amplio rango de recursos sobre el plano de datos en los dispositivos de interconexión, unificando y simplificando su configuración.

2.3.3. Capa de Infraestructura

Está constituida por los dispositivos físicos o virtuales que se encuentran en los nodos de la red realizando la conmutación y encaminamiento de los paquetes, sin la necesidad de utilizar el firmware de control que tiene cada dispositivo, pero con el requisito que soporten un protocolo de comunicación común, el cual les permitirá comunicarse con el controlador SDN.

2.4. El Controlador SDN

El controlador SDN es el cerebro en una arquitectura SDN, algunos autores le llaman el Sistema Operativo de la red. Es donde se centraliza la inteligencia de la red y sus tareas básicas incluyen el descubrimiento de los dispositivos, proveer una visión general de la topología de red, distribuir la configuración y políticas definidas por el administrador hacia los elementos de red, desde aquí se gestiona la lógica de funcionamiento de toda la red.

El controlador actúa como el cerebro de un dispositivo al asumir toda la responsabilidad del plano de control, tomando las decisiones de conmutación como en las capas 2, 3 y 4 del modelo OSI (Interconexión de Sistemas Abiertos), extendiendo las posibilidades de ejecutar varios módulos funcionales configurados para trabajar en conjunto de manera colaborativa entre ellos.

Estas características buscan administrar una cantidad elevada de solicitudes y permitir la tolerancia a fallos mediante la utilización de múltiples controladores en una infraestructura grande, aprovechando la ventaja de no contar con controlador estándar en las SDN's.

2.4.1. Funcionamiento del Controlador SDN

El funcionamiento del controlador SDN se basa en gestionar un conjunto de módulos de manera automática y centralizada, para lo cual se comunica en primera instancia con la capa de aplicaciones mediante la API hacia el norte para obtener las políticas de administración que son programadas en las aplicaciones, luego de obtener dichas políticas estas son implantadas en los elementos de red mediante la API hacia el sur, en el caso de existir más de un controlador en una arquitectura SDN estos se comunican entre ellos mediante las llamadas API's hacia el oeste y el este como se verá más adelante. La descripción de este funcionamiento es de forma genérica y se puede especificar según la funcionalidad que este ofrece.

Para comprender el funcionamiento del controlador SDN de forma específica, se describirá el proceso que se tiene cuando éste interactúa con un switch.

Una vez que se tiene el ingreso de un nuevo paquete o flujo en el switch, éste envía al controlador una petición para el análisis de una nueva entrada de flujo, entonces el controlador recibe el mensaje de ingreso de un nuevo paquete desde el switch y lo primero que hace es verificar si la entrada de flujo es de tipo Ethernet analizando la cabecera del paquete y si no es de tipo Ethernet envía un mensaje al switch para descartarlo, luego se obtiene los puertos de entrada/salida, las direcciones MAC origen/destino y las direcciones IP origen/destino como funciones básicas, con alguno de estos datos se consigue identificar la fuente de origen y destino del paquete, y en caso de no ser identificado se envía a la capa de aplicaciones para ver cómo será tratado dicho paquete según las aplicaciones o políticas de administración, una vez que el paquete es identificado se le aplica una determinada regla y el controlador envía un mensaje al switch con las acciones correspondientes para que este pueda procesar la entrada de flujo.

En la Figura 6 se muestra el diagrama de flujo que corresponde al funcionamiento básico del controlador SDN, desde que recibe el mensaje de ingreso de un nuevo paquete del switch hasta devolverlo para su respectivo proceso.

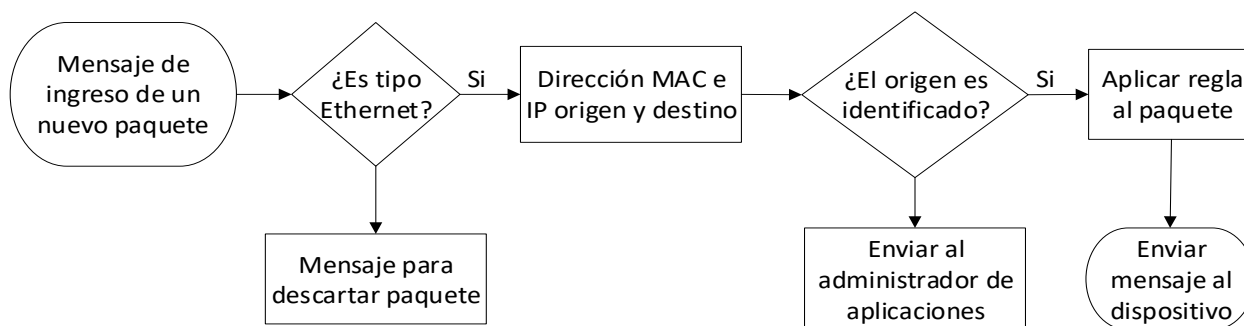


Figura 6. Funcionamiento del controlador SDN.
Fuente: Elaboración propia recopilada de diversas fuentes.

Una vez descrito el funcionamiento del controlador SDN, ahora se verán los diferentes tipos de configuración que se pueden presentar con los controladores, atendiendo a sus características:

Según su disposición: Se pueden distinguir tres tipos de configuraciones, una centralizada donde un solo controlador gestiona todos los equipos, otra distribuida donde hay un controlador para cada dispositivo o un conjunto de dispositivos y la última que se refiere a un control jerárquico donde existe un grupo de controladores que pueden llegar a ser controlados por un controlador superior a ellos, la Figura 7, muestra estas disposiciones.

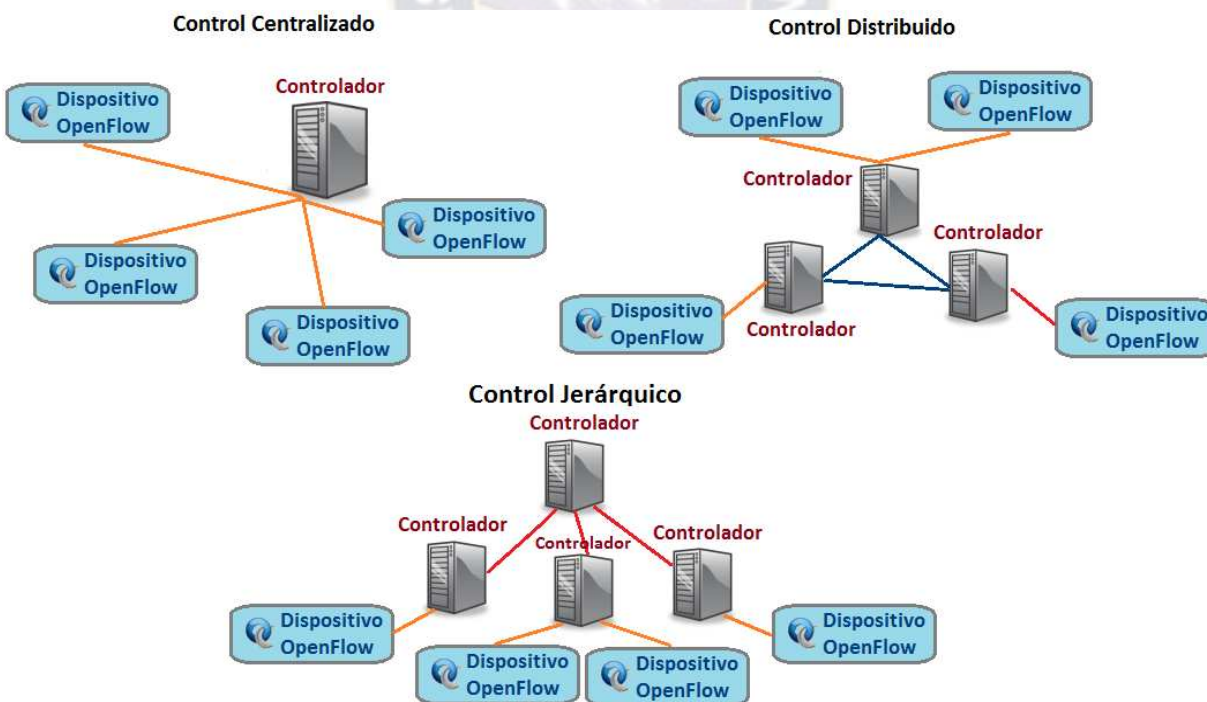


Figura 7. Tipos de control según su disposición.
Fuente: Elaboración propia recopilada de diversas fuentes.

Según el tipo de flujo: Se pueden distinguir dos: enrutamiento por flujo, donde cada entrada de flujo define un camino determinado; agregado, donde cada entrada de flujo en la tabla de flujos

corresponde a una categoría de flujos (estas entradas se configuran comúnmente mediante wildcards¹). El primer tipo sería adecuado para redes pequeñas tipo campus, y el segundo para redes con múltiples flujos tipo backbone.

Según su comportamiento: Existen dos tipos: reactivo, donde el primer paquete de un flujo desencadena que el controlador inserte las entradas de flujos en las tablas de flujos, cada flujo tiene un tiempo de conformación la primera vez que aparece en la red y si la conectividad con el controlador decae la existencia del paquete dependerá de su tiempo de vida; proactivo, donde el controlador completa las tablas de flujos a priori antes de que lleguen al switch, en este tipo de comportamiento no hay tiempo de establecimiento de flujo y la pérdida momentánea de conectividad del dispositivo de red con el controlador no afecta al funcionamiento.

2.4.2. Interfaz de Programación de Aplicaciones (API)

En esta sección se describen las API's con las que trabaja el controlador, las cuales hacen que se tenga una comunicación con los diferentes componentes de la red. La Figura 8, muestra las diferentes interfaces con las cuales puede trabajar un controlador SDN.

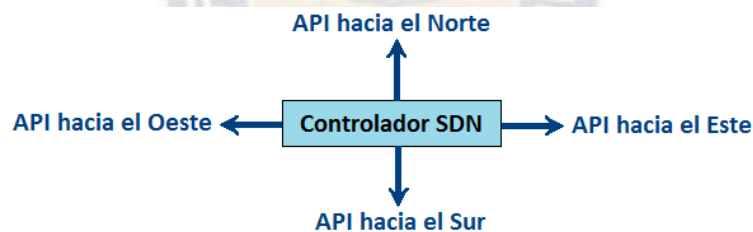


Figura 8. API's de un controlador SDN.

Fuente: Elaboración propia.

API hacia el Norte: Se comunica con las aplicaciones SDN mediante la traducción de sus requerimientos en un lenguaje de programación de alto nivel, proporcionando una información relevante a las aplicaciones, la cual servirá para la generación de nuevas instrucciones que serán encargadas al controlador SDN para que este los distribuya a los elementos de red.

En estas interfaces, no existe todavía una estandarización y es posible que nunca exista un estándar para estas API's, ya que son el punto de contacto con el exterior y las necesidades de seguridad, políticas de uso y acceso son variadas en distintas redes o lugares.

¹ Wildcards, o comodines son mascaras formadas por 32 bits que indican al dispositivo de red que parte de una dirección de red debe coincidir para llevar a cabo una determinada acción.

API hacia el Sur: Se comunica con la capa de infraestructura, recogiendo el estado global de la red, inventariando a todos los dispositivos de red, colecta capacidades de los dispositivos, agrupa eventos, realiza estadísticas de la red y se mandan a ejecutar las reglas que define el controlador o las aplicaciones, realizando cambios dinámicos de acuerdo a las demandas y necesidades en tiempo real. Mediante esta API se dirige el plano de datos de todos los dispositivos de interconexión compatibles con el protocolo de comunicación utilizado.

Existe una diversidad de API's hacia el sur como: NETCONF, OF-config, OVSDDB, POF, Opflex, OpenState, ROFL, HAL, entre otros, pero es aquí donde el protocolo OpenFlow es el máximo referente de la industria convirtiéndose en un estándar de facto para establecer el uso general del mismo en redes definidas por software y con la ventaja que es soportada por la mayoría de los controladores SDN, y es por eso que en el presente proyecto se eligió este protocolo y se lo describirá a detalle en la sección 2.5 de éste capítulo.

API hacia el Oeste y API hacia el Este: Son interfaces pensadas para la comunicación de múltiples controladores de un mismo proveedor de red, generalmente para hacer posible la distribución de la lógica del sistema de manera horizontal. Estas interfaces son propias y específicas del controlador que se utiliza y no todos los controladores ofrecen soporte para arquitecturas de control distribuidas.

2.4.3. Tipos de Controladores SDN

En cuanto a los tipos de controladores SDN existen diferentes opciones, implementando diferentes funcionalidades para ser desplegadas en ambientes virtuales o reales, en la actualidad los controladores SDN más importantes son: NOX, el controlador con el que nació OpenFlow escrito en C++ y centrado en ofrecer altos rendimientos; POX, se desarrolló a partir de NOX escrito en Python y centrado en el desarrollo rápido de módulos aprovechado ampliamente para investigación y educación; OpenDaylight (ODL), surgido como un proyecto de la fundación Linux, escrito en Java, es un controlador multipropósito muy usado en la industria por ser robusto; Floodlight, escrito en Java, con soporte para distribuir la lógica en múltiples controladores, presenta una documentación y comunidad con soporte regular; Ryu, está escrito en Python, dispone de un conjunto de API's bien definidas y un gran número de módulos que pueden ser utilizados según las necesidades de la organización en el desarrollo de nuevas aplicaciones para la gestión de redes, también cuenta con soporte para varios protocolos que gestionan los dispositivos

de red como: OpenFlow, NETCONF y OF-config, una de las ventajas importantes sobre los otros controladores mencionados es que soporta el protocolo OpenFlow desde la versión 1.0 hasta la versión 1.5. La Tabla 2 muestra un resumen de las características que ofrecen los controladores mencionados.

Tabla 2. *Controladores SDN de código abierto.*

CARACTERISTICAS	CONTROLADORES				
	NOX	POX	ODL	FLOODLIGHT	RYU
Soporte OpenFlow	v1.0/1.3	v1.0	v1.0/1.4	v1.0/1.4	v1.0/1.5
Lenguaje de desarrollo	C++	Python	Java	Java	Python
Lenguajes soportados por el controlador	C, C++, Python	Python	Java	Java, Python	Python
Soporte	Malo	Bueno	Bueno	Regular	Bueno
Módulos para el desarrollo de aplicaciones	Malo	Bueno	Regular	Bueno	Bueno
Provee REST API	No	No	Si	Si	Si
Interfaz gráfica	Python+ QT4	Python+ QT4, web	Web	Java, Web	Web
Soporta bucles en la topología	No	No	Si	Si	Si
Multiprocesos	Si	No	Si	Si	No
Soporte de plataforma	Linux	Linux, Windows, MacOS	Linux, Windows, MacOS	Linux, Windows, MacOS	Linux
Velocidad de procesamiento	Lenta	Rápido	Media	Rápido	Media
Documentación	Media	Pobre	Pobre	Media	Buena

Nota: Elaboración propia recopilada de diversas fuentes.

Los controladores expuestos en la Tabla 2 son de código abierto, pero también existen controladores comerciales que son estrictamente basados en software como propone el enfoque SDN, como: Cisco Open SDN, HP VAN SDN, ProgrammableFlow PF6800, NSX. Donde la principal ventaja de estos frente a los controladores de código abierto es el soporte que ofrecen.

2.4.4. Elección del Controlador SDN

Ya que la elección del controlador es de vital importancia, a continuación, se expondrán varios factores que ayudan a determinan la elección correcta de un controlador SDN.

Soporte OpenFlow: Los administradores de la red necesitan conocer las características del protocolo OpenFlow en sus diferentes versiones, ya que no todos los controladores soportan todas

las versiones del protocolo OpenFlow y su elección puede ser de vital importancia. Una razón por lo que esto es necesario es que algunas funciones importantes como, por ejemplo, el soporte de IPv6 no es parte de OpenFlow v1.0, pues recién se incluye a partir de OpenFlow v1.2.

Virtualización de red: Debido a los beneficios que ofrece la virtualización de red, un controlador SDN debe soportarla. Esta característica permite a los administradores crear dinámicamente las redes virtuales, disociadas de las redes físicas, para satisfacer una amplia gama de requisitos y permitir un aislamiento entre cada segmento de red.

Funcionalidad de la red: Para alcanzar un buen funcionamiento de la red mediante el enrutamiento de los flujos, es importante que el controlador SDN pueda tomar decisiones de enrutamiento basado en múltiples campos de la cabecera de OpenFlow, también es importante que el controlador pueda definir los parámetros de calidad de servicio flujo por flujo.

Escalabilidad: Una consideración fundamental con respecto a la escalabilidad de una red definida por software es el número de dispositivos que un controlador SDN puede soportar, se debe esperar que el controlador soporte al menos de 100 dispositivos. Otro aspecto de la escalabilidad es la capacidad del controlador SDN para crear y gestionar múltiples sitios reales y virtuales, el controlador SDN debe permitir que las políticas de red para el enrutamiento y reenvío se apliquen automáticamente en la migración de servidores y/o almacenamiento.

Rendimiento: Una de las principales funciones de un controlador SDN es establecer flujos, por ello, dos de los indicadores claves de rendimiento asociados con un controlador SDN son: el tiempo de conformación del flujo y el número de flujos por segundo que puede establecer el controlador.

Documentación existente: Siendo una tecnología relativamente nueva se dio mucho valor a los controladores que tienen una documentación, soporte y comunidad de usuarios significativa, que están a disposición de manera comprensible y ordenada, para que se pueda consultar sobre los inconvenientes que se presentan en el proceso de implementación.

Para la implementación del presente proyecto se eligió el controlador Ryu, ya que es un controlador muy flexible, su código fuente está disponible para utilizarlo libremente, el tiempo de aprendizaje es de corto a mediano plazo ya que se utiliza el lenguaje Python para el desarrollo de sus aplicaciones, cumple con las condiciones de virtualización de la red, presenta una buena

funcionalidad y escalabilidad en su implementación simulada, su rendimiento depende mucho de las características del dispositivo donde se trabaje, y finalmente se evidencio que cuenta con una documentación significativa en varios sitios web, blogs y una amplia comunidad que colabora con el soporte del mismo.

2.5. OpenFlow

Para que se pueda llevar a cabo una arquitectura de red definida por software se necesitan dos requisitos fundamentales: alguien quien entienda la arquitectura lógica común en todos los dispositivos de interconexión y alguien quien pueda comunicarse de manera segura de tal forma que garantice la comunicación entre el controlador y los dispositivos de red. El protocolo OpenFlow cumple con estos requisitos y es el primer protocolo de comunicación diseñado específicamente para redes definidas por software, fue desarrollado con base a switches Ethernet y se convirtió en un estándar abierto de comunicación entre el controlador SDN y los dispositivos de interconexión que soportan esta tecnología.

Para el estudio completo de OpenFlow se comenzará con los dispositivos que soportan esta tecnología, se describirán los componentes principales que lo conforman y más adelante se expondrá las características del protocolo OpenFlow como API hacia el sur.

2.5.1. Dispositivo OpenFlow

Cuando se habla de dispositivos OpenFlow se hace referencia a 2 tipos de dispositivos: los dispositivos OpenFlow dedicados y los dispositivos híbridos.

- **Dispositivos OpenFlow dedicados:** Estos dispositivos no tienen inteligencia propia y solo soportan operaciones OpenFlow, ya que tienen el protocolo OpenFlow incorporado para la comunicación con el controlador SDN. En estos dispositivos todos los paquetes son procesados de una sola manera llamada pipeline (canalización) OpenFlow. No da soporte a las capas 2 y 3 del modelo OSI.
- **Dispositivos híbridos:** Tienen la posibilidad de trabajar con operaciones de conmutación y enrutamiento tradicionales, creación de VLAN's, ACL, QoS y otros procesos. Pero estos dispositivos también tienen que proporcionar un mecanismo de clasificación que dirija el tráfico hacia un proceso de pipeline OpenFlow, estos mecanismos se habilitan mediante una actualización del Firmware o simplemente mediante la configuración del dispositivo.

Dentro del dispositivo OpenFlow, se explota el hecho de que la mayoría de los routers y switches modernos contienen tablas de flujos (Flows Tables), típicamente construidas con Memorias Direccional de Contenido Ternario (TCAM - Ternary Content Addressable Memory).

A pesar de que cada tabla de flujos es diferente en cada dispositivo, se han encontrado un conjunto de funciones similares o genéricas que son aprovechados por OpenFlow.

En la siguiente figura se muestra el dispositivo OpenFlow y sus principales elementos:

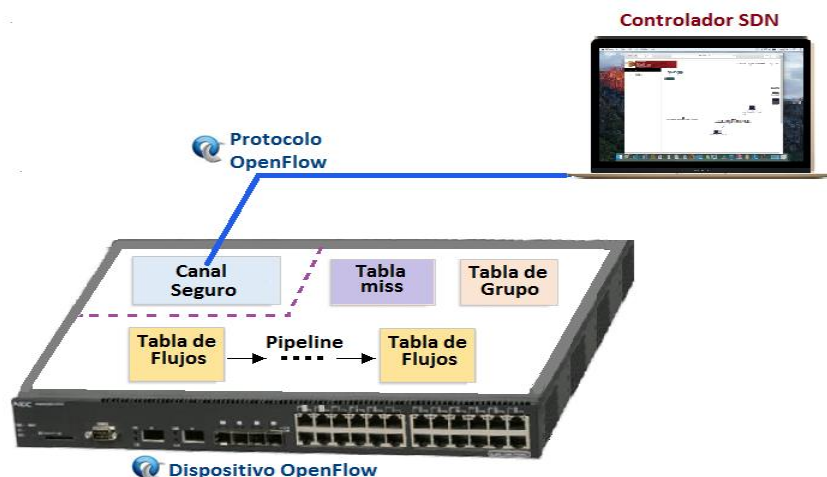


Figura 9. Dispositivo y protocolo OpenFlow.

Fuente: Elaboración propia.

2.5.1.1. Tabla de Flujos

La tabla de flujos está formada por un conjunto de entradas de flujos, y cada entrada de flujo (Flow Entry) está formado por los campos que se muestra en la Tabla 3.

Tabla 3. Elementos que conforman una entrada de flujo.

Campos de coincidencia	Prioridad	Contadores	Instrucciones	Tiempo de espera	Cookie
------------------------	-----------	------------	---------------	------------------	--------

Fuente: ONF TS-007 (2012). *OpenFlow Switch Specification - Open Networking Foundation*.

Campos de coincidencia: Realiza una comparación de coincidencia con una entrada de flujo, comprobando el puerto de entrada y la cabecera del paquete.

En la Tabla 3.1, se describen los campos del paquete que son utilizados para definir algún tipo de coincidencia después de la comparación con una determinada entrada de flujo.

Prioridad: Orden preferencial de la entrada de flujo.

Tabla 3.1. Campos que son utilizados para la comparación con una entrada de flujo.

Campo	Bits	Aplicable a	Notas
Puerto de Ingreso	(depende de la implementación)	Todos los paquetes	Representación numérica del puerto entrante.
Dirección MAC de Origen	48	Todos los paquetes en puertos habilitados	
Dirección MAC de destino	48	Todos los paquetes en puertos habilitados	
Tipo Ethernet	16	Todos los paquetes en puertos habilitados	
Id VLAN	12	Todos los puertos de tipo Ethernet 0x8100	
Prioridad VLAN	3	Todos los puertos de tipo Ethernet 0x8100	Campo PCP (Priority Code Point) de una VLAN
IP origen	32	Todos los paquetes IP y ARP	Se le puede aplicar una máscara de subred
IP destino	32	Todos los paquetes IP y ARP	Se le puede aplicar una máscara de subred
Protocolo IP	8	Todos los paquetes IP y ARP	Solo los 8 bits menos significativos del código ARP son utilizados
Bits ToS IP	6	Todos los paquetes IP	Tipo de servicio IP
Puerto TCP/UDP origen	16	Todos los paquetes TCP, UDP e ICMP	Solo son utilizados los 8 bits menos significativos del tipo ICMP
Puerto TCP/UDP destino	16	Todos los paquetes TCP, UDP e ICMP	Solo son utilizados los 8 bits menos significativos del tipo ICMP

Nota: Elaboración propia recopilada de diversas fuentes.

Contadores: Realizan un registro cuando los paquetes son coincidentes, los más relevantes son aquellos relacionados por: entrada de flujo, tabla de flujos, puerto, cola, grupo y recipiente de grupo, donde cada uno cuenta con diferentes tipos de contadores con un tamaño de 32 o 64 bits.

Instrucciones: Modifica el conjunto de acciones o proceso de pipeline a un paquete cuando el proceso de coincidencia es satisfactorio, las instrucciones más importantes son aquellas que escriben o borran acciones para cada entrada de flujo. Existen dos tipos de instrucciones:

- **Acciones Obligatorias:** Estas acciones deben estar soportadas por todos los dispositivos OpenFlow dedicados e híbridos.
- **Acciones Opcionales:** Son acciones opcionales que soporta un determinado dispositivo.

Tiempo de espera: Existen dos tipos de tiempos de espera definidos por cada entrada de flujo:

- **Tiempo de espera inactivo:** Tras la ausencia de paquetes coincidentes con dicho flujo en un intervalo de tiempo definido, el flujo es eliminado por el dispositivo.
- **Tiempo de espera máximo:** Siempre que transcurra un intervalo de tiempo determinado el flujo será eliminado de la tabla, a menos que el controlador envíe una actualización del intervalo temporal o modifique su comportamiento.

Cookie: Datos seleccionados por el controlador, pueden ser utilizados para la inserción, modificación, eliminación o realizar estadísticas de los flujos.

2.5.1.1.1. Proceso Pipeline OpenFlow

El proceso de pipeline (canalización) OpenFlow define como los paquetes interactúan con las tablas de flujos. Un dispositivo OpenFlow tiene que tener por lo menos una tabla de flujos, aunque normalmente tiene más de una tabla de flujos.

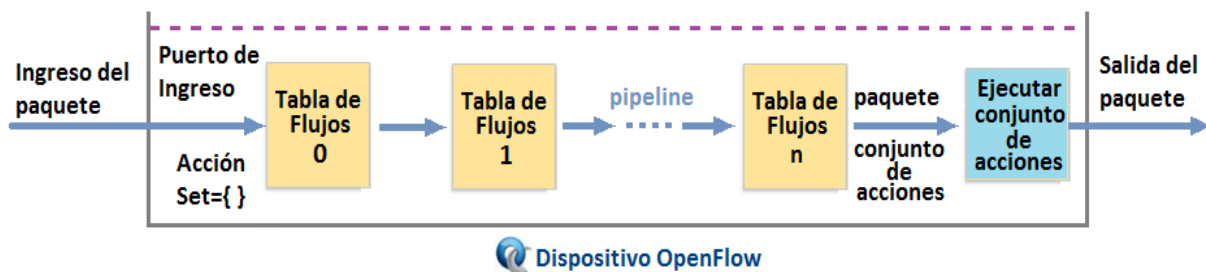


Figura 10. Proceso pipeline OpenFlow.

Fuente: Elaboración propia recopilada de diversas fuentes.

Las tablas de flujos de un dispositivo OpenFlow están numerados de manera secuencial, empezando por el 0. El proceso de pipeline siempre comienza en la primera tabla de flujos, el primer paquete se compara con las entradas de flujo de la tabla de flujos 0.

En caso de encontrar una coincidencia el conjunto de instrucciones incluidas es ejecutada en esta entrada de flujo, estas instrucciones pueden dirigir el paquete explícitamente a otra tabla de flujos siguiente o mayor a esta mediante la instrucción “Goto”, una vez que la entrada de flujo ya no tiene coincidencia en las otras tablas de flujos se detiene el proceso de pipeline en esta tabla, y el paquete es procesado con su acción asociada y es reenviada.

Si el paquete no coincide con ninguna entrada de flujo en ninguna tabla de flujos esta es llevada a la última tabla, que es la tabla miss que por defecto tiene una prioridad de 0 y sin una comparación

específica, este proceso se presenta principalmente cuando hay el ingreso de flujos nuevos, el comportamiento de esta tabla depende de la configuración que se establece previamente en el controlador.

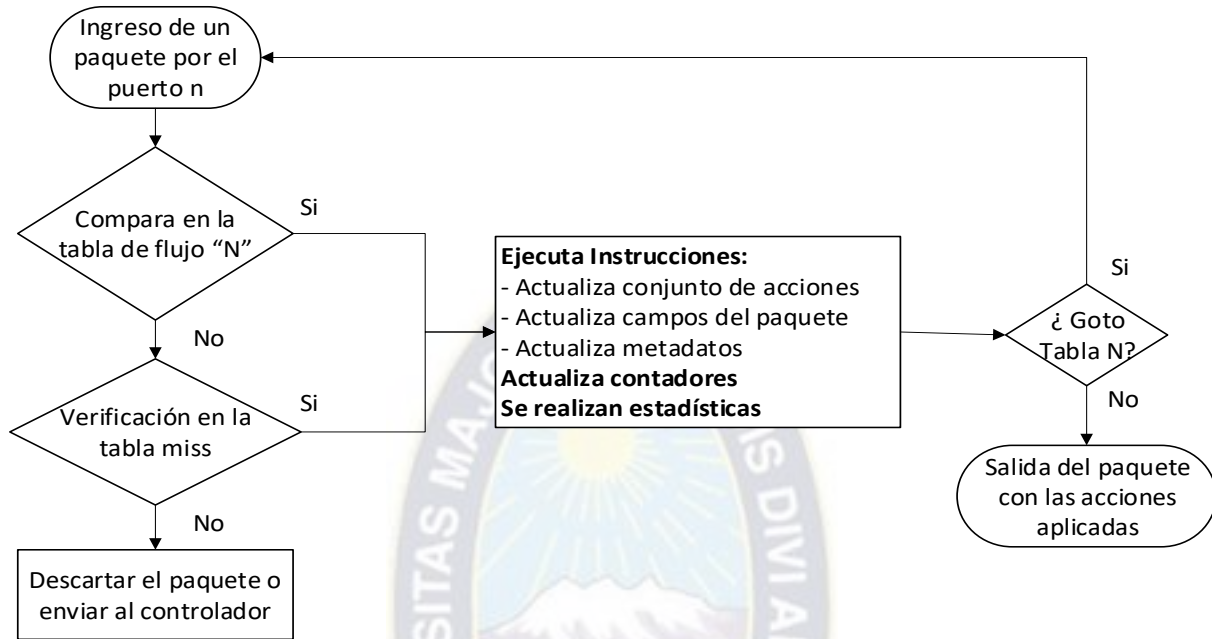


Figura 11. Proceso de canalización de una entrada de flujo.
Fuente: Elaboración propia recopilada de diversas fuentes.

Al realizar el envío de una copia de esta nueva entrada de flujo al controlador, éste tomara una decisión sobre cómo gestionar el paquete y responde al dispositivo con el mensaje “Flow-Mod” para insertar una nueva entrada de flujo en la tabla de flujos. Luego de insertar esta entrada de flujo los siguientes paquetes que coincidan con la nueva entrada añadida se envían directamente a su destino sin necesidad de enviarlos nuevamente al controlador.

2.5.1.2. Tabla de Grupo

Una tabla de grupo se compone de entradas de grupo que identifican las acciones a realizar por un determinado grupo, la habilidad de una entrada de flujo para apuntar a una entrada de grupo permite a OpenFlow presentar métodos adicionales de reenvío de múltiples flujos, y es muy útil al implementar tanto unicast y multicast.

Tabla 4. Elementos que conforman una Entrada de Grupo.

Identificador de grupo	Tipo de grupo	Contadores	Recipientes de acción
------------------------	---------------	------------	-----------------------

Fuente: ONF TS-007 (2012). *OpenFlow Switch Specification - Open Networking Foundation.*

Identificador de grupo: Un entero sin signo de 32 bits que solo identifica el grupo.

Tipo de grupo: Determina la semántica del grupo mediante un conjunto de acciones definidas.

Contadores: Se actualiza cuando los paquetes son procesados por un grupo.

Recipientes de acción: Es una lista ordenada de recipientes de acción, donde cada recipiente de acción contiene un conjunto de acciones a ejecutar y parámetros específicos asociados.

2.5.1.3. Tabla Miss

Es la que determina las medidas específicas de como procesar paquetes que por comparación no coinciden con ninguna entrada de flujo en las tablas de flujos, y se tienen tres posibilidades que son: enviar estos paquetes encapsulados al controlador con el objetivo de solicitar una acción para ejecutar en esta entrada de flujo, dirigir los paquetes a una determinada tabla de flujos o simplemente eliminarlos, de acuerdo a la configuración inicial que le asigna el controlador.

2.5.1.4. Canal Seguro

El canal seguro también conocido como canal OpenFlow es la interfaz que conecta cada dispositivo con el controlador, a través de esta interfaz el controlador configura y administra el dispositivo, permitiendo el intercambio de mensajes mediante el protocolo OpenFlow.

El canal OpenFlow puede cifrarse mediante TLS (Transport Layer Security) o TCP (Transmission Control Protocol), generalmente se utiliza TCP por el puerto 6633 para la versión 1.3 de OpenFlow, pero desde OpenFlow v1.4 la Autoridad de Asignación de Números de Internet (IANA) ha adjudicado a la ONF el puerto TCP 6653 con el objetivo de homogeneizar el uso de un único puerto.

2.5.2. Protocolo OpenFlow

El protocolo OpenFlow provee una manera abierta y estándar en la comunicación entre el switch y el controlador, permitiendo a este último insertar, modificar o eliminar entradas de flujo en las tablas de flujos. Este protocolo “permite acceder directamente y manipular el plano de redireccionamiento de los dispositivos de red como routers y switches, ya sean físicos o virtuales (basado en hipervisor)” (ONF White Paper Software-Defined Networking: The New Norm for Networks, 2012, p8), por lo que facilita el acceso remoto a los dispositivos mediante una interfaz, para ello usa el concepto de flujos para identificar el tráfico de la red, los cuales serán inventariados dentro de una tabla de flujos, de acuerdo a un conjunto de reglas que son programadas en el

controlador de manera estática o dinámica, las cuales definirán la ruta de los flujos a través de los dispositivos de red basados en ciertos parámetros, por ejemplo: patrones de uso, recursos de la red, aplicaciones, origen/destino, entre otros.

En resumen, el protocolo OpenFlow indica al tráfico de datos el camino por donde debe fluir dentro de una red, con lo que se tiene una gestión programable de toda la red.

2.5.2.1. Funcionamiento del Protocolo OpenFlow

A continuación, se detallará la manera básica de establecer la comunicación entre el dispositivo OpenFlow y el controlador SDN, mediante el protocolo OpenFlow.

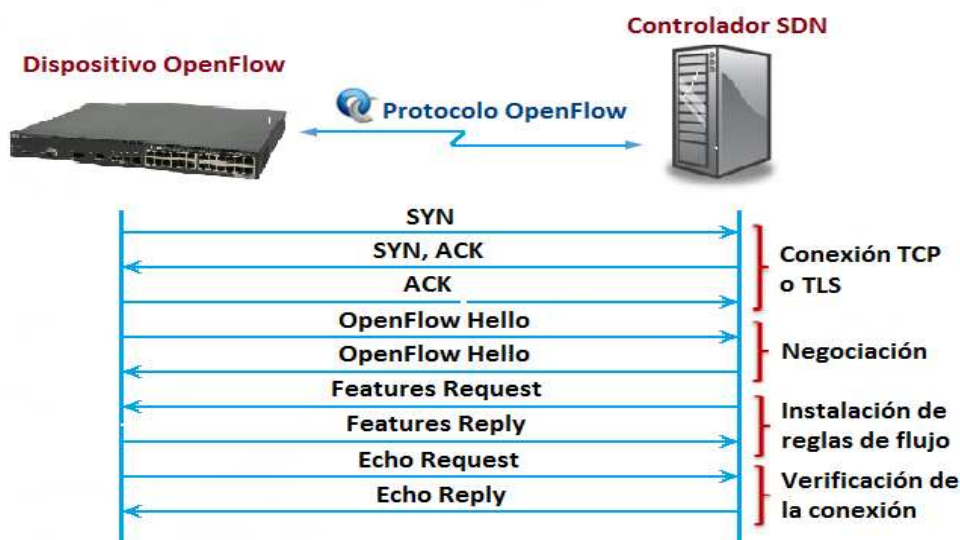


Figura 12. Mensajes para la conexión entre el dispositivo OpenFlow y el Controlador SDN.
Fuente: Elaboración propia recopilada de diversas fuentes.

El controlador se encuentra permanentemente a la escucha de mensajes provenientes de cualquier dispositivo de la red, y es configurado para establecer una conexión mediante el protocolo TCP generalmente, luego el dispositivo y controlador se autentican mediante el intercambio de certificados digitales para lo cual utilizan los mensajes “Hello” para iniciar y establecer la conexión, inmediatamente el controlador envía el mensaje Feature request al cual el dispositivo responde con un “Feature reply”, donde se incluye información de las características físicas y lógicas soportadas por el dispositivo e información adicional sobre el fabricante. Adicionalmente OpenFlow define un mecanismo de verificación del enlace mediante mensajes Echo, similar al Protocolo de Mensajes de Control de Internet - ICMP, para identificar que el canal establecido permanezca activo y funcional para la comunicación entre el controlador y switch.

2.5.2.2. Tipos de Mensajes OpenFlow

Este protocolo soporta tres tipos de mensajes: controlador a switch, asíncronos y simétricos.

2.5.2.2.1. Controlador a Switch

Este tipo de mensajes son iniciados por el controlador y pueden tener o no una respuesta por parte del dispositivo. Entre ellos están la solicitud de sus características, el cambio de parámetros en su configuración, modificación de sus tablas de flujos, entre otros.

2.5.2.2.2. Asíncronos

Los mensajes asíncronos son iniciados por el dispositivo sin que el controlador los solicite. Los dispositivos envían estos mensajes al controlador para notificar la llegada de un nuevo paquete, un cambio de estado o un error en el dispositivo.

2.5.2.2.3. Simétricos

Los mensajes simétricos son enviados bidireccionalmente, enviándose sin necesidad de solicitud previa. Los más importantes son: el mensaje Hello para establecer la conexión, y el mensaje Echo para obtener la latencia, ancho de banda o estado de una conexión.

2.5.2.3. Versiones del Protocolo OpenFlow

El protocolo OpenFlow se encuentra en constante desarrollo para mejorar sus características de funcionamiento, desde que el grupo colaborativo de investigadores lanzó la primera implementación de referencia en noviembre de 2007, con la versión 0.1.0. hasta la versión que se tiene actualmente la 1.5 agregando cada vez nuevas características y mejoras para hacer más flexible la identificación del flujo. Se destacan las versiones 1.0 y la 1.3 que son las más utilizadas ya que presentan una serie de funciones y características de estabilidad, que son importantes a la hora de implementar esta tecnología ya sea en ambientes reales o experimentales.

2.5.2.4. Dispositivos Compatibles con OpenFlow

El protocolo OpenFlow se cataloga como uno de los protocolos del futuro, su desarrollo y gran usabilidad han despertado un gran interés en fabricantes de dispositivos de networking como Cisco, HP, IBM, Ciena, Dell, entre otros, a ellos se suman las exitosas implementaciones en empresas generadoras de contenidos como Google, Facebook y Yahoo! que inspiran confianza para su aprendizaje, desarrollo e implementación.

A continuación, se muestran los fabricantes más destacados en esta rama y sus productos que ya están habilitados para trabajar con el protocolo OpenFlow.

Tabla 5. *Dispositivos compatibles con OpenFlow.*

Fabricante	Productos
Arista	Arista EOS, Arista 7124FX, Series 7050,7150,7500
Ciena	CoreDirector soporte actualización firmware 6.1.1
Cisco	Cisco cat6k, series catalyst 3750, 6500
Dell	Dell Z900 y S4810
Extreme	X870, X670-G2 Series, BlackDiamond 8000-Series
HP	HP series procurve: 5400 zl, 8200 zl, 6200 yl, 3800, 3500 yl
IBM	RackSwitch G8264, G8264T
Juniper	Juniper MX-240, T-640, EX9200
NEC	NEC IP8800, PF5240, PF5820
Pronto	Pronto 3240, 3290, 3780
Toroki	Toroki Lightswitch 4810
Quanta	Quanta LB4G

Fuente: Elaboración propia recopilada de diversas fuentes.

2.6. Mininet

En la actualidad se van desarrollando múltiples productos para la experimentación e implementación de SDN y en el caso de software para la simulación y emulación existen varios programas como ser: EstiNet, NS-3, STS y Mininet.

Mininet es un emulador diseñado especialmente para la virtualización de redes definidas por software y ampliamente recomendado para la investigación, desarrollo y uso académico. La página oficial del emulador Mininet ofrece una guía completa para su instalación, configuración y manejo de las herramientas con las que cuenta el emulador. El manejo de comandos y librerías requiere conocimientos previos de comandos Shell de Linux y el lenguaje de programación Python, Mininet incluye herramientas esenciales como Open vSwitch, el capturador de paquetes Wireshark, utilidades de administración de switches OpenFlow como Data Path Controller (dpctl), entre otros que se utilizan según el desarrollo que se quiere realizar.

Existen varias formas de instalar este emulador, la manera más recomendable para los que quieran experimentar y usarlo por primera vez es usando máquinas virtuales ya que pueden ser desplegadas en cualquier equipo de una manera rápida, sencilla, segura y no pone en riesgo la integridad de su información personal.

El software libre Mininet puede emular un conjunto de hosts, switches, routers, enlaces y controladores sobre el kernel Linux, utiliza técnicas de virtualización para emular redes y mayormente su comportamiento es similar a los elementos de hardware discretos, normalmente es posible crear una red virtual que se parezca a una red real o una red real que se parezca a una red creada en Mininet que pueda ejecutar el mismo código binario y las mismas aplicaciones. Por ejemplo, se puede acceder a cada host individualmente creando una conexión mediante el protocolo SSH (Secure Shell - intérprete de órdenes seguro), y desde allí ejecutar individualmente las aplicaciones instaladas en el sistema Linux.

Mininet posee una serie de características que le hacen un entorno adecuado para el desarrollo y pruebas en el ámbito de redes definidas por software, por ejemplo:

- Sencillo procedimiento de instalación.
- Implementar una red y probar su funcionamiento, posee una curva de aprendizaje corta.
- Posee mayor escalabilidad frente a otro software similar.
- Se pueden ejecutar aplicaciones en cada host, individualmente o desde un servidor web a una herramienta de monitorización.
- Es posible emular redes con múltiples dispositivos de red ejecutando scripts desarrollados mediante programación.
- Se pueden modelar las características de los enlaces como: retardo, velocidad del enlace, incluso se pueden modelar fallos de red con enlaces conectado/desconectado.

Sin embargo, Mininet tiene algunos inconvenientes, entre los cuales cabe mencionar:

- Como en toda virtualización se comparten los recursos del sistema "padre o maestro", lo cual implica que el tamaño y la operatividad de la red emulada vendrá condicionada por dichos recursos.
- No se hace NAT al exterior por defecto, por lo que un host virtual no se podrá conectar con Internet, aunque se pueden programar componentes para lograr dicha conectividad.
- Todos los hosts comparten el mismo sistema de archivos y los mismos PID's (Process Identification – Identificador de procesos), por lo que hay que tener precauciones al instalar daemons que requieran configuración específicamente en el directorio "/etc" del sistema operativo, y no eliminar un determinado proceso por error.

- A diferencia de un simulador, no tiene una fuerte noción de tiempo virtual, esto significa que las mediciones de temporización son basadas en tiempo real, y que los resultados para grandes cargas se verán afectadas en su emulación.

Todos los controladores mencionados anteriormente, trabajan en conjunto con este software de emulación para realizar pruebas de forma local o remota, de forma virtual o con máquinas reales, mediante un script o utilizando la interfaz de línea de comandos. Como se puede evidenciar es un emulador muy flexible y posee una comunidad de colaboradores significativa en internet, donde se puede encontrar una amplia documentación según las necesidades requeridas.

En el Anexo B se expone una lista de comandos, los cuales facilitan el manejo y la familiarización de este emulador.

2.7. Ventajas y Desventajas de las Redes Definidas por Software

La principal ventaja de las redes definidas por software es que esencialmente es una tecnología de código abierto, lo cual facilita el desarrollo y la innovación en diferentes áreas donde se trabaje con redes de datos, como ser: centro de datos, proveedores de servicio, puntos de acceso y demás, a través de la programación e implementada de acuerdo a políticas internas de cada organización, generando una mayor innovación, soluciones más flexibles y sobre todo interoperabilidad con protocolos estandarizados también de código abierto. A continuación, se especifican las principales ventajas que se tiene al implementar esta tecnología.

Mayor tasa de innovación: La adopción de esta tecnología acelera la innovación empresarial permitiendo a los operadores de red, literalmente programar la red en tiempo real, para satisfacer las necesidades específicas de negocio y de los usuarios a medida que estas son requeridas.

Mayor flexibilidad: Las aplicaciones SDN hacen posible definir sentencias de configuración y políticas de alto nivel, que luego son introducidas en los elementos de la red a través del controlador SDN. Una arquitectura de red definida por software elimina la necesidad de configurar individualmente los dispositivos de red cada vez que se realice un cambio, se agregue o elimine un servicio o aplicación, lo que reduce la probabilidad de fallas en la red debido a configuraciones erróneas o políticas inconsistentes.

Mayor fiabilidad y seguridad de la red: Dado que los controladores SDN proporcionan una completa visibilidad y control sobre la red, pueden asegurarse de que el control de acceso, la

ingeniería de tráfico, calidad de servicio, seguridad y otras políticas, son aplicadas consistentemente a través de las infraestructuras de redes cableadas e inalámbricas.

Reduccion de Capex y Opex: Existe una reducción en los costes de capital (Capex) ya que se habilita la posibilidad de utilizar dispositivos de diferentes fabricantes que soportan el protocolo OpenFlow, sin tener que depender de una sola marca. En cuanto a los costos de operación (Opex) tambien se tiene una reduccion significativa, ya que en las redes definidas por software es posible el desarrollo de herramientas que automatizan las tareas de administración de la red (que se realizan de forma manual e individual con las redes tradicionales), de tal manera que se pueden probar en un ambiente virtual y luego implementarlo inmediatamente a la red física.

Entre las desventajas que se presenta al implementar esta tecnología están:

La implementación física: Actualmente son pocos los equipos que soportan este tipo de tecnología y al querer realizar la implementación en equipos reales, se tiene que hacer una inversión para la adquisición de nuevos equipos que soporten el protocolo OpenFlow.

Poco interés al cambio de nuevas tecnologías: La mayoría de los encargados o administradores de red no tienen interés en la implementación de nuevas tecnologías que son poco conocidas y prefieren seguir operando su red de manera tradicional, esto dificulta la innovación, la experimentación y sobre todo el aprovechamiento de los beneficios que tiene esta tecnología.

Falta de personal capacitado en el área: Otra desventaja se presenta al momento de encontrar especialistas en el área, o personal con conocimientos en lenguajes de programación Python, Java o C++, que quiera dedicarle tiempo y esfuerzo en aprender el funcionamiento de esta nueva tecnología y desarrollar aplicaciones basadas en las políticas de la organización para su respectiva implementación.

Las redes definidas por software darán lugar a nuevos beneficiarios de la tecnología, pero también habrá quienes no puedan o no quieran estos cambios ya sea por motivos económicos o solo por costumbre a operar con redes tradicionales, en fin, se verán instituciones o empresas que emergerán con éxito y otras que no podrán superar esta transición de tecnología. En cualquier caso, y al igual que con cualquier otra tendencia del sector, las ventajas para la administración de la infraestructura de red y para el usuario final son reales, como también el cambio tecnológico en este sector será inevitable.

Capítulo 3.- DISEÑO Y DESARROLLO DE LA ARQUITECTURA DE RED DEFINIDA POR SOFTWARE

En este capítulo se presenta una infraestructura de red convencional, muy común en varias empresas, para la cual se realizará el diseño de una arquitectura de red definida por software, utilizando herramientas de software libre en su totalidad, y también se desarrollará el script que emula la red virtual y los módulos de la aplicación que gestionan los caminos redundantes de la topología de red en el lenguaje de programación Python.

3.1. Escenario de Implementación

Dentro de los escenarios que se presentan en las redes de datos, existen diferentes topologías implementadas de manera planificada según los requerimientos o necesidades que tiene la empresa, los empleados o los usuarios finales. Pero existe la posibilidad de que con el tiempo, se puedan realizar mejoras o ampliaciones en la infraestructura y también en la configuración de toda la red (o parte de ella), el escenario convencional en el que se implementará la arquitectura de red definida por software es muy típico en empresas, institutos académicos o edificaciones con oficinas, el cual cuenta con un ambiente o cuarto de telecomunicaciones en cada piso desde el cual se realiza el soporte, mantenimiento y administración de los recursos tecnológicos, como ser: routers, switches, servidor de correos electrónicos, servidores web, telefonía digital, bastidores, gabinetes, instrumentos de medición, administración de accesos a internet, administración de la base de datos, gestión de firewall, entre otros.

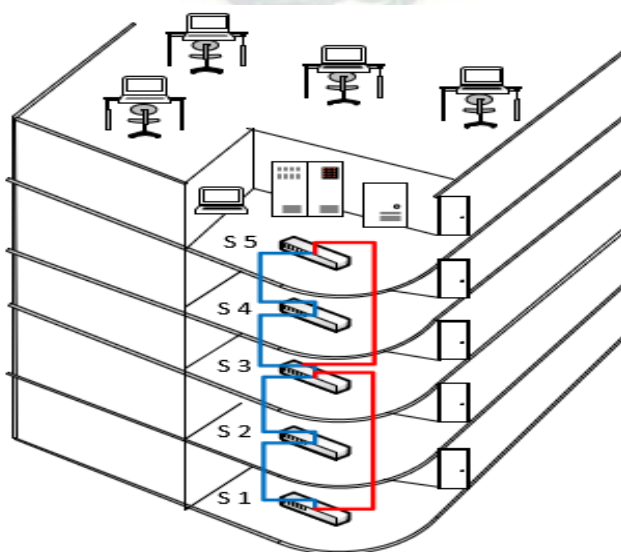


Figura 13. Escenario de implementación de la red definida por software.

Fuente: Elaboración propia.

En muchos casos solo se encontrarán pequeños gabinetes ubicados en un determinado sector de un piso, en los que se instalan y se configuran los equipos necesarios para esa área específica, y típicamente son administrados desde un punto central, dentro de la infraestructura más conocida como centro de operaciones de red (NOC - Network Operations Center).

Para realizar el diseño de la arquitectura de red definida por software para el escenario planteado, se tomará en cuenta un pequeño segmento de toda la infraestructura que consiste específicamente en extraer los switches de la red con su respectiva topología, que serán los elementos principales con los cuales se desarrollara virtualmente la red definida por software, y es en ese conjunto de dispositivos donde se ejecutan los módulos que administran la red automáticamente mediante el controlador SDN.

3.1.1. Descripción de la Topología de Red

De la Figura 13 se extraen solo los switches y su topología de cableado con el cual se encuentran conectados, obteniendo de esta forma una red tal como se muestra en la Figura 14. Entonces se tienen cinco switches, los cuales están conectados unos a otros teniendo por lo menos una ruta de respaldo o camino redundante. Este tipo de topología es implementado frecuentemente según el tipo de tráfico, la prioridad del sector, la ubicación geográfica o la distancia entre los diferentes nodos. Esto con el motivo principal de asegurar la disponibilidad de la red ante la caída de algún enlace o, en situaciones críticas de fuerza mayor, para que la interconexión no se vea afectada.

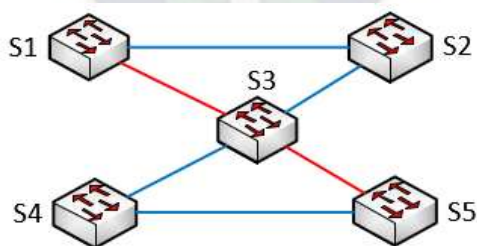


Figura 14. Topología de red ordenada.
Fuente: Elaboración propia.

Por otro lado, se tiene que tomar en cuenta que estos caminos de respaldo, también traen algunos inconvenientes dentro de una topología de red, entre los más significantes estan:

Bucles de capa 2: Las tramas de Ethernet no poseen un tiempo de vida (TTL - Time to Live) como los paquetes IP que viajan por los routers de capa 3. En consecuencia, si no finalizan de

manera adecuada en una red conmutada, las mismas siguen rebotando de switch en switch indefinidamente o hasta que se interrumpa un enlace y elimine el bucle.

Tormentas de Broadcast: Cuando la red está siendo saturada con la emisión constante de paquetes, durante este periodo estos paquetes se encuentran atrapados en un bucle y siguen siendo retransmitidos consumiendo un importante ancho de banda de la red, hasta que el puerto se encuentre colapsado, y pueden llegar a colgar el dispositivo hasta lograr interrumpir el servicio.

Tramas duplicadas: Se presenta cuando se tiene diferentes caminos hacia un mismo destino entregando varias copias de las tramas de unidifusión, estas tramas duplicadas provocan errores de los que no se pueden recuperar y tienden a interrumpir el servicio.

Inestabilidad de la tabla de direcciones MAC: Este problema se presenta por recibir copias de la misma trama que ingresan por diferentes puertos del switch, el reenvío de datos se ve afectado cuando el switch consume todos los recursos que tratan de hacer frente con la inestabilidad que se presenta en la tabla de direcciones MAC.

Entonces, cuando se tienen rutas alternas o caminos redundantes en una topología de red de capa 2, se tiene que tomar muy en cuenta estos aspectos en la configuración de la red y otros que no se expondrán ya que son de poco valor para el presente trabajo o, su caso de estudio es de manera exclusiva, por lo tanto, se demostrará la gestión de estos inconvenientes de forma genérica mediante la implementación del Protocolo de Árbol de Expansión (STP - Spanning Tree Protocol), desarrollada como una API para que se lo pueda utilizar con el controlador Ryu, y consecuentemente mostrar y analizar su funcionamiento dentro de la topología emulada.

3.2. Diseño de la Arquitectura de Red Definida por Software

Para que la topología de red tradicional (presentada anteriormente) se convierta en una arquitectura de red definida por software, es necesario que los switches de la red sean compatibles con una API hacia el sur para establecer una conexión con el controlador SDN, en este caso se utilizará la API hacia el sur OpenFlow en su versión 1.3, y como controlador SDN se utilizará el controlador Ryu, los cuales fueron descritos brevemente en el capítulo anterior. Con todos estos elementos se forma una arquitectura de red definida por software conformada por cinco switches, un controlador SDN y una topología de red mixta en forma de malla y estrella como se muestra en la Figura 15.

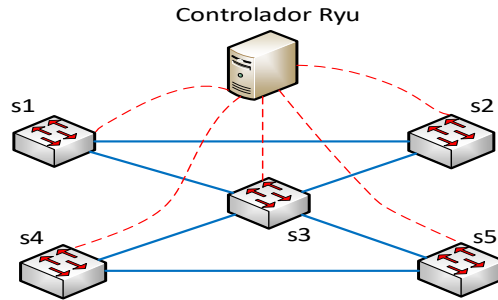


Figura 15. Arquitectura de red definida por software.
Fuente: Elaboración propia.

Una vez que se conoce teóricamente todos los elementos que conforman la arquitectura SDN, a continuación, se describe la metodología que se utilizará para determinar todo el software necesario para su implementación virtual, dividido en dos partes:

Instalación del software y herramientas necesarias para la simulación:

- Se efectuará la instalación del sistema base en el que se realizará toda la implementación virtual, se hace referencia al sistema operativo de código abierto Ubuntu 16.04, del cual se detallará toda su instalación desde el principio.
- Luego, se realizará la instalación del emulador Mininet, programa con el cual se emularán los switches, hosts y los enlaces de la red, creando de esta forma un entorno de red virtual dentro del sistema operativo Ubuntu.
- Por último, se describe detalladamente la instalación del controlador SDN llamado Ryu, que será el cerebro de la red virtual creada con Mininet.

Diseño y desarrollo de los módulos utilizados por el emulador Mininet y el controlador Ryu:

- Se diseñará y se desarrollará un script en Python para el emulador Mininet, que tiene la función de crear los dispositivos virtuales de toda la red con sus respectivas conexiones.
- Se diseñarán y se desarrollarán las aplicaciones que administran la red virtualizada, con el fin de gestionar los caminos redundantes dentro de la red de forma centralizada, con la ejecución del protocolo de árbol de expansión mediante el controlador Ryu.

3.2.1. Instalación del Software y Herramientas Necesarias para la Simulación

Para comenzar con la instalación de todo el software necesario, primero se verificará que se cumple con los requisitos mínimos de hardware que exigen los programas que se instalaran, para

no tener inconvenientes más adelante, y por parte del software se verificará que sea una versión estable y si es posible con soporte.

A continuación, se realizará la instalación del sistema operativo, luego los paquetes y las dependencias necesarias como requisito previo a la instalación del emulador y el controlador como se indica en el Anexo A, se indicará paso a paso el procedimiento que se realiza desde la fase inicial hasta tener listo todo el software que se utilizará en la simulación.

3.2.1.1. Instalación del Sistema Operativo Ubuntu

Se comenzará con la instalación del sistema operativo Ubuntu 16.04 LTS (Long Term Support), para lo cual se cuenta con una notebook de gama media que cumple con las exigencias requeridas de los programas y con las expectativas del presente trabajo, en la siguiente tabla se detalla sus características principales de hardware:

Tabla 6. *Especificación del hardware utilizado.*

HP Pavilion dv6-2151cl Entertainment Notebook PC	
Procesador	Intel Core i3, 2.26 GHz
Memoria RAM	4 GB DDR3
Disco duro	80 GB
Tarjeta de video integrado	Intel HD Graphics

Nota: Elaboración propia.

Para descargar el sistema operativo con el cual se trabajará, se ingresa a la página <http://releases.ubuntu.com/16.04/>, donde se encuentran varios archivos de tipo imagen para la instalación del sistema operativo Ubuntu 16.04 LTS con la denominación Xenial Xerus, en este caso, se elige la opción "64-bit PC (AMD64) desktop image", la cual descargará el archivo ubuntu-16.04.3-desktop-amd64.iso con un tamaño aproximado de 1.7 GB.

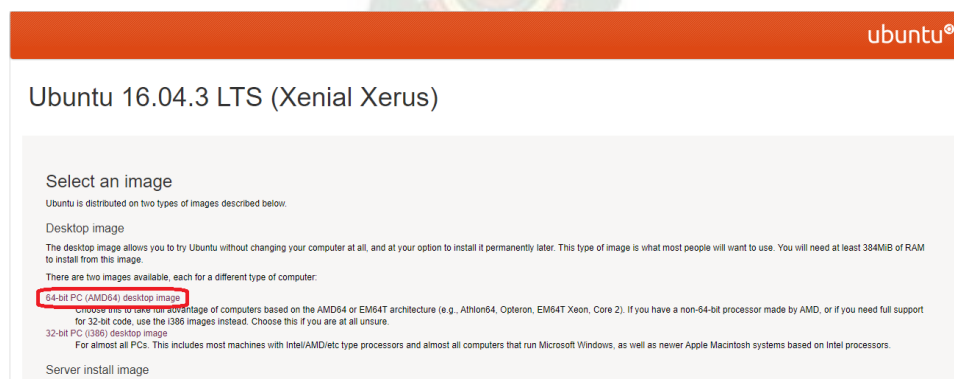


Figura 16. Elección del archivo imagen del Sistema Operativo Ubuntu 16.04 LTS.

Fuente: Elaboración propia.

Una vez descargado el archivo imagen del sistema operativo, se puede quemar este en un disco óptico o llevarlo a una memoria externa USB, en este caso se eligió la segunda opción, por que brinda una mayor velocidad al momento de instalar el sistema operativo y también la facilidad de utilizar cualquier puerto USB de la notebook.

Para la creación de un USB booteable existen varios programas como ser: Rufus, UNetbootin, Yumi, ISO to USB, Universal USB Installer, entre otros. En el presente trabajo se utilizó la herramienta Rufus, pero no se describirá en detalle su instalación o uso ya que no representa ninguna utilidad para el proyecto.

Una vez que se tiene listo la memoria USB booteable se ingresa al sistema básico de entrada y salida (BIOS - Basic Input/Output System) del equipo, para lo cual se presiona la tecla F10 repetidamente hasta que aparezca el menú del BIOS, y se lo configura de modo que arranque desde la Unidad de disquete USB como se muestra en la Figura 17, una vez realizado esta configuración se guardan los cambios y se sale del BIOS.

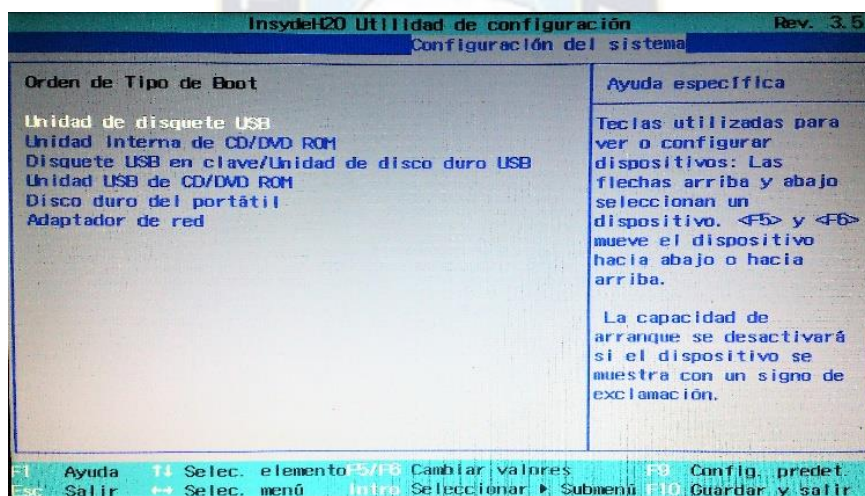


Figura 17. Configuración del BIOS.
Fuente: Elaboración propia.

Se inserta la memoria externa USB al equipo, y al reiniciar el equipo aparecerá la imagen de la Figura 18, en la cual se elige el idioma español en el lado izquierdo y luego la opción “Instalar Ubuntu”.

Posteriormente se marcan las opciones para descargar actualizaciones al instalar Ubuntu, e instalar software de terceros para multimedia, MP3, Flash y compatibilidad con gráficas y Wi-Fi, para no tener problemas con los drivers para la reproducción de los archivos más populares, páginas web con contenido flash, ni con la red Wi-Fi.



Figura 18. Pantalla inicial para la instalación de Ubuntu.
Fuente: Elaboración propia.

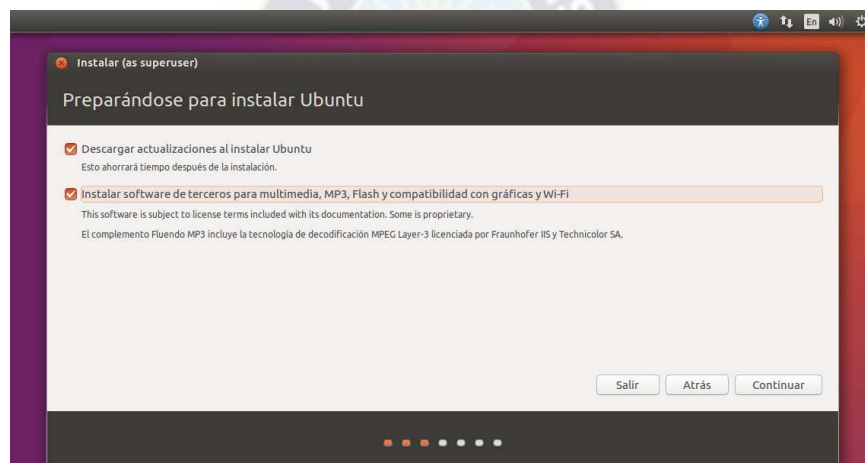


Figura 19. Opciones de instalación de Ubuntu.
Fuente: Elaboración propia.

Continuando con la instalación, se mostrarán los tipos de instalación que se tienen disponibles y se selecciona la última que dice “Más opciones” como muestra la Figura 20, ya que de esta forma se particiona el espacio del disco duro según las necesidades requeridas.

Luego, se mostrará la ventana donde se encuentran las particiones y se realizará las respectivas configuraciones a cada partición, en primera instancia se crea una partición para el área de intercambio de 4 GB como se muestra en la Figura 21, después se crea la partición donde se instalará el sistema operativo como se muestra en la Figura 22.

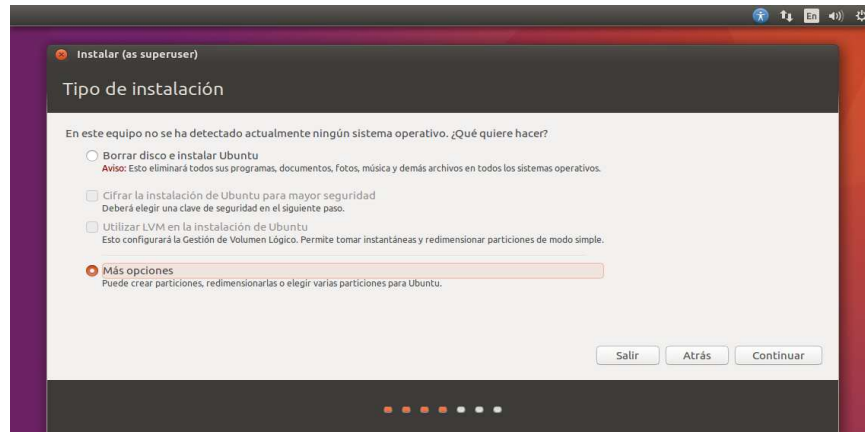


Figura 20. Tipo de instalación en Ubuntu.

Fuente: Elaboración propia.

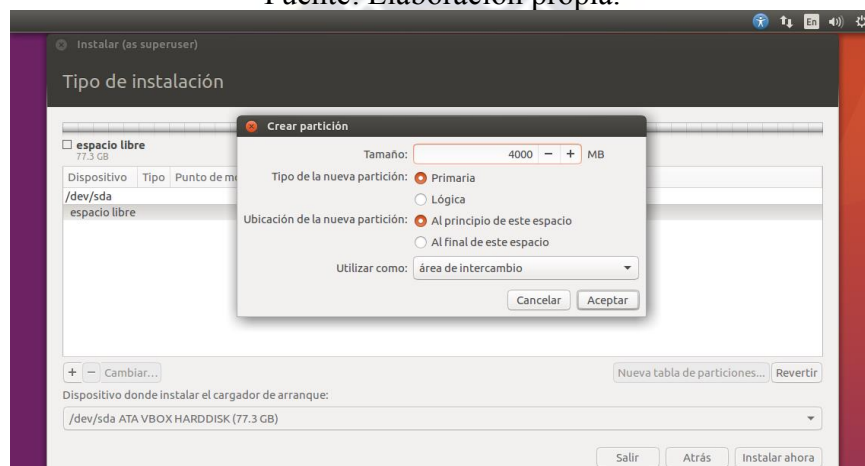


Figura 21. Creación y configuración de la partición del área de intercambio.

Fuente: Elaboración propia.

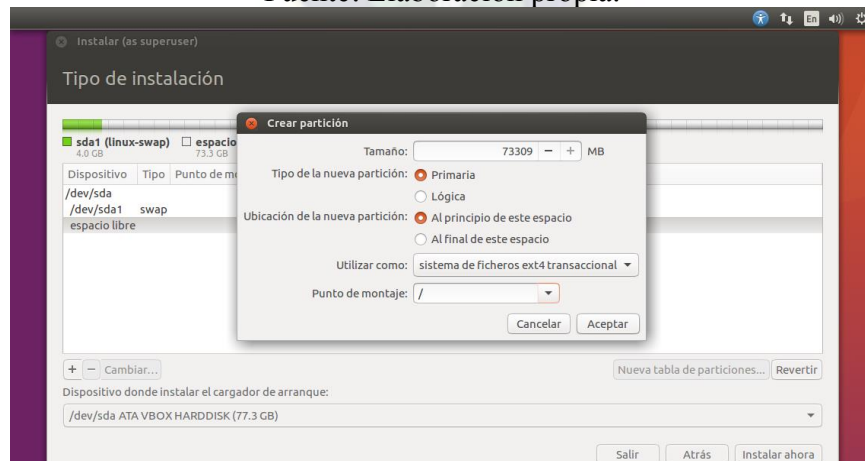


Figura 22. Creación y configuración de la partición del sistema operativo.

Fuente: Elaboración propia.

Siguiendo con la instalación, se debe indicar el lugar geográfico desde donde se está instalando el sistema operativo para obtener las actualizaciones del sistema según la región.

Después se elegirá adecuadamente la distribución del teclado, para no tener inconvenientes con símbolos especiales cuando se utilice la terminal o la interfaz de línea de comandos.

En el siguiente paso se definirá los datos del usuario, equipo y contraseña para utilizar el equipo.



Figura 23. Configuración de los datos del equipo y usuario.

Fuente: Elaboración propia.

Una vez realizado este proceso, solo queda esperar que la instalación y configuración del sistema se realice de manera automática.

Si la instalación se realizó correctamente aparecerá el mensaje de la Figura 24, y solo queda reiniciar el sistema, se quita el dispositivo de instalación y se tendrá listo Ubuntu 16.04 LTS.

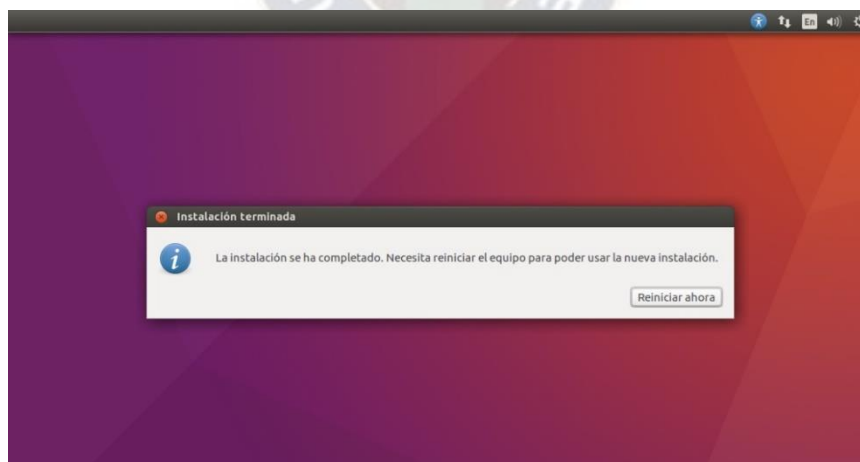


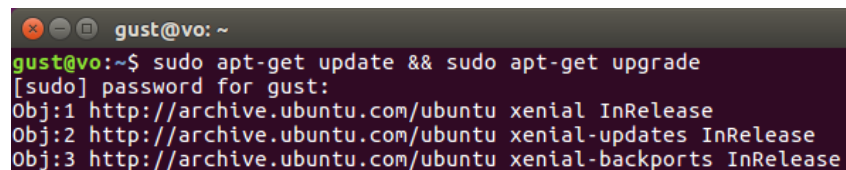
Figura 24. Mensaje de finalización de instalación del sistema operativo Ubuntu 16.04 LTS.

Fuente: Elaboración propia.

Una vez que reinicie el sistema operativo, lo primero que se recomienda hacer es la configuración manual del servidor que contienen los paquetes de actualización del sistema operativo, de preferencia elegimos el servidor para Bolivia para tener mayor velocidad de descarga

en las actualizaciones o se puede probar con otros servidores de acuerdo a la ubicación geográfica, en el caso que falle al actualizar algún repositorio, la descarga demore mucho tiempo o no se tenga ninguna respuesta de parte del servidor. En este caso se tuvo que cambiar al servidor principal para resolver los problemas que presentaba el servidor para Bolivia con algunos repositorios.

Entonces, para tener el sistema operativo actualizado y listo para poder trabajar con él, se verificará si existen actualizaciones disponibles y se instalarán las mismas mediante la terminal del sistema, con el comando mostrado en la Figura 25.



```
gust@vo: ~  
gust@vo:~$ sudo apt-get update && sudo apt-get upgrade  
[sudo] password for gust:  
Obj:1 http://archive.ubuntu.com/ubuntu xenial InRelease  
Obj:2 http://archive.ubuntu.com/ubuntu xenial-updates InRelease  
Obj:3 http://archive.ubuntu.com/ubuntu xenial-backports InRelease
```

Figura 25. Verificación de actualizaciones e instalación de repositorios de Ubuntu.

Fuente: Elaboración propia.

Existe mucha información sobre la instalación y configuración de este sistema operativo, lo que llevo a mostrar todo ese proceso es que muchas personas que trabajan en el área de tecnologías de la información utilizan el sistema operativo Windows y están familiarizados con sus herramientas, tipos de particionado, programas, archivos, extensiones y demás, y si bien tienen conocimiento sobre el uso de alguna distribución Linux no conocen o no recuerdan a detalle el proceso de instalación de la misma.

Antes de seguir con la instalación de los otros programas, se deben instalar las herramientas de software descritas en el Anexo A, que evitaren inconvenientes en la instalación, configuración y funcionamiento del emulador Mininet, del controlador Ryu y consecuentemente de la SDN.

3.2.1.2. Instalación del Emulador Mininet

Para comenzar con la instalación del emulador Mininet se debe tener instalado por lo menos el control de versiones de software Git, como se muestra en el Anexo A.

Para la instalación de este emulador de redes virtuales, se va directamente a su página web <http://mininet.org/>, y en el apartado <http://mininet.org/download/> se tiene cuatro opciones de instalación con sus respectivas instrucciones, como muestra la Figura 26.

Para este caso se elegirá la segunda opción que es la instalación nativa desde la fuente, entonces se clonará el código fuente del emulador Mininet desde la plataforma de desarrollo colaborativo GitHub, el cual aloja los proyectos que utiliza el sistema de control de versiones Git.

Download/Get Started With Mininet

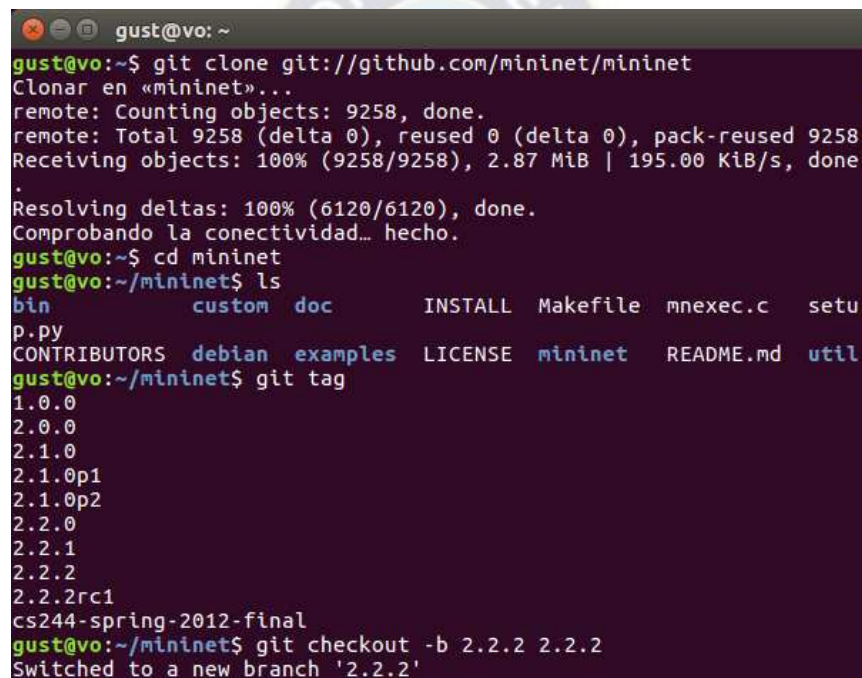
The easiest way to get started is to **download a pre-packaged Mininet/Ubuntu VM**. This VM includes Mininet itself, all OpenFlow binaries and tools pre-installed, and tweaks to the kernel configuration to support larger Mininet networks.

- [Option 1: Mininet VM Installation \(easy, recommended\)](#)
- [Option 2: Native Installation from Source](#)
- [Option 3: Installation from Packages](#)
- [Option 4: Upgrading an existing Mininet Installation](#)

Figura 26. Opciones de instalación del emulador Mininet.

Fuente: Elaboración propia.

Dentro de esta opción se describe el procedimiento con el que se instala el emulador Mininet, entonces se abre una terminal del sistema Ubuntu y se siguen las instrucciones de la Figura 27.



```
gust@vo: ~  
gust@vo:~$ git clone git://github.com/mininet/mininet  
Clonar en «mininet»...  
remote: Counting objects: 9258, done.  
remote: Total 9258 (delta 0), reused 0 (delta 0), pack-reused 9258  
Receiving objects: 100% (9258/9258), 2.87 MiB | 195.00 KiB/s, done  
.  
Resolving deltas: 100% (6120/6120), done.  
Comprobando la conectividad... hecho.  
gust@vo:~$ cd mininet  
gust@vo:~/mininet$ ls  
bin          custom  doc      INSTALL  Makefile  mnexec.c  setu  
p.py  
CONTRIBUTORS  debian  examples LICENSE  mininet  README.md  util  
gust@vo:~/mininet$ git tag  
1.0.0  
2.0.0  
2.1.0  
2.1.0p1  
2.1.0p2  
2.2.0  
2.2.1  
2.2.2  
2.2.2rc1  
cs244-spring-2012-final  
gust@vo:~/mininet$ git checkout -b 2.2.2 2.2.2  
Switched to a new branch '2.2.2'
```

Figura 27. Clonación del código fuente de Mininet a nuestra notebook.

Fuente: Elaboración propia.

A continuación, se detallará cada instrucción que se utilizó:

Para comenzar, se clono el código fuente del emulador desde github con el comando:

```
gust@vo:~$ git clone git://github.com/mininet/mininet
```

Una vez terminado el proceso, se ingresa a la carpeta “mininet” y se observan las carpetas que se clonaron con los siguientes comandos:

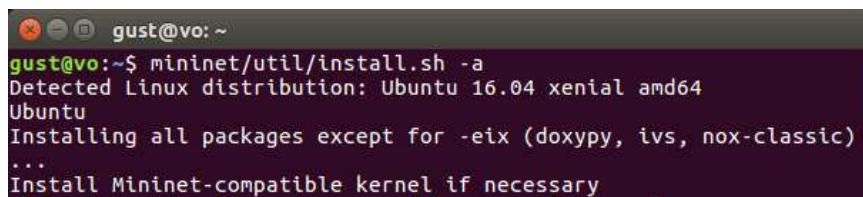
```
gust@vo:~$ cd mininet
```

```
gust@vo:~/mininet$ ls
```


Luego se verifica las diferentes versiones que existen disponibles para su instalación, y se elige la versión 2.2.2 con los siguientes comandos respectivamente:

```
gust@vo:~/mininet$ git tag
gust@vo:~/mininet$ git checkout -b 2.2.2 2.2.2
```

Ahora se instalará el emulador Mininet en el directorio “Home”, aunque existe la opción de crear un nuevo directorio para poder instalarlo allí.

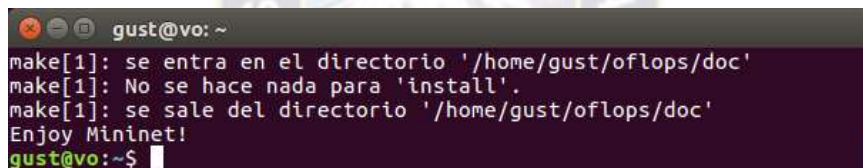


```
gust@vo: ~
gust@vo:~$ mininet/util/install.sh -a
Detected Linux distribution: Ubuntu 16.04 xenial amd64
Ubuntu
Installing all packages except for -eix (doxypy, ivs, nox-classic)
...
Install Mininet-compatible kernel if necessary
```

Figura 28. Instalación del emulador Mininet.

Fuente: Elaboración propia.

Con la instrucción mostrada en la Figura 28, se instaló todas las dependencias que incluye Mininet en directorios específicos, como: OpenFlow, Open vSwitch, el analizador de protocolos Wireshark y el controlador POX, que vienen por defecto junto con los archivos que se clonaron.



```
gust@vo: ~
make[1]: se entra en el directorio '/home/gust/oflops/doc'
make[1]: No se hace nada para 'install'.
make[1]: se sale del directorio '/home/gust/oflops/doc'
Enjoy Mininet!
gust@vo:~$
```

Figura 29. Mensaje de finalización de la instalación correcta del emulador Mininet.

Fuente: Elaboración propia.

Si la instalación no tuvo ningún problema se mostrará el mensaje de “Enjoy Mininet!” al final de la instalación, y se tendrá el emulador listo para utilizarlo.

3.2.1.3. Instalación del Controlador Ryu

Para la instalación de este controlador, es imprescindible la instalación de todo el software que se hace referencia en el Anexo A para no tener conflictos con los repositorios requeridos por este controlador, el proceso es muy similar a la instalación del emulador Mininet, se comenzará con la clonación del código fuente del controlador desde GitHub, y una vez que se tengan todos los archivos en la carpeta personal se realizará la instalación del mismo.

En la página oficial del controlador (<http://osrg.github.io/ryu/>), se muestran los métodos con los cuales se puede instalar este controlador y los programas necesarios para su funcionamiento, en este caso se elegirá la segunda opción y se seguirán las instrucciones recomendadas.

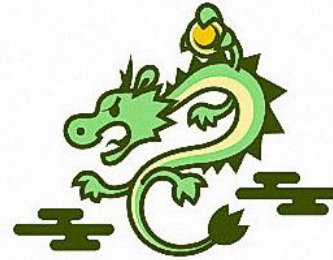
INSTALLATION IS A SNAP

Using `pip` command is the easiest option:

```
% pip install ryu
```

If you prefer to install from the source code:

```
% git clone git://github.com/osrg/ryu.git  
% cd ryu; python ./setup.py install
```



Do you want to know how to write Ryu applications? Let's start with [our tutorial](#).

Figura 30. Métodos de instalación del controlador Ryu.

Fuente: Elaboración propia.

Entonces, para comenzar se ejecutará el comando de la Figura 31, el cual clonará el código fuente del controlador Ryu al directorio personal del Sistema Operativo.

```
gust@vo: ~/ryu  
gust@vo:~$ git clone git://github.com/osrg/ryu.git  
Clonar en «ryu»...  
remote: Counting objects: 25841, done.  
remote: Total 25841 (delta 0), reused 0 (delta 0), pack-reused 25841  
Receiving objects: 100% (25841/25841), 23.53 MiB | 279.00 KiB/s, done.  
Resolving deltas: 100% (19223/19223), done.  
Comprobando la conectividad... hecho.
```

Figura 31. Clonación del código fuente del controlador Ryu desde GitHub.

Fuente: Elaboración propia.

Una vez que se tiene clonado todos los archivos necesarios en el directorio personal, que aproximadamente tiene un tamaño de 23,53 MB, se procede con la instalación del controlador Ryu en la notebook, con la instrucción mostrada en la Figura 32.

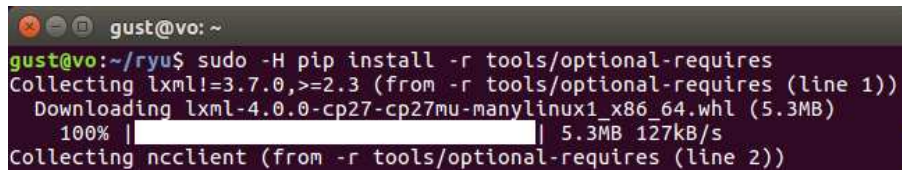
```
gust@vo: ~/ryu  
gust@vo:~$ cd ryu; pip install .  
Processing /home/gust/ryu  
Collecting eventlet!=0.18.3,!0.20.1,<0.21.0,>=0.18.2 (from ryu==4.17)  
  Downloading eventlet-0.20.0-py2.py3-none-any.whl (387kB)  
    10% |  
    3% |  
    51kB 119kB/s eta 0:00:0 15% |  
    61kB 143kB/s eta 0:00:0 18% |
```

Figura 32. Procedimiento de instalación del controlador Ryu.

Fuente: Elaboración propia.

Hasta aquí se tiene la instalación básica del controlador Ryu en la notebook, pero se añadirán herramientas extras de mucha utilidad para tener el controlador equipado con otras funcionalidades que pueden ser necesarios al momento de probar o poner en funcionamiento una determinada aplicación como ser: OF-config, NETCONF, escuchador BGP y el protocolo de servicio Zebra.

Para ello se ejecuta el comando de la Figura 33, y se verifica que no exista errores de descarga para su posterior instalación.

A terminal window with a dark background and light text. The prompt is 'gust@vo: ~'. The command entered is 'sudo -H pip install -r tools/optional-requirements'. The output shows the collection of 'lxml=3.7.0, >=2.3' and the downloading of 'lxml-4.0.0-cp27-cp27mu-manylinux1_x86_64.whl (5.3MB)' at 100% completion with a speed of 5.3MB 127kB/s. It also shows the collection of 'ncclient' from the same requirements file.

```
gust@vo: ~  
gust@vo:~/ryu$ sudo -H pip install -r tools/optional-requirements  
Collecting lxml=3.7.0, >=2.3 (from -r tools/optional-requirements (line 1))  
  Downloading lxml-4.0.0-cp27-cp27mu-manylinux1_x86_64.whl (5.3MB)  
    100% |#####| 5.3MB 127kB/s  
Collecting ncclient (from -r tools/optional-requirements (line 2))
```

Figura 33. Instalación de herramientas adicionales para el controlador Ryu.

Fuente: Elaboración propia.

Con este último procedimiento se tiene instalado todas las herramientas necesarias en la notebook para poder implementar virtualmente la arquitectura de red definida por software.

Los primeros pasos que se realizaron luego de la instalación fueron: hacer pruebas preliminares para la familiarización del software y verificar el funcionamiento tanto del emulador como del controlador Ryu como se expone en el Anexo B, y por otro lado, también se verificó la potencia del equipo físico cuando se llega a simular grandes escenarios o cargas mediante la virtualización, ya que en ciertas ocasiones se puede llegar a cargar tanto el sistema que la emulación puede llegar a presentar incoherencias en sus resultados y algunos retardos, como se mencionó anteriormente en las desventajas del software de emulación.

3.2.2. Diseño y Desarrollo de los Módulos Utilizados por el Emulador Mininet y el Controlador Ryu

Luego de haber instalado y probado el funcionamiento de las herramientas de software anteriormente descritas, lo que se necesita ahora es emular una red y una aplicación específica con las cuales se implementará la arquitectura SDN.

Primeramente, se describirá el desarrollo del script que emula la red virtual requerida mediante el programa Mininet, y luego se desarrollarán las aplicaciones o módulos con los que trabaja el controlador Ryu para la administración de esa red virtual mediante la ejecución del protocolo de árbol de expansión.

3.2.2.1. Diseño y Desarrollo del Script que Emula la Red Virtual

Como se mencionó anteriormente el emulador Mininet está desarrollado en el lenguaje Python, y para crear el prototipo de red propuesto se desarrollará un script también en el mismo lenguaje con todos los elementos necesarios como se muestra en la Figura 34.

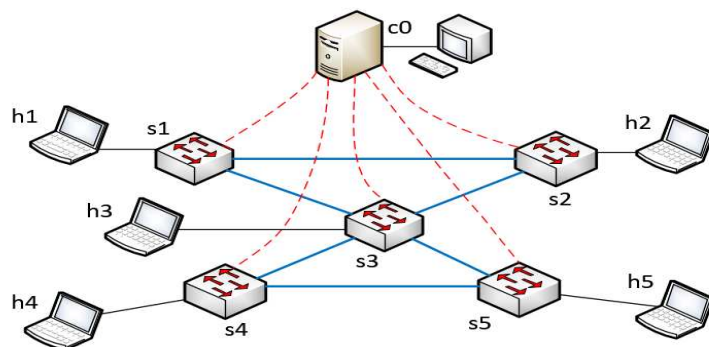


Figura 34. Topología de red y dispositivos que se emulan.
Fuente: Elaboración propia.

De la Figura 34, se registra en una tabla los elementos que se muestran y se asignará su respectiva denominación, dirección IP y dirección MAC.

Tabla 7. Dispositivos de la topología de red con sus respectivas características.

Nº	Dispositivo	Denominación	Dirección IP	Dirección MAC
1	Controlador	c0	127.0.0.1	--
2	Switch 1	s1	Aleatorio	Aleatorio
3	Switch 2	s2	Aleatorio	Aleatorio
4	Switch 3	s3	Aleatorio	Aleatorio
5	Switch 4	s4	Aleatorio	Aleatorio
6	Switch 5	s5	Aleatorio	Aleatorio
7	Host 1	h1	10.0.0.1	00:00:00:01:01:01
8	Host 2	h2	10.0.0.2	00:00:00:02:02:02
9	Host 3	h3	10.0.0.3	00:00:00:03:03:03
10	Host 4	h4	10.0.0.4	00:00:00:04:04:04
11	Host 5	h5	10.0.0.5	00:00:00:05:05:05

Nota: Elaboración propia.

El desarrollo del script para el emulador se lo realizara como una API Mininet de nivel medio, ya que es la más concisa y comprensible para el presente trabajo, además que presenta una curva de aprendizaje también media y solo se describirán las clases, métodos, funciones y variables que se utilizaran en el archivo correspondiente al script.

Para el desarrollo del script correspondiente a la arquitectura de red de la Figura 34, se realizó el siguiente procedimiento descrito en forma de pseudocódigo, con el objetivo de comprender previamente que es lo que se necesita hacer dentro del script y como se lo hará:

1. Se define que el archivo tiene que ser interpretado por Python.
2. Se importan las librerías necesarias para la creación de la topología de red.

3. Se define una función y dentro de ella se realizan los siguientes procesos:
 - a. Se definen las condiciones de configuración para el controlador.
 - b. Se establece la cantidad y el tipo de switches que se utilizarán.
 - c. Se establece la cantidad de hosts con sus respectivas características.
 - d. Se crean los enlaces correspondientes de la topología de red.
 - e. Se realiza la interconexión de los dispositivos.
 - f. Se inician los dispositivos y la red en general.
 - g. Los switches estarán a la espera del controlador.
 - h. Se inicia la interfaz de línea de comandos de Mininet.

A continuación, se muestra el código desarrollado para la arquitectura de red propuesta y el archivo será nombrado como “redmininet.py”.

Código 1. Script desarrollado para la emulación de la red virtual.

```
#!/usr/bin/env python
# <<< CODIFICACION DE LA ARQUITECTURA DE RED >>>

# Importacion de las librerias necesarias de Mininet
from mininet.net import Mininet
from mininet.node import Switch, Host, Node
from mininet.node import RemoteController
from mininet.node import OVSSwitch
from mininet.cli import CLI
from mininet.log import setLogLevel, info
from mininet.link import TCLink, Intf
from subprocess import call

def myNetwork():
    net = Mininet(controller=RemoteController, link=TCLink)
    info( '*** configuracion para el controlador\n' )
    c0=net.addController(name='c0', protocol='tcp', protocols='OpenFlow13',
port=6633)

    info( '*** creando switches...\n')
    s1 = net.addSwitch('s1', cls=OVSSwitch, protocols='OpenFlow13')
    s2 = net.addSwitch('s2', cls=OVSSwitch, protocols='OpenFlow13')
    s3 = net.addSwitch('s3', cls=OVSSwitch, protocols='OpenFlow13')
    s4 = net.addSwitch('s4', cls=OVSSwitch, protocols='OpenFlow13')
    s5 = net.addSwitch('s5', cls=OVSSwitch, protocols='OpenFlow13')

    info( '*** creando hosts...\n')
    h1 = net.addHost('h1', cls=Host, ip='10.0.0.1', mac='00:00:00:01:01:01')
    h2 = net.addHost('h2', cls=Host, ip='10.0.0.2', mac='00:00:00:02:02:02')
    h3 = net.addHost('h3', cls=Host, ip='10.0.0.3', mac='00:00:00:03:03:03')
    h4 = net.addHost('h4', cls=Host, ip='10.0.0.4', mac='00:00:00:04:04:04')
    h5 = net.addHost('h5', cls=Host, ip='10.0.0.5', mac='00:00:00:05:05:05')

    info( '*** creando enlaces entre dispositivos...\n')
    net.addLink('h1', 's1', cls=TCLink, bw=10)
    net.addLink('h2', 's2', cls=TCLink, bw=10)
    net.addLink('h3', 's3', cls=TCLink, bw=10)
    net.addLink('h4', 's4', cls=TCLink, bw=10)
    net.addLink('h5', 's5', cls=TCLink, bw=10)
```



```

net.addLink('s1', 's2', cls=TCLink, bw=100)
net.addLink('s2', 's3', cls=TCLink, bw=100)
net.addLink('s3', 's4', cls=TCLink, bw=100)
net.addLink('s4', 's5', cls=TCLink, bw=100)
net.addLink('s1', 's3', cls=TCLink, bw=100)
net.addLink('s3', 's5', cls=TCLink, bw=100)

info( '*** iniciando la red...\n')
net.build()

for controller in net.controllers:
    controller.start()
info( '*** switches esperando por el controlador remoto...\n')
net.get('s1').start([c0])
net.get('s2').start([c0])
net.get('s3').start([c0])
net.get('s4').start([c0])
net.get('s5').start([c0])

info( '*** RED ESTABLECIDA ***\n')

CLI(net)
net.stop()

if __name__ == '__main__':
    setLogLevel('info')
    myNetwork()

```

Una vez explicado y ejecutado el procedimiento con el que se desarrolla el script necesario para emular la topología requerida, ahora se explicará el programa según cada proceso.

La primera línea del código hace referencia al proceso que es utilizado por el cargador de programas en Linux.

Código 1.1. Cabecera del script para ejecutar código python.

```
#!/usr/bin/env python
```

Con las siguientes líneas se importan las librerías necesarias con las que cuenta Mininet para el funcionamiento de la arquitectura de red.

Código 1.2. Importación de librerías Mininet.

```

# Importacion de las librerías necesarias de Mininet
from mininet.net import Mininet
from mininet.node import Switch, Host, Node
from mininet.node import RemoteController
from mininet.node import OVSSwitch
from mininet.cli import CLI
from mininet.log import setLogLevel, info
from mininet.link import TCLink, Intf
from subprocess import call

```

A continuación se define la función `myNetwork()`, en el que se indica que el controlador de la red será implementada de manera remota con el nombre `c0`, se conectará mediante el protocolo TCP, tendrá soporte para OpenFlow v1.3 y su puerto de comunicación será el 6633.

Código 1.3. Definición de la función principal y configuración para el controlador.

```
def myNetwork():  
  
    net = Mininet(controller=RemoteController, link=TCLink)  
    info( '*** configuracion para el controlador\n' )  
    c0=net.addController(name='c0', protocol='tcp', protocols='OpenFlow13',  
port=6633)
```

Con las siguientes líneas se define el nombre de los switches, y también el tipo de switch virtual que se utilizará, en este caso será el Open vSwitch.

Código 1.4. Asignación de características principales a los switches virtuales.

```
info( '*** creando switches...\n')  
s1 = net.addSwitch('s1', cls=OVSSwitch, protocols='OpenFlow13')  
s2 = net.addSwitch('s2', cls=OVSSwitch, protocols='OpenFlow13')  
s3 = net.addSwitch('s3', cls=OVSSwitch, protocols='OpenFlow13')  
s4 = net.addSwitch('s4', cls=OVSSwitch, protocols='OpenFlow13')  
s5 = net.addSwitch('s5', cls=OVSSwitch, protocols='OpenFlow13')
```

De la misma forma que se hizo con los switches, ahora se definen las características de los hosts, como ser: denominación del host, dirección IP y dirección MAC.

Código 1.5. Asignación de características principales a los hosts virtuales.

```
info( '*** creando hosts...\n')  
h1 = net.addHost('h1', cls=Host, ip='10.0.0.1', mac='00:00:00:01:01:01')  
h2 = net.addHost('h2', cls=Host, ip='10.0.0.2', mac='00:00:00:02:02:02')  
h3 = net.addHost('h3', cls=Host, ip='10.0.0.3', mac='00:00:00:03:03:03')  
h4 = net.addHost('h4', cls=Host, ip='10.0.0.4', mac='00:00:00:04:04:04')  
h5 = net.addHost('h5', cls=Host, ip='10.0.0.5', mac='00:00:00:05:05:05')
```

Una vez definidas las características principales de los switches y hosts, se definen las conexiones correspondientes dentro de la red según la Figura 34.

Código 1.6. Creación de los enlaces correspondientes entre switches y hosts.

```
info( '*** creando enlaces entre dispositivos...\n')  
net.addLink('h1', 's1', cls=TCLink, bw=10)  
net.addLink('h2', 's2', cls=TCLink, bw=10)  
net.addLink('h3', 's3', cls=TCLink, bw=10)  
net.addLink('h4', 's4', cls=TCLink, bw=10)  
net.addLink('h5', 's5', cls=TCLink, bw=10)  
net.addLink('s1', 's2', cls=TCLink, bw=100)  
net.addLink('s2', 's3', cls=TCLink, bw=100)  
net.addLink('s3', 's4', cls=TCLink, bw=100)  
net.addLink('s4', 's5', cls=TCLink, bw=100)  
net.addLink('s1', 's3', cls=TCLink, bw=100)  
net.addLink('s3', 's5', cls=TCLink, bw=100)
```

Luego se inicia la red, y se deja en espera las conexiones de los switches con el controlador.

Código 1.7. Construcción de la red e inicialización del controlador.

```
info( '*** iniciando la red...\n')  
net.build()  
  
for controller in net.controllers:  
    controller.start()  
info( '*** switches esperando por el controlador remoto...\n')  
net.get('s1').start([c0])  
net.get('s2').start([c0])
```



```
net.get('s3').start([c0])
net.get('s4').start([c0])
net.get('s5').start([c0])
```

Con las siguientes instrucciones se inicia la interfaz de línea de comandos de Mininet, y luego se utiliza el método llamado `stop()` para realizar un alto en la red.

Código 1.8. Inicialización de la interfaz de línea de comandos de Mininet.

```
info( '*** RED ESTABLECIDA ***\n')
CLI(net)
net.stop()
```

En la primera línea del último proceso, se utilizan sentencias de python que implican la posibilidad del script de poder importarlo o usarlo desde otro modulo, en la segunda línea se utiliza el comando que muestra en pantalla la información que se ejecuta dentro de la función y en la tercera línea se hace una llamada a la función principal llamada `myNetwork()`.

Código 1.9. Comandos complementarios que realizan funciones generales.

```
if __name__ == '__main__':
    setLogLevel( 'info' )
    myNetwork()
```

Como se pudo observar, la creación del script para la red virtual requerida no es muy compleja, requiere conocimientos de programación en el lenguaje Python de nivel medio ya que todas las variables son tratadas como objetos, facilitando su interpretación y flexibilidad al momento de implementarlos, y por otro lado, estudiar y comprender los métodos que utiliza Mininet con sus respectivas librerías, funciones y clases, además que cuenta con varias herramientas útiles que se mostraran al momento de verificar el funcionamiento de la red virtualizada.

3.2.2.2. Diseño y Desarrollo de la API para el Controlador Ryu

El diseño y desarrollo de la API que se utilizará con el controlador Ryu se dividirá en dos módulos, el primero consiste en la creación de un módulo para gestionar los switches emulados con las funciones básicas de un switch estándar, y en el segundo módulo se desarrollan las funciones que realizan el cálculo del protocolo de árbol de expansión, con el objetivo de gestionar los caminos redundantes en la red virtualizada.

Para el desarrollo del primer módulo, se configuran las propiedades del controlador y se define el modo en que las variables serán gestionadas a nivel global, luego se establece como se instalara un flujo en la tabla miss, para lo cual se define las siguientes variables: el identificador de la ruta de datos, la prioridad, la coincidencia (el match) y las acciones, las cuales son gestionadas por las librerías del controlador, luego se construye el mensaje modificador de flujos que es enviado al

switch de acuerdo a las características de cada dispositivo, después se define la función que será la encargada de verificar si el paquete entrante es de tipo ethernet para seguir con el proceso y si no es del tipo ethernet desecharlo, continuando con el proceso se seleccionan varias características del paquete de entrada como ser: el identificador del camino de datos, origen del paquete, destino del paquete y puerto de entrada del paquete, y estos datos son aprendidos por el switch, de tal forma que, cuando exista un nuevo ingreso que tenga esa misma dirección (origen y destino) el reenvío por el puerto correspondiente será automático sin necesidad de enviarlo al controlador, pero en el caso en el que la entrada de flujo no tenga datos similares se realiza una búsqueda del destino mediante el proceso de inundación, para verificar si se tiene una respuesta del dispositivo destino y poder enviarlo, el procedimiento se resume en la Figura 35.

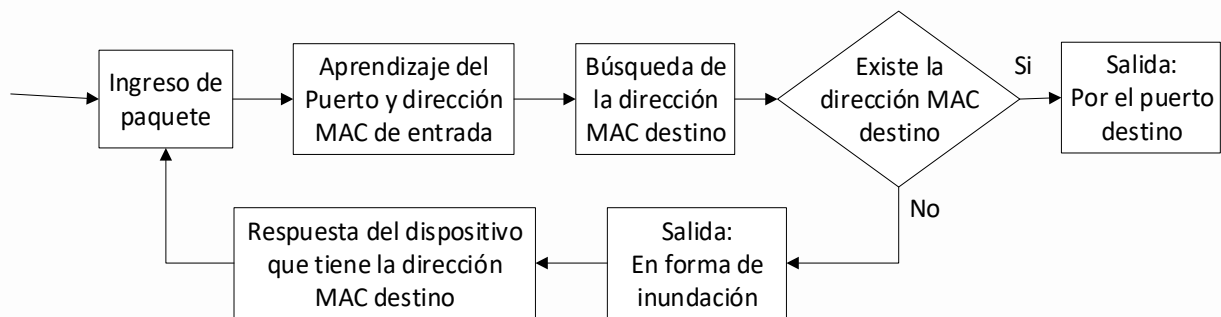


Figura 35. Procesamiento del paquete de entrada al switch.

Fuente: Elaboración propia.

Con el procedimiento explicado anteriormente se desarrolla el primer módulo para el controlador Ryu, el cual se llamará “switch_proyecto.py”.

Código 2. Módulo que gestiona al switch emulado.

```

# <<< CODIFICACION PARA EL SWITCH CON LA FUNCION DE APRENDER LAS DIRECCIONES MAC >>>

# Importacion de las librerias requeridas
from ryu.base import app_manager
from ryu.controller import ofp_event
from ryu.controller.handler import CONFIG_DISPATCHER, MAIN_DISPATCHER
from ryu.controller.handler import set_ev_cls
from ryu.ofproto import ofproto_v1_3
from ryu.lib.packet import packet
from ryu.lib.packet import ethernet
from ryu.lib.packet import ether_types

class SwitchProyecto(app_manager.RyuApp):
    # Definicion de la version OpenFlow
    OFP_VERSIONS = [ofproto_v1_3.OFP_VERSION]

    def __init__(self, *args, **kwargs):
        super(SwitchProyecto, self).__init__(*args, **kwargs)
        # Inicializacion de la tabla MAC address
        self.mac_to_port = {}

    @set_ev_cls(ofp_event.EventOFPSwitchFeatures, CONFIG_DISPATCHER)

```

```

def switch_features_handler(self, ev):
    datapath = ev.msg.datapath
    ofproto = datapath.ofproto
    parser = datapath.ofproto_parser
    # Instalacion de la entrada de flujo en la tabla miss
    match = parser.OFPMatch()
    actions = [parser.OFPActionOutput(ofproto.OFPP_CONTROLLER,
                                      ofproto.OFPCML_NO_BUFFER)]

    self.add_flow(datapath, 0, match, actions)

def add_flow(self, datapath, priority, match, actions, buffer_id=None):
    ofproto = datapath.ofproto
    parser = datapath.ofproto_parser
    # Construcción del mensaje Flow Mod
    inst = [parser.OFPInstructionActions(ofproto.OFPIT_APPLY_ACTIONS,
                                         actions)]

    if buffer_id:
        mod = parser.OFPFlowMod(datapath=datapath, buffer_id=buffer_id,
                                priority=priority, match=match, instructions=inst)
    else:
        mod = parser.OFPFlowMod(datapath=datapath, priority=priority,
                                match=match, instructions=inst)

    datapath.send_msg(mod)

@set_ev_cls(ofp_event.EventOFPPacketIn, MAIN_DISPATCHER)
def _packet_in_handler(self, ev):
    # En esta parte se puede modificar la "miss_send_length" del switch
    if ev.msg.msg_len < ev.msg.total_len:
        self.logger.debug("Paquete truncado: solo %s de %s bytes",
                          ev.msg.msg_len, ev.msg.total_len)

    msg = ev.msg
    datapath = msg.datapath
    ofproto = datapath.ofproto
    parser = datapath.ofproto_parser

    # Se obtiene el numero de puerto recibido del mensaje Packet-In
    in_port = msg.match['in_port']

    # Analizar el paquete recibido para verificar que sea de tipo ethernet
    pkt = packet.Packet(msg.data)
    eth = pkt.get_protocols(ethernet.ethernet)[0]
    if eth.ethertype == ether_types.ETH_TYPE_LLDP:
        # Ignorar el paquete LLDP Link Layer Discovery Protocol
        return

    dst = eth.dst
    src = eth.src

    # Obtener un identificador de ruta de datos para identificar el switch
    dpid = datapath.id
    self.mac_to_port.setdefault(dpid, {})

    self.logger.info("packet in dpid:%s puerto_in[%s] origen:%s -> destino:%s",
                     dpid, in_port, src, dst)

    # Aprender una direccion MAC para evitar FLOOD(inundacion) la proxima vez
    self.mac_to_port[dpid][src] = in_port

    # Si la direccion MAC destino ya esta aprendida direccionar al puerto de
    # salida al paquete, de lo contrario realizar una inundacion
    if dst in self.mac_to_port[dpid]:
        out_port = self.mac_to_port[dpid][dst]
    else:
        out_port = ofproto.OFPP_FLOOD

```

```

# Construir una lista de acciones para el puerto de salida
actions = [parser.OFPActionOutput(out_port)]

# Instalar un flujo para evitar packet_in la siguiente vez
if out_port != ofproto.OFPP_FLOOD:
    match = parser.OFPMatch(in_port=in_port, eth_dst=dst)
    # verificar si se tiene un buffer_id valido
    if msg.buffer_id != ofproto.OFP_NO_BUFFER:
        self.add_flow(datapath, 1, match, actions, msg.buffer_id)
        return
    else:
        self.add_flow(datapath, 1, match, actions)
data = None
if msg.buffer_id == ofproto.OFP_NO_BUFFER:
    data = msg.data

out = parser.OFPacketOut(datapath=datapath, buffer_id=msg.buffer_id,
                        in_port=in_port, actions=actions, data=data)
datapath.send_msg(out)

```

Aunque ya se tiene en el código mismo una breve descripción del funcionamiento de cada parte del código, ahora se ampliara la explicación de forma concisa por cada proceso.

Las primeras líneas hacen referencia a la importación de las librerías y módulos que se utilizaran en la aplicación, y que por defecto vienen con la instalación del controlador Ryu.

Código 2.1. Importación de librerías y módulos requeridos.

```

# Importacion de las librerias requeridas
from ryu.base import app_manager
from ryu.controller import ofp_event
from ryu.controller.handler import CONFIG_DISPATCHER, MAIN_DISPATCHER
from ryu.controller.handler import set_ev_cls
from ryu.ofproto import ofproto_v1_3
from ryu.lib.packet import packet
from ryu.lib.packet import ethernet
from ryu.lib.packet import ether_types

```

Luego de la importación de los elementos necesarios, se inicia la clase SwitchProyecto en el que se especifica la versión 1.3 del protocolo OpenFlow (la cual se utilizará en todo el proyecto), se define el uso de las variables a nivel global y también el diccionario mac_to_port, que ayudará a gestionar la tabla de direcciones MAC dentro del switch.

Código 2.2. Inicialización de la clase SwitchProyecto y definición de variables.

```

class SwitchProyecto(app_manager.RyuApp):
    # Definicion de la version OpenFlow
    OFP_VERSIONS = [ofproto_v1_3.OFP_VERSION]

    def __init__(self, *args, **kwargs):
        super(SwitchProyecto, self).__init__(*args, **kwargs)
        # Inicializacion de la tabla MAC address
        self.mac_to_port = {}

```

Cuando un mensaje OpenFlow es recibido por el controlador Ryu, se genera un evento para responder correctamente al mensaje entrante, por lo que existe un manejador o gestor de eventos,

este gestor de eventos define una función de acuerdo a sus respectivos argumentos, y usa el llamado decorador `ryu.controller.handler.set_ev_cls`. En la presente sección `set_ev_cls` está esperando recibir las características del switch.

Luego se define la función `switch_features_handler`, la cual se encargará de procesar las características del switch creando una entrada de flujo en la tabla miss.

La variable `datapath`, establece el camino de datos para la comunicación actual con el switch y la emisión del evento correspondiente al mensaje recibido.

Los atributos `ofproto` y `ofproto_parser`, indican los módulos que serán utilizados por el controlador según la versión correspondiente de OpenFlow (en este caso la versión 1.3).

Como se observa en la codificación, este flujo tiene la más baja prioridad que es cero y es comparada con todos los paquetes entrantes, en la instrucción de este flujo se especifica la acción de salida hacia el puerto del controlador con la instancia de la clase `OFPACTIONOutput`.

Entonces se especifica al controlador como el destino de salida, y con la instrucción `OFPCML_NO_BUFFER` se especifica el tamaño máximo de los paquetes que son enviados al controlador con un respectivo orden.

Finalmente, se especifica la prioridad cero en este flujo, y el método `add_flow()` es ejecutado para enviar el mensaje modificador de flujos.

Código 2.3. Definición de la función `switch_features_handler`.

```
@set_ev_cls(ofp_event.EventOFPSwitchFeatures, CONFIG_DISPATCHER)
def switch_features_handler(self, ev):
    datapath = ev.msg.datapath
    ofproto = datapath.ofproto
    parser = datapath.ofproto_parser

    # Instalacion de la entrada de flujo en la tabla miss
    match = parser.OFPMatch()
    actions = [parser.OFPACTIONOutput(ofproto.OFPP_CONTROLLER,
                                     ofproto.OFPCML_NO_BUFFER)]

    self.add_flow(datapath, 0, match, actions)
```

Para cada entrada de flujo existe una coincidencia que indica las condiciones del paquete y una instrucción que indica la gestión del paquete, el nivel de prioridad y el tiempo efectivo.

Dentro de la variable `inst`, tenemos la clase `OFPT_APPLY_ACTIONS` que se utiliza para configurar las instrucciones que especifican la acción requerida inmediatamente, dependiendo si contamos con un `buffer_id` o no.

Luego se instala una entrada de flujo a la tabla miss usando el mensaje Flow Mod, la clase correspondiente al mensaje Flow Mod es la clase `OFPPFlowMod`, esta clase es generada y el mensaje es enviada al switch usando el método `datapath.send_msg()`.

Luego que la entrada de flujo es añadida en la tabla miss, el switch está listo para recibir entradas de paquetes y enviarlos al controlador.

Código 2.4. Definición de la función *add_flow* para crear el mensaje Flow Mod.

```
def add_flow(self, datapath, priority, match, actions, buffer_id=None):
    ofproto = datapath.ofproto
    parser = datapath.ofproto_parser
    # Construcción del mensaje Flow Mod
    inst = [parser.OFPInstructionActions(ofproto.OFPIT_APPLY_ACTIONS,
                                         actions)]

    if buffer_id:
        mod = parser.OFPPFlowMod(datapath=datapath, buffer_id=buffer_id,
                                  priority=priority, match=match,
                                  instructions=inst)
    else:
        mod = parser.OFPPFlowMod(datapath=datapath, priority=priority,
                                  match=match, instructions=inst)

    datapath.send_msg(mod)
```

Por último, se expone la función manejadora de paquetes de entrada `packet_in_handler`, se inicia con la creación del evento que recibirá los nuevos paquetes que ingresan, se define la función correspondiente a la recepción y manejo de entradas de paquetes con varios atributos que usa la clase `OFPPacketIn`, como ser:

Tabla 8. *Atributos de la clase OFPPacketIn.*

Atributo	Explicación
Match	Se compara la información de los paquetes recibidos.
Data	Datos binarios que indican los propios paquetes recibidos.
Local_len	Longitud de datos de los paquetes recibidos.
buffer_id	Indica si los paquetes recibidos son almacenados en el switch con una ID o de lo contrario son almacenados en una memoria intermedia sin ID.

Nota: Elaboración propia recopilada de diversas fuentes.

Luego se actualiza la tabla de direcciones MAC, para lo cual se registra el puerto de entrada en la variable `in_port`, y se utiliza la librería `ethertype` de `Ryu` para analizar la cabecera del paquete entrante para comprobar si es de tipo ethernet para continuar con el proceso o si no es de tipo ethernet lo rechazara. Continuando con el proceso en caso de ser un paquete ethernet serán registrados: su identificador de camino de datos, su puerto de ingreso, su dirección MAC origen y la dirección MAC destino, y con este proceso se actualiza la tabla de direcciones MAC del switch.

Para soportar la conexión con múltiples switches, la tabla de direcciones MAC está diseñada

para ser manejada por cada switch independientemente. El identificador de la ruta de datos o `datapath.id` se utiliza para identificar a cada switch.

Para seleccionar el puerto de salida del paquete, se utiliza la tabla de direcciones MAC si existe dentro de esta y si no se encuentra se genera la instancia de la clase `OFPACTIONOutput` especificando el proceso de inundación (`OFPP_FLOOD`), entonces si la dirección MAC destino es encontrada, una entrada es añadida a la tabla de flujos del switch.

Finalmente, independientemente de si la dirección MAC destino se encuentra en la tabla de direcciones MAC, al final se emite el mensaje `Packet-Out` y los paquetes recibidos son transferidos según las reglas impuestas por el controlador, la clase que corresponde a este mensaje es `OFPPacketOut`.

Código 2.5. Definición de la función `packet in handler`.

```
@set_ev_cls(ofp_event.EventOFPPacketIn, MAIN_DISPATCHER)
def _packet_in_handler(self, ev):
    # En esta parte se puede modificar la "miss_send_length" del switch
    if ev.msg.msg_len < ev.msg.total_len:
        self.logger.debug("Paquete truncado: solo %s de %s bytes",
                          ev.msg.msg_len, ev.msg.total_len)

    msg = ev.msg
    datapath = msg.datapath
    ofproto = datapath.ofproto
    parser = datapath.ofproto_parser

    # Se obtiene el numero de puerto recibido del mensaje Packet-In
    in_port = msg.match['in_port']

    # Analizar el paquete recibido para verificar que sea de tipo ethernet
    pkt = packet.Packet(msg.data)
    eth = pkt.get_protocols(ethernet.ethernet)[0]

    if eth.ethertype == ether_types.ETH_TYPE_LLDP:
        # Ignorar el paquete LLDP Link Layer Discovery Protocol
        return
    dst = eth.dst
    src = eth.src
    # Obtener un identificador de ruta de datos para identificar el switch
    dpid = datapath.id
    self.mac_to_port.setdefault(dpid, {})

    self.logger.info("packet in dpid:%s puerto_in[%s] origen:%s -> destino:%s",
                     dpid, in_port, src, dst)

    # Aprender una direccion MAC para evitar FLOOD(inundacion) la proxima vez
    self.mac_to_port[dpid][src] = in_port

    # Si la direccion MAC destino ya esta aprendida direccionar al puerto de
    # salida al paquete, de lo contrario realizar una inundacion
    if dst in self.mac_to_port[dpid]:
        out_port = self.mac_to_port[dpid][dst]
    else:
        out_port = ofproto.OFPP_FLOOD
    # Construir una lista de acciones para el puerto de salida
```

```

actions = [parser.OFPActionOutput(out_port)]

# Instalar un flujo para evitar packet_in la siguiente vez
if out_port != ofproto.OFPP_FLOOD:
    match = parser.OFPMatch(in_port=in_port, eth_dst=dst)
    # verificar si se tiene un buffer_id valido
    if msg.buffer_id != ofproto.OFP_NO_BUFFER:
        self.add_flow(datapath, 1, match, actions, msg.buffer_id)
        return
    else:
        self.add_flow(datapath, 1, match, actions)
data = None
if msg.buffer_id == ofproto.OFP_NO_BUFFER:
    data = msg.data

out = parser.OFPPacketOut(datapath=datapath, buffer_id=msg.buffer_id,
                          in_port=in_port, actions=actions, data=data)
datapath.send_msg(out)

```

Antes de comenzar con el desarrollo del segundo módulo, se describirá brevemente el funcionamiento básico que tiene el protocolo de árbol de expansión o más conocido en el área de redes simplemente como STP del inglés Spanning Tree Protocol, que se tomó como referencia para desarrollar el código, creando funciones que realicen una tarea similar, y contando con el apoyo de las librerías y módulos que tiene el controlador Ryu.

Entonces para evitar confusiones en cuanto a las versiones y nombres que tiene este protocolo, el presente estudio se basó en la documentación del Instituto de Ingeniería Eléctrica y Electrónica (IEEE - Institute of Electrical and Electronics Engineers) acerca del protocolo de árbol de expansión que está expuesta en el estándar 802.1D.

El protocolo de árbol de expansión se ejecuta en la capa 2 del modelo OSI, y en el presente caso se lo aplicará a la red virtualizada por Mininet, su objetivo es asegurar que exista una sola ruta lógica entre todos los dispositivos interconectados para evitar bucles en la topología de red.

Su funcionamiento se basa en realizar un bloqueo de forma intencional a un puerto del switch que forma una ruta redundante y que pueda ocasionar un bucle dentro de la red, un puerto se considera bloqueado cuando el tráfico de la red no puede ingresar ni salir del puerto. Esto no incluye las tramas de Unidad de Datos del Protocolo Puente (BPDU - Bridge Protocol Data Unit). Si en algún caso es necesario compensar la falla de un cable de red o de un switch, este protocolo vuelve a calcular las rutas y desbloquea los puertos necesarios creando de nuevo un camino lógico sin redundancia.

Para alcanzar una estabilidad en la red y comenzar a enviar tramas de datos, el protocolo de árbol de expansión realiza el proceso que será explicado a continuación:

Al encender los switches no poseen ningún conocimiento sobre la red por lo que sus puertos se encuentran en el estado bloqueado, entonces comienzan a intercambiar mensajes de datos a través de las BPDU's y los puertos de los switches pasan al estado escucha, utilizando la dirección MAC de ese puerto como dirección de origen y ya que no sabe de la existencia de otro switch las BPDU's son enviadas con la dirección universal reservada y conocida como Grupo de Direcciones Puente, que tiene el formato 01:80:c2:00:00:0X, donde la "X" es un número que tiene un intervalo del 0 a F en números hexadecimales. El intercambio de estos mensajes tiene la finalidad de conseguir una topología estable, libre de bucles y obtener información constante sobre el estado de la red.

Tabla 9. *Parámetros y formato de la BPDU.*

Nº	Nombre del campo	Bytes
1	Protocol Identifier	2
2	Protocol Version Identifier	1
3	BPDU type	1
4	Flags	1
5	Root Identifier	8
6	Root Path Cost	4
7	Bridge Identifier	8
8	Port Identifier	2
9	Message Age	2
10	Max Age	2
11	Hello Time	2
12	Forward Delay	2

Fuente: Estándar IEEE 802.1D.

Existen dos tipos de BPDU's, una que es utilizada para el cálculo del STP, y la otra que es utilizada para notificar o anunciar cambios en la topología de red.

La primera misión de las BPDU's consiste en la elección del switch denominado Puente raíz o más conocido por su nombre en inglés como Root Bridge, para lo cual se necesita el Identificador Puente (Bridge Identifier) de cada switch, el cual está formado por 8 Bytes.

Tabla 10. *Formato del campo Identificador Puente.*

Prioridad del Puente	Dirección MAC
2 Bytes	6 Bytes

Fuente: Estándar IEEE 802.1D.

El valor de la prioridad del puente (Bridge Priority) está comprendido entre 0 y 65535, siendo el valor por defecto el 32768 o en hexadecimal 8000.

La dirección MAC está formado por 6 bytes y asignada según el fabricante del dispositivo.

En el proceso de búsqueda del Root Bridge participan todos los switches de la red, enviándose tramas de BPDU entre ellos cada 2 segundos, y el que tenga más bajo su campo Identificador Puente será nombrado Root Bridge y los demás serán nombrados Bridges Not Root.

El siguiente paso es asignar funciones a los puertos de cada switch, dentro de este protocolo se tienen tres tipos:

Puerto Raíz (Root Port): Es el puerto de cada bridge que se encuentra en el camino más corto al Root Bridge, o técnicamente tendrá la ruta de menor coste hasta el Root Bridge.

El puerto raíz siempre apunta hacia el Root Bridge elegido, el protocolo de árbol de expansión utiliza el concepto de coste para determinar y seleccionar un puerto raíz, lo que significa evaluar el coste de la ruta o camino (Root Path Cost). Este valor es el coste acumulativo de todos los enlaces hasta el Root Bridge. Como se observa en la Tabla 11, cuanto mayor sea el ancho de banda del enlace, menor será el coste del enlace. El estándar IEEE 802.1D define un Root Path Cost de 1000 Mbps dividido por el ancho de banda del enlace en Mbps.

Tabla 11. *Costes de camino para el STP.*

Enlace	Valor recomendado	Rango recomendado
10 Mbps	100	50-600
100 Mbps	19	10-60
1 Gbps	4	3-10
10 Gbps	2	1-5

Fuente: Estándar IEEE 802.1D.

Puerto Designado (Designated Port): El puerto designado es el que presenta el mejor camino hacia el Root Bridge.

Puerto No Designado (No Designated Port): Es el puerto teóricamente bloqueado para evitar bucles, pero recibe BPDU's y esta alerta para ser activado cuando exista algún fallo o cambio dentro de la topología de la red.

Estos puertos son nombrados con uno de estos cinco estados, según corresponda:

- **BLOQUEADO:** No reenvía tramas de datos, pero si procesan mensajes de BPDU's. Es el estado por defecto de los puertos cuando un switch se enciende y su función es la de prevenir bucles.
- **ESCUCHA:** Recibe, analiza y envía BPDU's para asegurarse que no existen bucles dentro de la topología de red.

- **APRENDE:** Al igual que el estado ESCUCHA, recibe, analiza y envía BPDU's, pero es aquí donde se comienza a armar la tabla de direcciones MAC. Aunque en este estado aún no se reenvían tramas de datos.
- **REENVÍA:** Envía y recibe todas las tramas de datos.
- **DESHABILITADO:** Es un puerto deshabilitado administrativamente y que no participará en el STP. Un puerto en este estado es como si no existiera para el STP.

Por último, se describirán otros campos principales que conforman las BPDU's conocidos como temporizadores BPDU, los cuales indican la cantidad de tiempo que un puerto permanece en uno de los cinco estados descritos anteriormente. Es importante aclarar que sólo el switch designado como Root Bridge puede enviar información a través del árbol para ajustar estos temporizadores.

Tiempo de saludo (Hello Time): El tiempo de saludo es el intervalo de tiempo en el cual las configuraciones de las BPDU's se envían desde el Root Bridge. La configuración se realiza únicamente en el Root Bridge y determina el temporizador Hello de todos los demás switches. Aun así, todos los demás switches tienen un temporizador local incorporado utilizado para temporizar las BPDU's cuando hay cambios de topología. El valor recomendado es 2 segundos.

Retraso en el envío (Forward Delay): Es el tiempo que transcurre en pasar del estado ESCUCHA al estado APRENDE, como también del estado APRENDE al estado REENVÍA.

Este valor es igual a 15 segundos de manera predeterminada para cada estado.

Antigüedad máxima (Max Age): El temporizador de antigüedad máxima es el intervalo de tiempo que un switch guarda una BPDU antes de descartarla. Mientras se está ejecutando STP cada puerto del switch mantiene un registro de la mejor BPDU que ha escuchado. En caso de que el puerto del switch pierda contacto con el origen de esta BPDU el switch asume que ha ocurrido algún cambio en la topología, cuando el período de antigüedad máxima termine la BPDU es eliminada. su valor por defecto es 20 segundos.

Tabla 12. *Parámetros de los temporizadores de BPDU.*

Parámetro	Valor recomendado [s]	Intervalo [s]
Hello Time	2	1-10
Forward Delay	15	4-30
Max Age	20	6-40

Fuente: Estándar IEEE 802.1D.

Entonces, según el estándar IEEE 802.1D cada bridge cumplirá con las siguientes relaciones:

$$2 * (\text{Forward Delay} - 1) \geq \text{Max Age} \geq 2 * (\text{Hello Time} + 1)$$

Cuando existe algún cambio en la red se utilizan las BPDU's de notificación de cambios en la topología, formados por los 3 primeros campos de la Tabla 9. Puede considerarse un cambio de topología cuando un switch cambia el estado de un puerto a REENVIA, o lo hace desde uno de los estados REENVIA/APRENDE al estado BLOQUEADO, el switch envía una BPDU a través de su puerto raíz para que lleguen al Root Bridge para informar del cambio de topología. El switch continúa enviando estas BPDU's en intervalos de Hello Time hasta recibir un ACK (acknowledgement - Admitir) desde su vecino. Cuando los vecinos reciben las BPDU's de cambio de topología las propagan hacia el Root Bridge y envían sus propios mensajes ACK. Lo mismo ocurre con el Root Bridge, pero además éste agrega la etiqueta llamada Topology Change para señalar el cambio de topología. Esta información sirve para que los otros switches acorten los temporizadores de 300 segundos al tiempo del Forward Delay, que por recomendación es 15 segundos.

Los cambios de topología pueden considerarse de tres maneras:

Directos, son aquellos que pueden ser detectados directamente en la interfaz de algún switch, puede ser cuando cae un enlace, cambia la topología de red y todos los switches son informados para que se realice un nuevo cálculo del STP.

Indirectos, son los cambios que no involucran directamente a los puertos de los switches pero que afectan el funcionamiento de la topología, como por ejemplo el filtrado de algún firewall que no deja pasar las BPDU's hasta su destino, entonces se puede considerar como un fallo indirecto porque no cambia la topología, pero existe la ausencia de BPDU.

Insignificantes, cuando existen modificaciones que no ameritan un cálculo nuevo del STP, pero si son tomados en cuenta para informar al Root Bridge, un ejemplo de este tipo es cuando se cambia un host de un puerto a otro puerto del mismo switch.

Para realizar la respectiva codificación del protocolo de árbol de expansión se guiará del diagrama de flujo de la Figura 36, que se realizó con el objetivo de simplificar el procedimiento que se expuso sobre el funcionamiento de este protocolo.

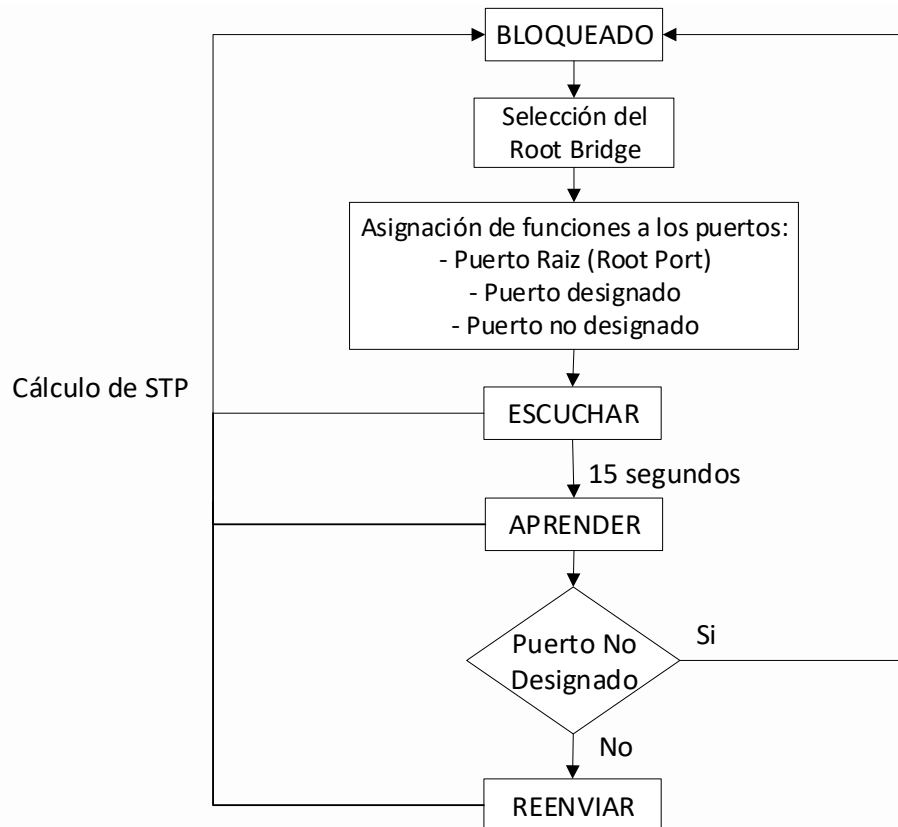


Figura 36. Procedimiento para el cálculo del protocolo de árbol de expansión (STP).
Fuente: Elaboración propia recopilada de diversas fuentes.

Con las características y funcionamiento descritos sobre el cálculo del STP se expondrá la codificación que realiza un proceso similar, y que trabaje en conjunto con el primer módulo desarrollado llamado `switch_proyecto.py`.

El segundo módulo desarrollado para proporcionar el cálculo del STP, se llamará “servicioSTP.py” cuya codificación es la siguiente:

Código 3. Codificación para el cálculo del protocolo de árbol de expansión (STP).

```

# <<< CODIFICACION PARA EL CALCULO DEL PROTOCOLO
#   DE ARBOL DE EXPANSION (STP) >>>

# Importacion de librerias y modulos requeridos
from ryu.base import app_manager
from ryu.controller import ofp_event
from ryu.controller.handler import CONFIG_DISPATCHER, MAIN_DISPATCHER
from ryu.controller.handler import set_ev_cls
from ryu.ofproto import ofproto_v1_3
from ryu.lib import dpid as dpid_lib
from ryu.lib import stplib
from ryu.lib.packet import packet
from ryu.lib.packet import ethernet
from ryu.app import switch_proyecto

class SwitchProyecto(switch_proyecto.SwitchProyecto):

```

```

# Configuración de la versión OpenFlow y de la librería stp
OFP_VERSIONS = [ofproto_v1_3.OFP_VERSION]
_CONTEXTS = {'stplib': stplib.Stp}

def __init__(self, *args, **kwargs):
    # Se inician las variables locales y globales
    super(SwitchProyecto, self).__init__(*args, **kwargs)
    self.mac_to_port = {}
    self.stp = kwargs['stplib']

    # Configuración de la librería stplib
    config = {dpid_lib.str_to_dpid('0000000000000001'):
               {'bridge': {'priority': 0x8000}},
              dpid_lib.str_to_dpid('0000000000000002'):
               {'bridge': {'priority': 0x9000}},
              dpid_lib.str_to_dpid('0000000000000003'):
               {'bridge': {'priority': 0xa000}},
              dpid_lib.str_to_dpid('0000000000000004'):
               {'bridge': {'priority': 0xb000}},
              dpid_lib.str_to_dpid('0000000000000005'):
               {'bridge': {'priority': 0xc000}}}
    self.stp.set_config(config)

def delete_flow(self, datapath):
    ofproto = datapath.ofproto
    parser = datapath.ofproto_parser

    # Verificación del destino en la tabla de direcciones MAC
    # y reemplazo con el envío del mensaje Flow Mod
    for dst in self.mac_to_port[datapath.id].keys():
        match = parser.OFPMatch(eth_dst=dst)
        mod = parser.OFPFlowMod(
            datapath, command=ofproto.OFPFC_DELETE,
            out_port=ofproto.OFPP_ANY, out_group=ofproto.OFPG_ANY,
            priority=1, match=match)
        datapath.send_msg(mod)

@set_ev_cls(stplib.EventPacketIn, MAIN_DISPATCHER)
def _packet_in_handler(self, ev):
    # Recolección de información de la entrada de flujo
    msg = ev.msg
    datapath = msg.datapath
    ofproto = datapath.ofproto
    parser = datapath.ofproto_parser
    in_port = msg.match['in_port']

    pkt = packet.Packet(msg.data)
    eth = pkt.get_protocols(ethernet.ethernet)[0]
    dst = eth.dst
    src = eth.src

    dpid = datapath.id
    self.mac_to_port.setdefault(dpid, {})

    self.logger.info("packet in dpid:%s Puerto_in [%s] origen:%s -> destino:%s",
        dpid, src, dst, in_port)

    # Aprender una dirección MAC para evitar inundación la siguiente vez
    self.mac_to_port[dpid][src] = in_port

    # Comparación con la tabla de direcciones MAC
    if dst in self.mac_to_port[dpid]:
        out_port = self.mac_to_port[dpid][dst]

```

```

else:
    out_port = ofproto.OFPP_FLOOD

    # Accion tomada para la salida del flujo
    actions = [parser.OFPACTIONOutput(out_port)]

    # Instalar un flujo para evitar packet_in la siguiente vez
    if out_port != ofproto.OFPP_FLOOD:
        match = parser.OFPMATCH(in_port=in_port, eth_dst=dst)
        self.add_flow(datapath, 1, match, actions)

    data = None
    if msg.buffer_id == ofproto.OFP_NO_BUFFER:
        data = msg.data

    out = parser.OFPPACKETOut(datapath=datapath, buffer_id=msg.buffer_id,
                              in_port=in_port, actions=actions, data=data)

    datapath.send_msg(out)

@set_ev_cls(stplib.EventTopologyChange, MAIN_DISPATCHER)
def _topology_change_handler(self, ev):
    # Con la ayuda de la libreria dpid_lib se gestiona un cambio en la topologia
    dp = ev.dp
    dpid_str = dpid_lib.dpid_to_str(dp.id)
    msg = 'cambio detectado en la topologia >>> re-calculando STP...'
    self.logger.debug("[dpid=%s] %s", dpid_str, msg)

    if dp.id in self.mac_to_port:
        self.delete_flow(dp)
        del self.mac_to_port[dp.id]

@set_ev_cls(stplib.EventPortStateChange, MAIN_DISPATCHER)
def _port_state_change_handler(self, ev):
    # Registro de estados de los puertos en STP
    dpid_str = dpid_lib.dpid_to_str(ev.dp.id)
    of_state = {stplib.PORT_STATE_DISABLE: 'DESHABILITADO',
                stplib.PORT_STATE_BLOCK: 'BLOQUEADO',
                stplib.PORT_STATE_LISTEN: 'ESCUCHA',
                stplib.PORT_STATE_LEARN: 'APRENDE',
                stplib.PORT_STATE_FORWARD: 'REENVIA'}
    self.logger.debug("[dpid=%s][puerto=%d] Estado=%s",
                      dpid_str, ev.port no, of_state[ev.port_state])

```

Con las primeras líneas de código se importan las librerías y módulos requeridos, es importante notar que también se está importando el primer módulo llamado `switch_proyecto.py`, que se lo utilizará de manera conjunta con este.

Código 3.1. Importación de librerías y módulos requeridos para la codificación.

```

# Importacion de librerias y modulos requeridos
from ryu.base import app_manager
from ryu.controller import ofp_event
from ryu.controller.handler import CONFIG_DISPATCHER, MAIN_DISPATCHER
from ryu.controller.handler import set_ev_cls
from ryu.ofproto import ofproto_v1_3
from ryu.lib import dpid as dpid_lib
from ryu.lib import stplib
from ryu.lib.packet import packet
from ryu.lib.packet import ethernet
from ryu.app import switch_proyecto

```

Como primera instancia se heredan las variables del módulo `switch_proyecto.py` para que también se las pueda utilizar aquí, y se define la versión 1.3 del protocolo OpenFlow.

Una aplicación que hereda `ryu.base.app_manager.RyuApp` como en el presente caso, comienza otras aplicaciones utilizando subprocesos mediante la especificación de las mismas en el diccionario `_CONTEXTS` de Ryu, en este caso la clase `stplib` de la librería STP se configura en `_CONTEXTS` con el nombre de `stplib`.

Código 3.2. Configuración de variables de inicio y de la librería stp como subproceso.

```
class SwitchProyecto(switch_proyecto.SwitchProyecto):
    # Configuración de la versión OpenFlow y de la librería stp
    OFP_VERSIONS = [ofproto_v1_3.OFP_VERSION]
    _CONTEXTS = {'stplib': stplib.Stp}
```

Luego se inician las variables heredadas, se crea la lista de argumentos de la variable `stp` y se asignan valores de prioridad para la elección de los bridges utilizados por la librería `stp`, recordando que por defecto se tiene el 32768 o 8000h como prioridad en los dispositivos físicos y se lo define de la misma forma en la red virtual, incrementando en 1000h para la siguiente prioridad.

Código 3.3. Inicialización de variables y configuración de valores para la librería stp.

```
def __init__(self, *args, **kwargs):
    # Se inician las variables locales y globales
    super(SwitchProyecto, self).__init__(*args, **kwargs)
    self.mac_to_port = {}
    self.stp = kwargs['stplib']
    # Configuración de la librería stplib
    config = {dpid_lib.str_to_dpid('0000000000000001'):
              {'bridge': {'priority': 0x8000}},
             dpid_lib.str_to_dpid('0000000000000002'):
              {'bridge': {'priority': 0x9000}},
             dpid_lib.str_to_dpid('0000000000000003'):
              {'bridge': {'priority': 0xa000}},
             dpid_lib.str_to_dpid('0000000000000004'):
              {'bridge': {'priority': 0xb000}},
             dpid_lib.str_to_dpid('0000000000000005'):
              {'bridge': {'priority': 0xc000}}}
    self.stp.set_config(config)
```

La siguiente función es definida como parte del evento de cambios en la topología, y es llamada con el objetivo de inicializar el registro de direcciones MAC y entradas de flujo.

Código 3.4. Creación del mensaje Flow Mod con prioridad igual a uno.

```
def delete_flow(self, datapath):
    ofproto = datapath.ofproto
    parser = datapath.ofproto_parser
    # Verificación del destino en la tabla de direcciones MAC
    # y reemplazo con el envío del mensaje Flow Mod
    for dst in self.mac_to_port[datapath.id].keys():
        match = parser.OFPMatch(eth_dst=dst)
        mod = parser.OFPFlowMod(
            datapath, command=ofproto.OFPFC_DELETE,
            out_port=ofproto.OFPP_ANY, out_group=ofproto.OFPG_ANY,
            priority=1, match=match)
```

```
datapath.send_msg(mod)
```

El manejador de eventos en este caso tiene un funcionamiento muy similar al manejador de eventos del switch del primer módulo, con la diferencia que ahora se utilizará `stplib.EventPacketIn` que tiene la virtud de recibir paquetes BPDU, que son definidos en la librería `stp`.

Código 3.5. Manejador de paquetes entrantes usando la librería `stplib.EventPacketIn`.

```
@set_ev_cls(stplib.EventPacketIn, MAIN_DISPATCHER)
def _packet_in_handler(self, ev):
    # Recoleccion de informacion de la entrada de flujo
    msg = ev.msg
    datapath = msg.datapath
    ofproto = datapath.ofproto
    parser = datapath.ofproto_parser
    in_port = msg.match['in_port']
    pkt = packet.Packet(msg.data)
    eth = pkt.get_protocols(ethernet.ethernet)[0]
    dst = eth.dst
    src = eth.src
    dpid = datapath.id
    self.mac_to_port.setdefault(dpid, {})
    self.logger.info("packet in dpid:%s Puerto_in [%s] origen:%s -> destino:%s",
dpid, src, dst, in_port)
    # Aprender una direccion MAC para evitar inundacion la siguiente vez
    self.mac_to_port[dpid][src] = in_port
    # Comparacion con la tabla de direcciones MAC
    if dst in self.mac_to_port[dpid]:
        out_port = self.mac_to_port[dpid][dst]
    else:
        out_port = ofproto.OFPP_FLOOD
    # Accion tomada para la salida del flujo
    actions = [parser.OFPActionOutput(out_port)]

    # Instalar un flujo para evitar packet_in la siguiente vez
    if out_port != ofproto.OFPP_FLOOD:
        match = parser.OFPMatch(in_port=in_port, eth_dst=dst)
        self.add_flow(datapath, 1, match, actions)

    data = None
    if msg.buffer_id == ofproto.OFP_NO_BUFFER:
        data = msg.data

    out = parser.OFPPacketOut(datapath=datapath,buffer_id=msg.buffer_id,
in_port=in_port,actions=actions,data=data)

    datapath.send_msg(out)
```

Para gestionar el evento de cambios en la topología de red se utilizará la librería `stplib.EventTopologyChange`, y se iniciará los registros de flujos llamando a la función `delete_flow()` para reiniciar el proceso de cálculo del STP.

Código 3.6. Función que maneja los cambios de topología.

```
@set_ev_cls(stplib.EventTopologyChange, MAIN_DISPATCHER)
def _topology_change_handler(self, ev):
    # Con la ayuda de la librería dpid_lib se gestiona un cambio en la topología
    dp = ev.dp
    dpid_str = dpid_lib.dpid_to_str(dp.id)
    msg = 'cambio detectado en la topología >>> re-calculando STP...'
    self.logger.debug("[dpid=%s] %s", dpid_str, msg)

    if dp.id in self.mac_to_port:
        self.delete_flow(dp)
        del self.mac_to_port[dp.id]
```

Con la librería `stplib.EventPortStateChange`, se reciben los cambios de estado en los puertos y se registra la depuración de los estados de cada puerto de salida.

Código 3.7. Recepción y registro de estados de los puertos de salida.

```
@set_ev_cls(stplib.EventPortStateChange, MAIN_DISPATCHER)
def _port_state_change_handler(self, ev):
    # Registro de estados de los puertos en STP
    dpid_str = dpid_lib.dpid_to_str(ev.dp.id)
    of_state = {stplib.PORT_STATE_DISABLE: 'DESHABILITADO',
                stplib.PORT_STATE_BLOCK: 'BLOQUEADO',
                stplib.PORT_STATE_LISTEN: 'ESCUCHA',
                stplib.PORT_STATE_LEARN: 'APRENDE',
                stplib.PORT_STATE_FORWARD: 'REENVIA'}
    self.logger.debug("[dpid=%s][puerto=%d] Estado=%s",
                      dpid_str, ev.port_no, of_state[ev.port_state])
```

Cabe notar que el desarrollo de este código se lo realizó heredando variables del módulo `switch_proyecto.py` que se lo expuso en primera instancia, y también se utilizó varias herramientas de la librería `stplib.py`.

Por otro lado, el manejo de las BPDUs se los procesa mediante el script `bpdu.py` que están incorporados en las librerías del controlador Ryu.

3.3. Costo del Proyecto

La implementación de este tipo de tecnología es un aporte en la administración de la red ya que los servicios y aplicaciones pueden ser desarrolladas de acuerdo a los requerimientos y necesidades del lugar, convirtiéndose de mediano a largo plazo en una reducción de costos en cuanto a Capex y Opex (como se mencionó en el primer capítulo).

En el presente trabajo se realizó la implementación virtual de esta tecnología utilizando exclusivamente software libre, por lo que no se hizo ningún gasto relacionado con la compra de software o de equipos de interconexión, y se puede decir que el recurso que más se invirtió fue el tiempo en la adquisición de nuevos conocimientos para comprender los conceptos utilizados en esta tecnología.

La Tabla 13, detalla el costo total aproximado que se invirtió en el presente proyecto.

Tabla 13. *Costo aproximado del proyecto.*

Herramientas utilizadas	Tiempo estimado	Costo [Bs]
Computador personal (Notebook)	8 meses	1.500
Electricidad (60 Bs x mes)	8 meses	480
Curso básico de Python 2.7	1 mes	350
Conexión a internet (143 Bs x mes)	6 meses	858
Sistema Operativo Ubuntu 16.04 LTS	6 meses	0
Emulador Mininet 2.2.2	5 meses	0
Controlador Ryu	5 meses	0
Wireshark	4 meses	0
Otras Librerías o programas	4 meses	0
Implementación y pruebas (1.500 Bs x mes)	8 meses	12.000
	Total	15.188

Fuente: Elaboración propia.

Hasta este punto se acaba de exponer la obtención, instalación, configuración y desarrollo de todas las herramientas y el software necesario para la implementación virtual de la arquitectura de red definida por software, la cual será probada y analizada en el siguiente capítulo.

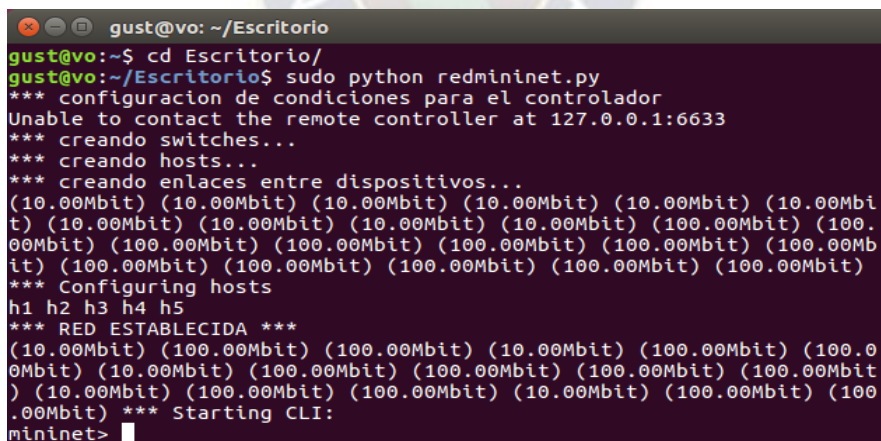
Capítulo 4.- SIMULACIÓN Y ANÁLISIS DE FUNCIONAMIENTO DE LA SDN

En este capítulo se expondrá la simulación de la red definida por software mediante la ejecución de los módulos desarrollados anteriormente, se comprobará el cumplimiento de protocolos entre los dispositivos con el programa Wireshark, se realizarán diferentes pruebas con la finalidad de verificar su funcionamiento y analizar los resultados obtenidos.

4.1. Simulación de la Arquitectura de Red Definida por Software

Teniendo todas las herramientas y el software listo para la ejecución de las pruebas de simulación, en primer lugar, se comenzará con la emulación de la red virtual ejecutando el script desarrollado para el emulador Mininet y después se continuará con la ejecución de los módulos desarrollados para el controlador Ryu.

Para comenzar la simulación, se abre una terminal del sistema operativo Ubuntu mediante el cual se dirige al directorio donde se encuentra el archivo redmininet.py (el cual está ubicado en el escritorio), y se ejecuta el script desarrollado para la emulación de la red como se muestra en la Figura 37, allí se puede apreciar cómo se imprime en pantalla toda la información relevante en cuanto al proceso que realiza el script, y al final se establece la interfaz de línea de comandos de Mininet, la cual se encuentra en espera para recibir instrucciones.



```
gust@vo: ~/Escritorio
gust@vo:~$ cd Escritorio/
gust@vo:~/Escritorio$ sudo python redmininet.py
*** configuracion de condiciones para el controlador
Unable to contact the remote controller at 127.0.0.1:6633
*** creando switches...
*** creando hosts...
*** creando enlaces entre dispositivos...
(10.00Mbit) (10.00Mbit) (10.00Mbit) (10.00Mbit) (10.00Mbit) (10.00Mbit)
(10.00Mbit) (10.00Mbit) (10.00Mbit) (10.00Mbit) (10.00Mbit) (10.00Mbit)
(10.00Mbit) (10.00Mbit) (10.00Mbit) (10.00Mbit) (10.00Mbit) (10.00Mbit)
(10.00Mbit) (10.00Mbit) (10.00Mbit) (10.00Mbit) (10.00Mbit) (10.00Mbit)
*** Configuring hosts
h1 h2 h3 h4 h5
*** RED ESTABLECIDA ***
(10.00Mbit) (10.00Mbit) (10.00Mbit) (10.00Mbit) (10.00Mbit) (10.00Mbit)
(10.00Mbit) (10.00Mbit) (10.00Mbit) (10.00Mbit) (10.00Mbit) (10.00Mbit)
(10.00Mbit) (10.00Mbit) (10.00Mbit) (10.00Mbit) (10.00Mbit) (10.00Mbit)
(10.00Mbit) (10.00Mbit) (10.00Mbit) (10.00Mbit) (10.00Mbit) (10.00Mbit)
*** Starting CLI:
mininet>
```

Figura 37. Ejecución del script que emula la topología de red.

Fuente: Elaboración propia.

Una vez que se tiene la topología de red emulada, se verifican sus respectivas conexiones escribiendo en la interfaz de línea de comandos de Mininet el comando net, el cual muestra todas las interfaces utilizadas para las conexiones entre los hosts y los switches.

```

gust@vo: ~/Escritorio
mininet> net
h1 h1-eth0:s1-eth1
h2 h2-eth0:s2-eth1
h3 h3-eth0:s3-eth1
h4 h4-eth0:s4-eth1
h5 h5-eth0:s5-eth1
s1 lo: s1-eth1:h1-eth0 s1-eth2:s2-eth2 s1-eth3:s3-eth4
s2 lo: s2-eth1:h2-eth0 s2-eth2:s1-eth2 s2-eth3:s3-eth2
s3 lo: s3-eth1:h3-eth0 s3-eth2:s2-eth3 s3-eth3:s4-eth2 s3-eth4:s1-eth3 s3-eth5:s5-eth3
s4 lo: s4-eth1:h4-eth0 s4-eth2:s3-eth3 s4-eth3:s5-eth2
s5 lo: s5-eth1:h5-eth0 s5-eth2:s4-eth3 s5-eth3:s3-eth5
c0
mininet>

```

Figura 38. Interfaces de conexión de los dispositivos de la red en Mininet.

Fuente: Elaboración propia.

Para una mejor comprensión de lo que muestra el proceso realizado anteriormente, se ilustra la Figura 39, en la que se tiene la topología de red emulada con sus respectivas interfaces de conexión entre los hosts, switches y el controlador.

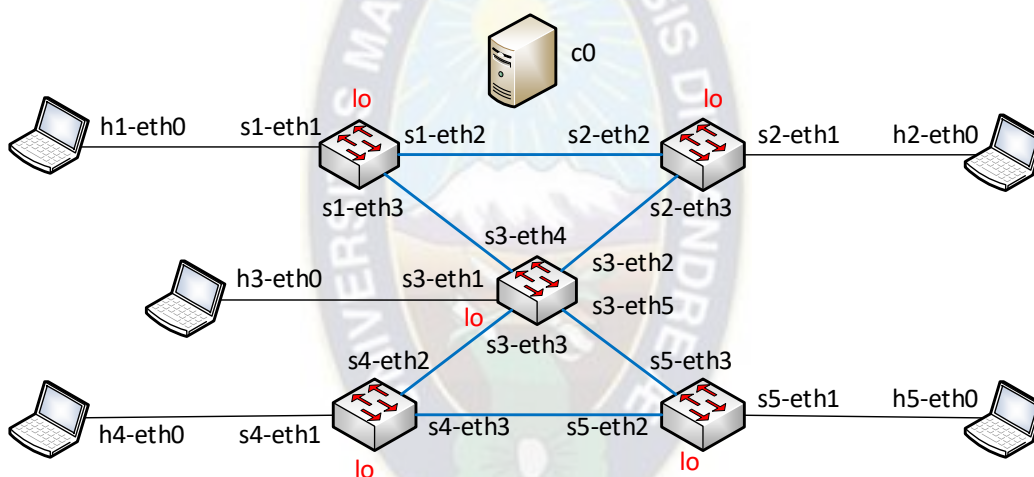


Figura 39. Interfaces de conexión de la arquitectura de red.

Fuente: Elaboración propia.

Luego se abre un emulador de terminal para cada switch y el controlador, para lo cual se ejecutó en el CLI de Mininet la instrucción: `xterm [nombre_del_dispositivo]`.

```

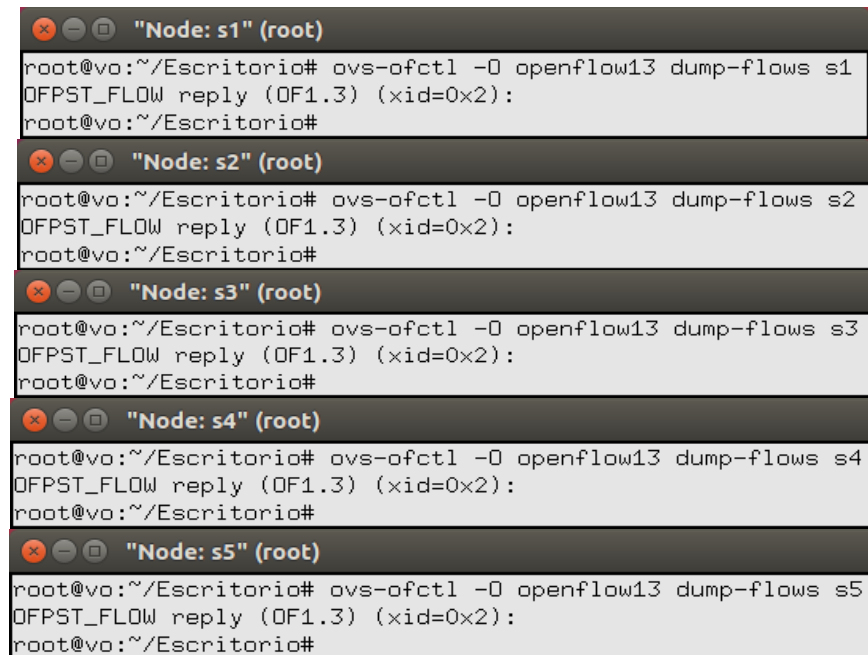
gust@vo: ~/Escritorio
mininet> xterm s1 s2 s3 s4 s5 c0
mininet>

```

Figura 40. Abriendo emuladores de terminal para los switches y el controlador.

Fuente: Elaboración propia.

Para cada uno de los switches de la topología, se verificará el estado en el que se encuentran sus tablas de flujos ejecutando el comando: `ovs-ofctl -O openflow13 dump-flows sX`, en cada terminal emulada (que se abrió anteriormente), donde X representa el número de switch donde se ejecuta el comando.



```
"Node: s1" (root)
root@vo:~/Escritorio# ovs-ofctl -O openflow13 dump-flows s1
OFPST_FLOW reply (OF1.3) (xid=0x2):
root@vo:~/Escritorio#

"Node: s2" (root)
root@vo:~/Escritorio# ovs-ofctl -O openflow13 dump-flows s2
OFPST_FLOW reply (OF1.3) (xid=0x2):
root@vo:~/Escritorio#

"Node: s3" (root)
root@vo:~/Escritorio# ovs-ofctl -O openflow13 dump-flows s3
OFPST_FLOW reply (OF1.3) (xid=0x2):
root@vo:~/Escritorio#

"Node: s4" (root)
root@vo:~/Escritorio# ovs-ofctl -O openflow13 dump-flows s4
OFPST_FLOW reply (OF1.3) (xid=0x2):
root@vo:~/Escritorio#

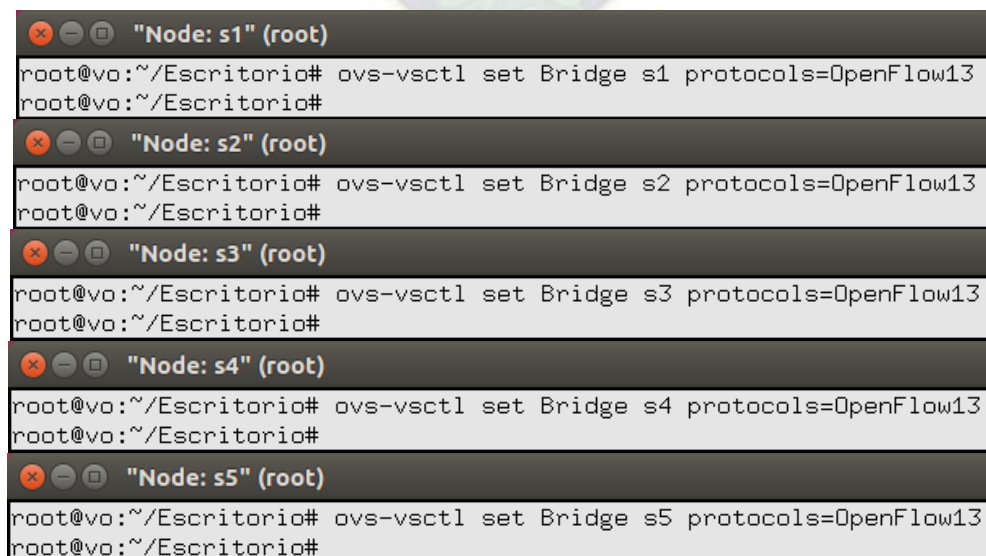
"Node: s5" (root)
root@vo:~/Escritorio# ovs-ofctl -O openflow13 dump-flows s5
OFPST_FLOW reply (OF1.3) (xid=0x2):
root@vo:~/Escritorio#
```

Figura 41. Verificación de las tablas de flujo en cada switch.

Fuente: Elaboración propia.

Como se observa en la Figura 41, se comprueba que las tablas de flujo de cada uno de los switches se encuentran vacías.

A continuación, se configura todos los switches de la topología para que realicen cálculos como bridges junto con las librerías stp del controlador Ryu y el protocolo OpenFlow versión 1.3. Para ello, se ejecuta el siguiente comando en la consola de cada switch: `ovs-vsctl set Bridge sX protocols=OpenFlow13`, donde la X representa al número del switch.



```
"Node: s1" (root)
root@vo:~/Escritorio# ovs-vsctl set Bridge s1 protocols=OpenFlow13
root@vo:~/Escritorio#

"Node: s2" (root)
root@vo:~/Escritorio# ovs-vsctl set Bridge s2 protocols=OpenFlow13
root@vo:~/Escritorio#

"Node: s3" (root)
root@vo:~/Escritorio# ovs-vsctl set Bridge s3 protocols=OpenFlow13
root@vo:~/Escritorio#

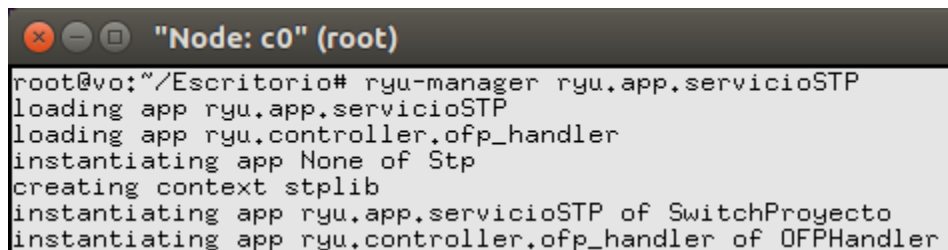
"Node: s4" (root)
root@vo:~/Escritorio# ovs-vsctl set Bridge s4 protocols=OpenFlow13
root@vo:~/Escritorio#

"Node: s5" (root)
root@vo:~/Escritorio# ovs-vsctl set Bridge s5 protocols=OpenFlow13
root@vo:~/Escritorio#
```

Figura 42. Configuración de los switches como Bridges y que soporten OpenFlow v1.3.

Fuente: Elaboración propia.

Con este procedimiento ya se tienen listos los switches virtuales de la red, a continuación se ejecutará la aplicación servicioSTP.py en la terminal emulada del controlador c0 mediante el comando: `ryu-manager ryu.app.servicioSTP`, como se muestra en la Figura 43, se cargaran y crearan las librerías y servicios necesarios para el controlador.



```
root@c0:~/Escritorio# ryu-manager ryu.app.servicioSTP
loading app ryu.app.servicioSTP
loading app ryu.controller.ofp_handler
instantiating app None of Stp
creating context stplib
instantiating app ryu.app.servicioSTP of SwitchProyecto
instantiating app ryu.controller.ofp_handler of OFPHandler
```

Figura 43. Ejecución de la API servicioSTP.py en la terminal del controlador.

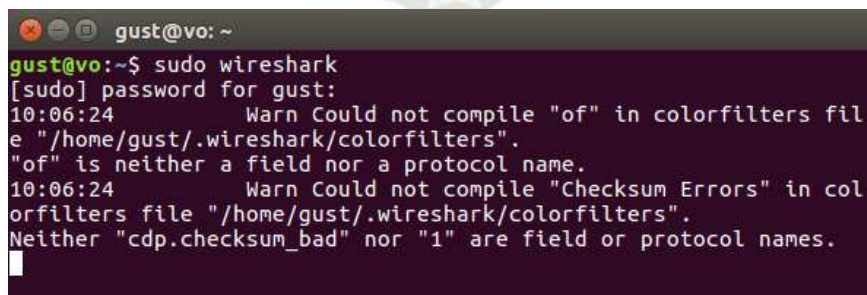
Fuente: Elaboración propia.

Una vez que se inició el funcionamiento de todo el software que conforma la arquitectura de red definida por software, primeramente, se verificará el intercambio de paquetes para el establecimiento del protocolo OpenFlow v1.3 entre los switches y el controlador utilizando Wireshark, luego se analizará el funcionamiento del controlador Ryu con las aplicaciones desarrolladas para implementar el STP en esta red virtual.

4.1.1. Verificación y Análisis del Establecimiento del Protocolo OpenFlow

Para realizar la verificación del establecimiento del canal OpenFlow entre los switches y el controlador se utiliza el programa Wireshark en su versión 2.2.6, el código fuente para la instalación de este programa viene junto con la clonación de los paquetes de instalación del emulador Mininet, entonces, cuando se instaló el emulador Mininet en su versión completa (como se expuso en el anterior capítulo) también se instaló Wireshark.

Para iniciar Wireshark se abre otra terminal del sistema operativo Ubuntu y se ejecuta el comando: `sudo Wireshark`, como se muestra a continuación.



```
gust@vo: ~
gust@vo:~$ sudo wireshark
[sudo] password for gust:
10:06:24 Warn Could not compile "of" in colorfilters file
"/home/gust/.wireshark/colorfilters".
"of" is neither a field nor a protocol name.
10:06:24 Warn Could not compile "Checksum Errors" in col
orfilters file "/home/gust/.wireshark/colorfilters".
Neither "cdp.checksum_bad" nor "1" are field or protocol names.
█
```

Figura 44. Comando para abrir el analizador de paquetes Wireshark.

Fuente: Elaboración propia.

Se abrirá una ventana como muestra la Figura 45, en la sección “Capture” solo se seleccionan las interfaces Loopback: lo, s1-eth1 y s5-eth1, que son las que se elegirán para analizar el tráfico o intercambio de paquetes entre el controlador y las interfaces de red de los switches s1 y s5 de la red virtual, ya que ocurre un procedimiento similar en la conexión e intercambio de mensajes de los otros switches con el controlador.

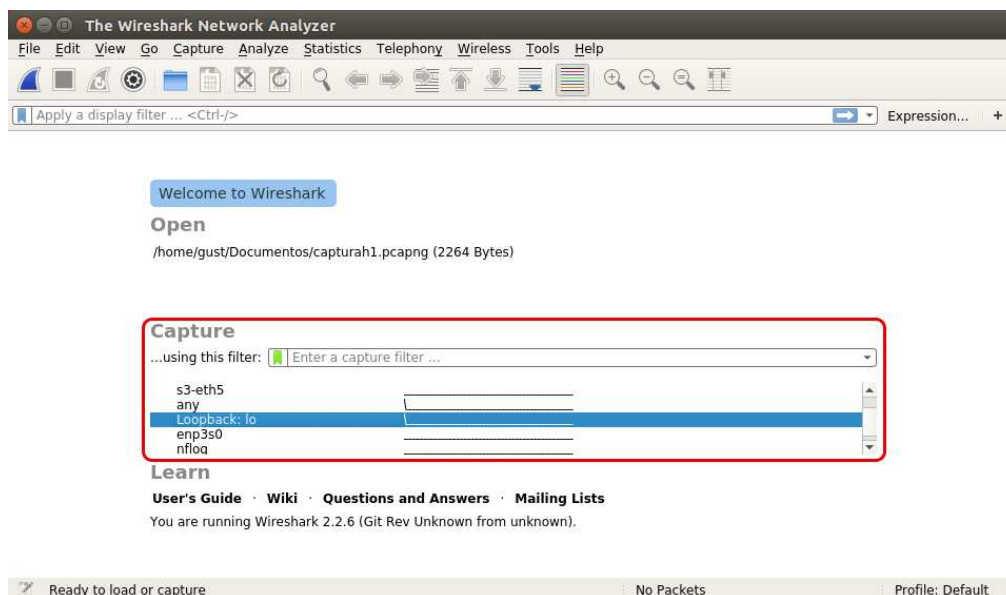


Figura 45. Selección de las interfaces de red que se analizarán.

Fuente: Elaboración propia.

En la captura de paquetes de la Figura 46, se puede ver que el switch s1 envía un paquete SYN (Sincronizar) por el puerto 42358 al controlador que se encuentra escuchando constantemente por el puerto 6633, el controlador recibe el paquete y le responde al switch s1 con un paquete SYN, ACK (Sincronizar, Admitir), luego de recibir la respuesta del controlador el switch s1 le responde al controlador con un mensaje ACK(admitir), estableciendo así la conexión TCP.

No.	Time	Source	Destination	Protocol	Length	Info
2	27.999662788	localhost	localhost	TCP	74	42358 → 6633 [SYN] Seq=0 Win=43690 [TCP CHECKSUM ...]
287	27.999679674	localhost	localhost	TCP	74	6633 → 42358 [SYN, ACK] Seq=0 Ack=1 Win=43690 [TCP ...]
288	27.999678493	localhost	localhost	TCP	66	42358 → 6633 [ACK] Seq=1 Ack=1 Win=44032 [TCP CHECKSUM ...]

Frame 286: 74 bytes on wire (592 bits), 74 bytes captured (592 bits) on interface 2
Ethernet II, Src: 00:00:00:00:00:00 (00:00:00:00:00:00), Dst: 00:00:00:00:00:00 (00:00:00:00:00:00)
Internet Protocol Version 4, Src: localhost (127.0.0.1), Dst: localhost (127.0.0.1)
Transmission Control Protocol, Src Port: 42358 (42358), Dst Port: 6633 (6633) Seq: 0, Len: 0
Source Port: 42358 (42358)
Destination Port: 6633 (6633)
[Stream index: 141]
[TCP Segment Len: 0]
Sequence number: 0 (relative sequence number)
Acknowledgment number: 0
Header length: 40 bytes
Flags: 0x002 (SYN)
000. SYN = Reserved: Not set
...0 = Nonce: Not set
...0 = Congestion Window Reduced (CWR): Not set
...0 = ECN-Echo: Not set
...0 = Urgent: Not set
...0 = Acknowledgment: Not set
...0 = Push: Not set
...0 = Reset: Not set
...1 = Syn: Set
...0 = Fin: Not set
[TCP Flags:S.]

No.	Time	Source	Destination	Protocol	Length	Info
286	27.999662708	localhost	localhost	TCP	74	42358 → 6633 [SYN] Seq=0 Win=43690 [TCP CHECKSUM I...
287	27.999670674	localhost	localhost	TCP	74	6633 → 42358 [SYN, ACK] Seq=0 Ack=1 Win=43690 [TC...
288	27.999678493	localhost	localhost	TCP	66	42358 → 6633 [ACK] Seq=1 Ack=1 Win=44032 [TCP CHEC...
▶ Frame 287: 74 bytes on wire (592 bits), 74 bytes captured (592 bits) on interface 2 ▶ Ethernet II, Src: 00:00:00:00:00:00 (00:00:00:00:00:00), Dst: 00:00:00:00:00:00 (00:00:00:00:00:00) ▶ Internet Protocol Version 4, Src: localhost (127.0.0.1), Dst: localhost (127.0.0.1) ▶ Transmission Control Protocol, Src Port: 6633 (6633), Dst Port: 42358 (42358), Seq: 0, Ack: 1, Len: 0 Source Port: 6633 (6633) Destination Port: 42358 (42358) [Stream index: 141] [TCP Segment Len: 0] Sequence number: 0 (relative sequence number) Acknowledgment number: 1 (relative ack number) Header Length: 40 bytes Flags: 0x012 (SYN, ACK) 000. = Reserved: Not set ...0 = Nonce: Not set0 = Congestion Window Reduced (CWR): Not set0 = ECN-Echo: Not set0 = Urgent: Not set1 = Acknowledgment: Set0 = Push: Not set0 = Reset: Not set1 = Syn: Set0 = Fin: Not set [TCP Flags:A..S.]						
286	27.999662708	localhost	localhost	TCP	74	42358 → 6633 [SYN] Seq=0 Win=43690 [TCP CHECKSUM I...
287	27.999670674	localhost	localhost	TCP	74	6633 → 42358 [SYN, ACK] Seq=0 Ack=1 Win=43690 [TCP...
288	27.999678493	localhost	localhost	TCP	66	42358 → 6633 [ACK] Seq=1 Ack=1 Win=44032 [TCP CHE...
▶ Frame 288: 66 bytes on wire (528 bits), 66 bytes captured (528 bits) on interface 2 ▶ Ethernet II, Src: 00:00:00:00:00:00 (00:00:00:00:00:00), Dst: 00:00:00:00:00:00 (00:00:00:00:00:00) ▶ Internet Protocol Version 4, Src: localhost (127.0.0.1), Dst: localhost (127.0.0.1) ▶ Transmission Control Protocol, Src Port: 42358 (42358), Dst Port: 6633 (6633), Seq: 1, Ack: 1, Len: 0 Source Port: 42358 (42358) Destination Port: 6633 (6633) [Stream index: 141] [TCP Segment Len: 0] Sequence number: 1 (relative sequence number) Acknowledgment number: 1 (relative ack number) Header Length: 32 bytes Flags: 0x010 (ACK) 000. = Reserved: Not set ...0 = Nonce: Not set0 = Congestion Window Reduced (CWR): Not set0 = ECN-Echo: Not set0 = Urgent: Not set1 = Acknowledgment: Set0 = Push: Not set0 = Reset: Not set0 = Syn: Not set0 = Fin: Not set [TCP Flags:A....]						

Figura 46. Establecimiento de la conexión TCP entre el switch s1 y controlador.

Fuente: Elaboración propia.

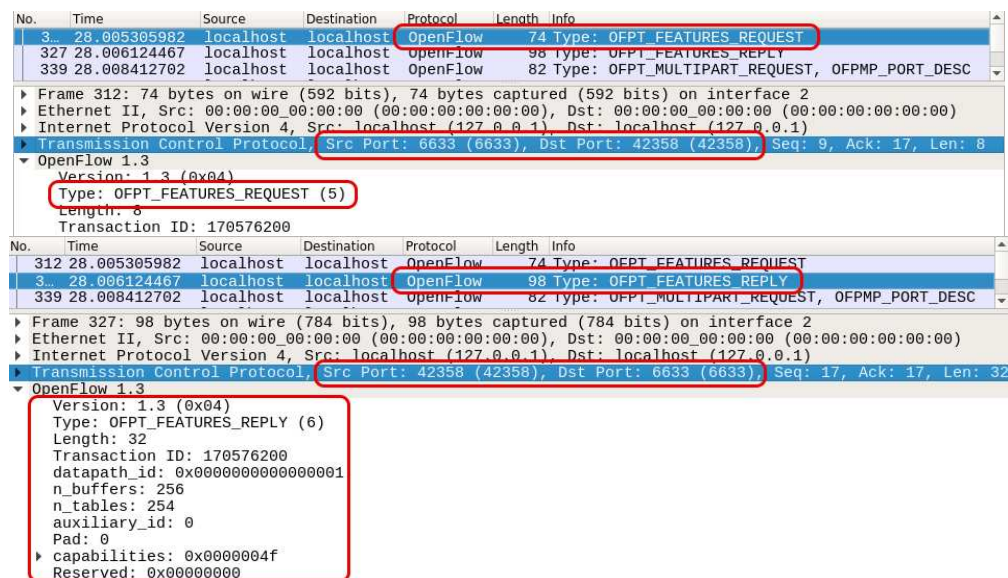
Siguiendo con el establecimiento del protocolo OpenFlow, el switch s1 envía el paquete OFPT_HELLO al controlador y este le responde también con un mensaje OFPT_HELLO, estableciendo los mensajes de saludo entre ambos. Para el filtrado de paquetes OpenFlow en esta versión de Wireshark el protocolo OpenFlow v1.3 es denominada openflow_v4.

No.	Time	Source	Destination	Protocol	Length	Info
2	27.999697130	localhost	localhost	OpenFlow	82	Type: OFPT_HELLO
310	28.005270445	localhost	localhost	OpenFlow	74	Type: OFPT_HELLO
312	28.005305982	localhost	localhost	OpenFlow	74	Type: OFPT_FEATURES_REQUEST
▶ Frame 289: 82 bytes on wire (656 bits), 82 bytes captured (656 bits) on interface 2 ▶ Ethernet II, Src: 00:00:00:00:00:00 (00:00:00:00:00:00), Dst: 00:00:00:00:00:00 (00:00:00:00:00:00) ▶ Internet Protocol Version 4, Src: localhost (127.0.0.1), Dst: localhost (127.0.0.1) ▶ Transmission Control Protocol, Src Port: 42358 (42358), Dst Port: 6633 (6633), Seq: 1, Ack: 1, Len: 16 OpenFlow 1.3 Version: 1.3 (0x04) Type: OFPT_HELLO (0) Length: 16 Transaction ID: 61 Element						
289	27.999697130	localhost	localhost	OpenFlow	82	Type: OFPT_HELLO
310	28.005270445	localhost	localhost	OpenFlow	74	Type: OFPT_HELLO
312	28.005305982	localhost	localhost	OpenFlow	74	Type: OFPT_FEATURES_REQUEST
▶ Frame 310: 74 bytes on wire (592 bits), 74 bytes captured (592 bits) on interface 2 ▶ Ethernet II, Src: 00:00:00:00:00:00 (00:00:00:00:00:00), Dst: 00:00:00:00:00:00 (00:00:00:00:00:00) ▶ Internet Protocol Version 4, Src: localhost (127.0.0.1), Dst: localhost (127.0.0.1) ▶ Transmission Control Protocol, Src Port: 6633 (6633), Dst Port: 42358 (42358), Seq: 1, Ack: 17, Len: 8 OpenFlow 1.3 Version: 1.3 (0x04) Type: OFPT_HELLO (0) Length: 8 Transaction ID: 170576199						

Figura 47. Intercambio de mensajes HELLO entre switch y controlador.

Fuente: Elaboración propia.

El siguiente mensaje es el OFPT_FEATURES_REQUEST, que es enviado por el controlador al switch s1 solicitando las características básicas del switch, entonces la respuesta a esos requerimientos la realiza el switch s1 mediante el mensaje OFPT_FEATURES_REPLY, donde se especifica las características principales del switch, entre ellas se destacan el identificador de la ruta de datos (datapath_id=dpid) que en este caso es datapath_id: 0x0000000000000001, también el número de tablas de flujos admitidos por este switch con el valor n_tables:254, y otras características más del switch como muestra la Figura 48.



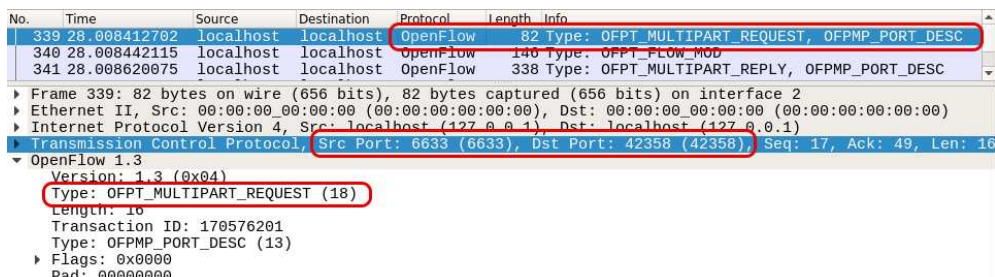
The figure shows a Wireshark packet capture of OpenFlow messages. The first packet (No. 3, Time 28.005305982) is an OFPT_FEATURES_REQUEST (Type 5) from localhost to localhost. The second packet (No. 312, Time 28.006124467) is an OFPT_FEATURES_REPLY (Type 6) from localhost to localhost. The details pane for the second packet shows the following fields:

Field	Value
Version	1.3 (0x04)
Type	OFPT_FEATURES_REPLY (6)
Length	32
Transaction ID	170576200
datapath_id	0x0000000000000001
n_buffers	256
n_tables	254
auxiliary_id	0
Pad	0
capabilities	0x0000004f
Reserved	0x00000000

Figura 48. Intercambio de mensajes FEATURES entre controlador y switch.
Fuente: Elaboración propia.

A continuación, se observa el mensaje OFPT_MULTIPART_REQUEST, OFPMP_PORT_DESC, que es el requerimiento del controlador solicitando una configuración detallada de los puertos del switch, en los que se encuentran estadísticas e información del estado de cada puerto.

En respuesta a esto, el switch s1 envía al controlador el mensaje OFPT_MULTIPART_REPLY, OFPMP_PORT_DESC, donde se puede ver el nombre, la dirección MAC, el estado, la velocidad y otras características de cada puerto del switch como muestra la Figura 49.



The figure shows a Wireshark packet capture of OpenFlow messages. The third packet (No. 339, Time 28.008412702) is an OFPT_MULTIPART_REQUEST (Type 18) from localhost to localhost. The fourth packet (No. 340, Time 28.008442115) is an OFPT_MULTIPART_REPLY (Type 13) from localhost to localhost. The details pane for the fourth packet shows the following fields:

Field	Value
Version	1.3 (0x04)
Type	OFPT_MULTIPART_REQUEST (18)
Length	16
Transaction ID	170576201
Type	OFPMP_PORT_DESC (13)
Flags	0x0000
Pad	00000000

No.	Time	Source	Destination	Protocol	Length	Info
339	28.008412702	localhost	localhost	OpenFlow	82	Type: OFPT_MULTIPART_REQUEST, OFPMP_PORT_DESC
340	28.008442115	localhost	localhost	OpenFlow	146	Type: OFPT_FLOW_MOD
341	28.008620075	localhost	localhost	OpenFlow	338	Type: OFPT_MULTIPART_REPLY, OFPMP_PORT_DESC

Frame 341: 338 bytes on wire (2704 bits), 338 bytes captured (2704 bits) on interface 2
 Ethernet II, Src: 00:00:00:00:00:00 (00:00:00:00:00:00), Dst: 00:00:00:00:00:00 (00:00:00:00:00:00)
 Internet Protocol Version 4, Src: localhost (127.0.0.1), Dst: localhost (127.0.0.1)
 Transmission Control Protocol, Src Port: 42358 (42358), Dst Port: 6633 (6633), Seq: 49, Ack: 113, Len: 2

OpenFlow 1.3
 Version: 1.3 (0x04)
 Type: OFPT_MULTIPART_REPLY (19)
 Length: 272
 Transaction ID: 170576201
 Type: OFPMP_PORT_DESC (13)
 Flags: 0x0000
 Pad: 00000000
 Port
 Port
 Port no: 1
 Pad: 00000000
 Hw addr: 7e:8d:ce:7f:02:5e (7e:8d:ce:7f:02:5e)
 Pad: 0000
 Name: s1-eth1
 Config: 0x00000000
 State: 0x00000000
 Current: 0x00000840
 Advertised: 0x00000000
 Supported: 0x00000000
 Peer: 0x00000000
 Curr speed: 10000000
 Max speed: 0

Figura 49. Intercambio de mensajes MULTIPART entre controlador y switch.
Fuente: Elaboración propia.

Con esto queda verificado el establecimiento del canal OpenFlow entre el switch denominado s1 y el controlador Ryu. También se comprobó que ocurre un procedimiento similar en el intercambio de mensajes de los demás switches de la red virtualizada con el controlador.

4.1.2. Verificación y Análisis del Establecimiento del STP

Una vez que se tiene establecido el canal de comunicación entre los switches y el controlador, ahora se verificará el establecimiento del STP en la arquitectura de red virtualizada.

En esta instancia se verificará el funcionamiento de la aplicación llamada servicioSTP.py, para lo cual se dejará los switches de la red con sus valores predeterminados proporcionados por el emulador, y se realizará el análisis del sistema con las siguientes variables: el identificador de ruta de datos (dpid), la prioridad de bridge (bridge_priority) y la dirección MAC (address MAC).

El identificador de ruta de datos (dpid) es asignado por defecto con un orden ascendente comenzando desde el número 1 hasta enumerar todos los switches de la red, de acuerdo a la posición en la que se encuentran escritos o definidos en el programa.

La prioridad de bridge (bridge_priority) tiene un valor por defecto igual a 32768 (8000 en hexadecimal) en todos los switches reales y también ocurre lo mismo en este caso.

Las direcciones MAC (address MAC) de cada switch son asignadas con valores aleatorios cada vez que se inicia la emulación con Mininet.

Una vez que se inicia la aplicación “servicioSTP.py”, en la terminal del controlador se observa el cálculo del STP, comenzando a trabajar con el reconocimiento y la unión de cada uno de los switches como bridges stp, solicitando información sobre las características de cada switch y verificando también la funcionalidad y el estado de cada puerto.

En la Figura 50, se puede apreciar que el primer switch que fue reconocido por la aplicación es el switch con dpid=0000000000000004, y todos sus puertos se encuentran en estados de ESCUCHA como es de esperar para éste y para todos los demás switches de la topología.

```

[STP][INFO] dpid=0000000000000004: Se unio como un bridge stp.
[STP][INFO] dpid=0000000000000004: [puerto=1] PUERTO_DESIGNADO Estado: ESCUCHA
[STP][INFO] dpid=0000000000000004: [puerto=2] PUERTO_DESIGNADO Estado: ESCUCHA
[STP][INFO] dpid=0000000000000005: Se unio como un bridge stp.
[STP][INFO] dpid=0000000000000004: [puerto=3] PUERTO_DESIGNADO Estado: ESCUCHA
[STP][INFO] dpid=0000000000000005: [puerto=1] PUERTO_DESIGNADO Estado: ESCUCHA
[STP][INFO] dpid=0000000000000005: [puerto=2] PUERTO_DESIGNADO Estado: ESCUCHA
[STP][INFO] dpid=0000000000000001: Se unio como un bridge stp.
[STP][INFO] dpid=0000000000000005: [puerto=3] PUERTO_DESIGNADO Estado: ESCUCHA
[STP][INFO] dpid=0000000000000001: [puerto=1] PUERTO_DESIGNADO Estado: ESCUCHA
[STP][INFO] dpid=0000000000000001: [puerto=2] PUERTO_DESIGNADO Estado: ESCUCHA
[STP][INFO] dpid=0000000000000003: Se unio como un bridge stp.
[STP][INFO] dpid=0000000000000001: [puerto=3] PUERTO_DESIGNADO Estado: ESCUCHA
[STP][INFO] dpid=0000000000000003: [puerto=1] PUERTO_DESIGNADO Estado: ESCUCHA
[STP][INFO] dpid=0000000000000003: [puerto=2] PUERTO_DESIGNADO Estado: ESCUCHA
[STP][INFO] dpid=0000000000000003: [puerto=3] PUERTO_DESIGNADO Estado: ESCUCHA
[STP][INFO] dpid=0000000000000003: [puerto=4] PUERTO_DESIGNADO Estado: ESCUCHA
[STP][INFO] dpid=0000000000000002: Se unio como un bridge stp.
[STP][INFO] dpid=0000000000000003: [puerto=5] PUERTO_DESIGNADO Estado: ESCUCHA
[STP][INFO] dpid=0000000000000002: [puerto=1] PUERTO_DESIGNADO Estado: ESCUCHA
[STP][INFO] dpid=0000000000000002: [puerto=2] PUERTO_DESIGNADO Estado: ESCUCHA
[STP][INFO] dpid=0000000000000003: [puerto=3] Recibe un BPDU superior.
[STP][INFO] dpid=0000000000000002: [puerto=3] PUERTO_DESIGNADO Estado: ESCUCHA

```

Figura 50. Reconocimiento de los switches por el controlador.

Fuente: Elaboración propia.

Una vez que todos los switches fueron reconocidos por la aplicación mediante el intercambio de paquetes de BPDU entre switches, se comienza con la elección del Root Bridge de la red, por lo que los puertos pasan del estado ESCUCHA al estado BLOQUEADO sucesivamente hasta encontrar el switch con la más alta prioridad que será denominado Root Bridge.

En el presente caso se tiene los siguientes datos para cada uno de los switches:

Tabla 14. Valores de interés de los switches emulados.

Switch	datapath_id	Bridge_id	
		Bridge_priority	address_MAC
s1	0000000000000001	8000	7e:8d:ce:7f:02:5e
s2	0000000000000002	8000	3a:70:29:5c:2f:2f
s3	0000000000000003	8000	32:f9:ac:af:b4:39
s4	0000000000000004	8000	0e:e3:3b:11:b7:53
s5	0000000000000005	8000	06:52:1c:16:0b:58

Nota: Elaboración propia.

Según la Tabla 14, el Root Bridge debería ser el switch s5 porque cuenta con el número más bajo de Bridge_id comparado con los demás switches.

En la Figura 51, se muestra la designación del Root Bridge al switch s5 por parte de la aplicación como se esperaba por tener la más alta prioridad.

```

"Node: c0" (root)
[STP][INFO] dpid=0000000000000005: [puerto=3] PUERTO_DESIGNADO Estado: BLOQUEADO
[STP][INFO] dpid=0000000000000005: Es Root bridge.
[STP][INFO] dpid=0000000000000005: [puerto=1] PUERTO_DESIGNADO Estado: ESCUCHA

```

Figura 51. Designación del Root Bridge al switch s5.

Fuente: Elaboración propia.

Para verificar lo anterior, solo se analizará el intercambio de mensajes entre el controlador c0 y el switch s1 ya que para los demás switches se cumple un procedimiento similar, se busca en Wireshark las variables que ayudan a verificar la designación del Root Bridge, comenzando por el identificador de la ruta de datos que se encuentra en el mensaje OFPT_FEATURES_REPLY que el switch s1 envió al controlador en el momento que se establece el canal OpenFlow, tal como muestra la Figura 48, una vez obtenido el dpid y el puerto de comunicación del switch s1 se puede verificar el Root Bridge y su dirección MAC en el mensaje OFPT_PACKET_OUT.

La Figura 52, muestra el valor de Root Bridge que tiene el switch s1 antes de que la aplicación le asigne un nuevo valor a éste.

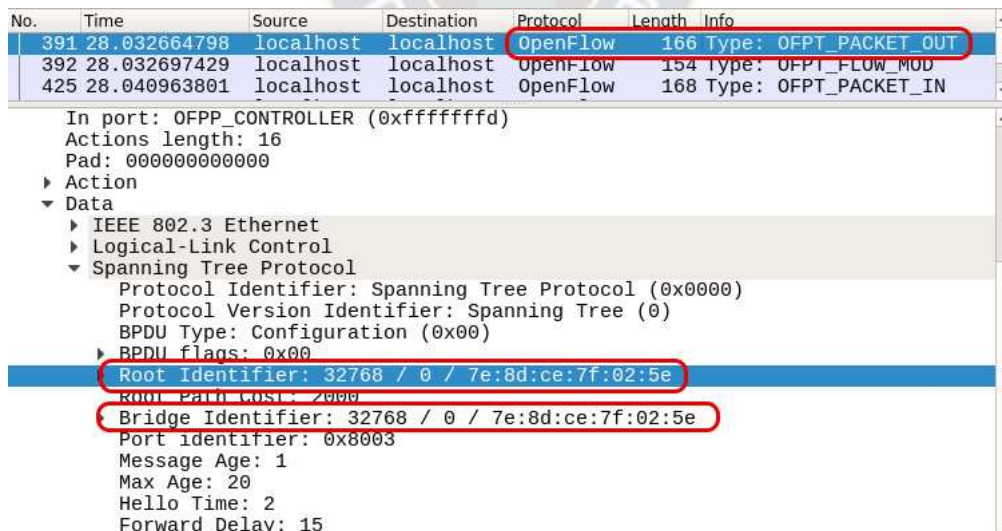


Figura 52. Root Bridge del switch s1 antes de la asignación.

Fuente: Elaboración propia.

Luego del intercambio de paquetes de BPDU entre todos los switches, el controlador asigna al switch s5 como Root Bridge ya que tiene el Bridge_ID (Identificador Puente) más bajo con

respecto a los demás switches.

En la Figura 53, se muestra cómo cambia el valor del Root Bridge (Puente raíz) en el switch s1 con el Identificador Puente del switch s5.

No.	Time	Source	Destination	Protocol	Length	Info
938	28.243140937	localhost	localhost	OpenFlow	166	Type: OFPT_PACKET_OUT
939	28.243181766	localhost	localhost	OpenFlow	106	Type: OFPT_PORT_MOD
940	28.243206448	localhost	localhost	OpenFlow	130	Type: OFPT_FLOW_MOD

In port: OFPP_CONTROLLER (0xfffffdd)
Actions length: 16
Pad: 000000000000
▶ Action
▼ Data
▶ IEEE 802.3 Ethernet
▶ Logical-Link Control
▼ Spanning Tree Protocol
Protocol Identifier: Spanning Tree Protocol (0x0000)
Protocol Version Identifier: Spanning Tree (0)
BPDU Type: Configuration (0x00)
BPDU flags: 0x00
Root Identifier: 32768 / 0 / 06:52:1c:16:0b:58
Root Path Cost: 0000
Bridge Identifier: 32768 / 0 / 7e:8d:ce:7f:02:5e
Port Identifier: 0x8002
Message Age: 3
Max Age: 20
Hello Time: 2
Forward Delay: 15

Figura 53. Mensaje enviado al switch s1 con la elección del Root Bridge.

Fuente: Elaboración propia.

Entonces, el controlador va enviando mensajes OFPT_PACKET_OUT a todos los switches adhiriendo los datos del Root Bridge seleccionado con sus respectivas características, además de los valores que corresponden a los temporizadores de BPDU.

El siguiente paso consiste en el reconocimiento de las tablas de direcciones MAC de los switches y tenerlos listos para su funcionamiento, aunque todavía no se puede transferir ninguno tipo de datos (excepto las BPDU's). El estado de todos los puertos cambia a APRENDE.

"Node: c0" (root)		
[STP][INFO]	dpid=0000000000000005: [puerto=1]	PUERTO_DESIGNADO Estado: APRENDE
[STP][INFO]	dpid=0000000000000005: [puerto=2]	PUERTO_DESIGNADO Estado: APRENDE
[STP][INFO]	dpid=0000000000000005: [puerto=3]	PUERTO_DESIGNADO Estado: APRENDE
[STP][INFO]	dpid=0000000000000004: [puerto=1]	PUERTO_DESIGNADO Estado: APRENDE
[STP][INFO]	dpid=0000000000000004: [puerto=2]	PUERTO_NO_DESIGNADO Estado: APRENDE
[STP][INFO]	dpid=0000000000000004: [puerto=3]	PUERTO_ROOT Estado: APRENDE
[STP][INFO]	dpid=0000000000000003: [puerto=1]	PUERTO_DESIGNADO Estado: APRENDE
[STP][INFO]	dpid=0000000000000003: [puerto=2]	PUERTO_DESIGNADO Estado: APRENDE
[STP][INFO]	dpid=0000000000000003: [puerto=3]	PUERTO_DESIGNADO Estado: APRENDE
[STP][INFO]	dpid=0000000000000003: [puerto=4]	PUERTO_DESIGNADO Estado: APRENDE
[STP][INFO]	dpid=0000000000000003: [puerto=5]	PUERTO_ROOT Estado: APRENDE
[STP][INFO]	dpid=0000000000000001: [puerto=1]	PUERTO_DESIGNADO Estado: APRENDE
[STP][INFO]	dpid=0000000000000001: [puerto=2]	PUERTO_NO_DESIGNADO Estado: APRENDE
[STP][INFO]	dpid=0000000000000001: [puerto=3]	PUERTO_ROOT Estado: APRENDE
[STP][INFO]	dpid=0000000000000002: [puerto=1]	PUERTO_DESIGNADO Estado: APRENDE
[STP][INFO]	dpid=0000000000000002: [puerto=2]	PUERTO_DESIGNADO Estado: APRENDE
[STP][INFO]	dpid=0000000000000002: [puerto=3]	PUERTO_ROOT Estado: APRENDE

Figura 54. Estado APRENDE en los puertos de los switches.

Fuente: Elaboración propia.

Para finalizar el establecimiento del STP mediante la aplicación servicioSTP.py, se observa que se asignan de manera permanente las funcionalidades y el estado a cada puerto, hasta que se presente un cambio dentro de la topología de red.

```

"Node: c0" (root)
[STP][INFO] dpid=0000000000000005: [puerto=1] PUERTO_DESIGNADO Estado: REENVIA
[STP][INFO] dpid=0000000000000005: [puerto=2] PUERTO_DESIGNADO Estado: REENVIA
[STP][INFO] dpid=0000000000000005: [puerto=3] PUERTO_DESIGNADO Estado: REENVIA
[STP][INFO] dpid=0000000000000004: [puerto=1] PUERTO_DESIGNADO Estado: REENVIA
[STP][INFO] dpid=0000000000000004: [puerto=2] PUERTO_NO_DESIGNADO Estado: BLOQUEADO
[STP][INFO] dpid=0000000000000004: [puerto=3] PUERTO_ROOT Estado: REENVIA
[STP][INFO] dpid=0000000000000003: [puerto=1] PUERTO_DESIGNADO Estado: REENVIA
[STP][INFO] dpid=0000000000000003: [puerto=2] PUERTO_DESIGNADO Estado: REENVIA
[STP][INFO] dpid=0000000000000003: [puerto=3] PUERTO_DESIGNADO Estado: REENVIA
[STP][INFO] dpid=0000000000000003: [puerto=4] PUERTO_DESIGNADO Estado: REENVIA
[STP][INFO] dpid=0000000000000003: [puerto=5] PUERTO_ROOT Estado: REENVIA
[STP][INFO] dpid=0000000000000001: [puerto=1] PUERTO_DESIGNADO Estado: REENVIA
[STP][INFO] dpid=0000000000000001: [puerto=2] PUERTO_NO_DESIGNADO Estado: BLOQUEADO
[STP][INFO] dpid=0000000000000001: [puerto=3] PUERTO_ROOT Estado: REENVIA
[STP][INFO] dpid=0000000000000002: [puerto=1] PUERTO_DESIGNADO Estado: REENVIA
[STP][INFO] dpid=0000000000000002: [puerto=2] PUERTO_DESIGNADO Estado: REENVIA
[STP][INFO] dpid=0000000000000002: [puerto=3] PUERTO_ROOT Estado: REENVIA

```

Figura 55. Estado REENVÍA en los puertos de los switches.
Fuente: Elaboración propia.

Hasta aquí se comprobó que la aplicación realizó el cálculo del STP en la topología de red, asignando a los puertos de los switches las funcionalidades PUERTO_DESIGNADO y PUERTO_ROOT que se encuentran en el estado de REENVIA para el intercambio de datos entre ellos, y los puertos con la funcionalidad PUERTO_NO_DESIGNADO se encuentran en el estado de BLOQUEADO no pudiendo transferir datos, excepto intercambiar mensajes de BPDU para verificar algún tipo de cambio en la red. En la Figura 56 se ilustra cómo se encuentra la topología de red según los resultados que arrojó el cálculo del STP con la aplicación servicioSTP.py.

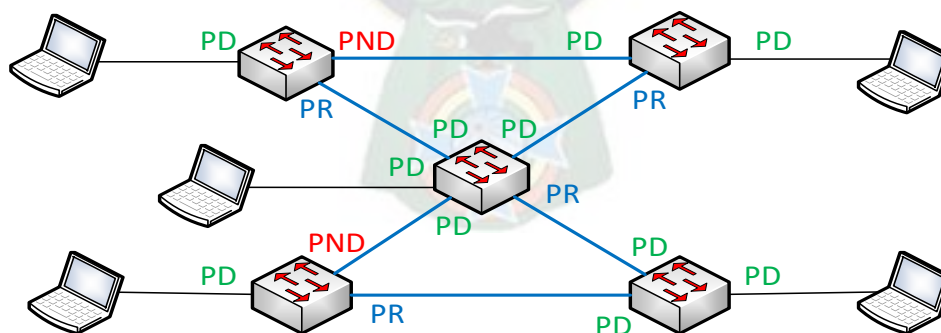


Figura 56. Esquema de la designación de las funciones y estados de cada puerto.
Fuente: Elaboración propia.

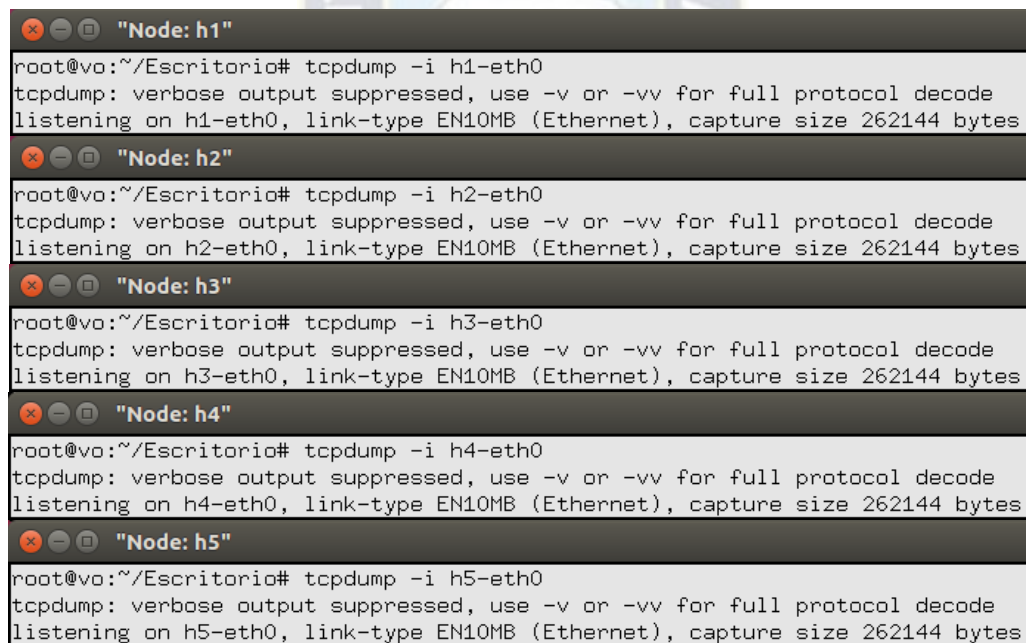
Una vez que se tiene establecida la red con el servicio que proporcionó el STP para la gestión de bucles o caminos redundante, se analizará la conectividad entre los hosts de la red mediante el

intercambio de mensajes ARP e ICMP para lo cual se utiliza nuevamente Wireshark, y también se expondrán los cambios que ocurren dentro de los dispositivos de la red.

4.2. Pruebas Básicas de Funcionamiento de la Red Definida por Software

Para comenzar a realizar operaciones básicas dentro de la arquitectura de red establecida, primeramente, se mostrará que es lo que ocurre dentro de los dispositivos de la red al momento de enviar un paquete de datos desde el host h1 al host h5 con el comando ping (Packet Internet Groper - Buscador de paquetes de internet), el cual también ayudara a diagnosticar el estado de la conexión de los dispositivos en cualquier momento.

Entonces, se pondrá en modo escucha las interfaces de los hosts conectados a los switches con el objetivo de analizar el intercambio de paquetes que ocurre dentro de cada uno de ellos al momento de aplicar el comando ping mencionado en el párrafo anterior, por lo que en la terminal emulada de cada host se ejecuta el comando: `tcpdump -i hX-eth0`, donde X representa el número de host.



```
"Node: h1"
root@vo:~/Escritorio# tcpdump -i h1-eth0
tcpdump: verbose output suppressed, use -v or -vv for full protocol decode
listening on h1-eth0, link-type EN10MB (Ethernet), capture size 262144 bytes

"Node: h2"
root@vo:~/Escritorio# tcpdump -i h2-eth0
tcpdump: verbose output suppressed, use -v or -vv for full protocol decode
listening on h2-eth0, link-type EN10MB (Ethernet), capture size 262144 bytes

"Node: h3"
root@vo:~/Escritorio# tcpdump -i h3-eth0
tcpdump: verbose output suppressed, use -v or -vv for full protocol decode
listening on h3-eth0, link-type EN10MB (Ethernet), capture size 262144 bytes

"Node: h4"
root@vo:~/Escritorio# tcpdump -i h4-eth0
tcpdump: verbose output suppressed, use -v or -vv for full protocol decode
listening on h4-eth0, link-type EN10MB (Ethernet), capture size 262144 bytes

"Node: h5"
root@vo:~/Escritorio# tcpdump -i h5-eth0
tcpdump: verbose output suppressed, use -v or -vv for full protocol decode
listening on h5-eth0, link-type EN10MB (Ethernet), capture size 262144 bytes
```

Figura 57. Modo escucha de los hosts.

Fuente: Elaboración propia.

De la misma forma que se hizo para los hosts, también se configura los puertos de cada switch en modo escucha para analizar el intercambio de paquetes dentro del switch, para lo cual se ejecuta el comando: `tcpdump -i sX-ethY`, donde X representa el número de switch y Y es el número de puerto.

```

"Node: s1" (root)
root@vo:~/Escritorio# tcpdump -i s1-eth3
tcpdump: verbose output suppressed, use -v or -vv for full protocol decode
listening on s1-eth3, link-type EN10MB (Ethernet), capture size 262144 bytes

"Node: s2" (root)
root@vo:~/Escritorio# tcpdump -i s2-eth3
tcpdump: verbose output suppressed, use -v or -vv for full protocol decode
listening on s2-eth3, link-type EN10MB (Ethernet), capture size 262144 bytes

"Node: s3" (root)
root@vo:~/Escritorio# tcpdump -i s3-eth5
tcpdump: verbose output suppressed, use -v or -vv for full protocol decode
listening on s3-eth5, link-type EN10MB (Ethernet), capture size 262144 bytes

"Node: s4" (root)
root@vo:~/Escritorio# tcpdump -i s4-eth3
tcpdump: verbose output suppressed, use -v or -vv for full protocol decode
listening on s4-eth3, link-type EN10MB (Ethernet), capture size 262144 bytes

"Node: s5" (root)
root@vo:~/Escritorio# tcpdump -i s5-eth3
tcpdump: verbose output suppressed, use -v or -vv for full protocol decode
listening on s5-eth3, link-type EN10MB (Ethernet), capture size 262144 bytes

```

Figura 58. Modo escucha de los switches.

Fuente: Elaboración propia.

Una vez que se tiene los puertos de los hosts y switches en modo escucha, se ejecuta el comando ping desde el CLI de Mininet como muestra la Figura 59.

```

gust@vo: ~/Escritorio
mininet> h1 ping -c 1 h5
PING 10.0.0.5 (10.0.0.5) 56(84) bytes of data:
64 bytes from 10.0.0.5: icmp_seq=1 ttl=64 time=33.9 ms

--- 10.0.0.5 ping statistics ---
1 packets transmitted, 1 received, 0% packet loss, time 0ms
rtt min/avg/max/mdev = 33.941/33.941/33.941/0.000 ms
mininet>

```

Figura 59. Prueba de conexión entre el host h1 y h5.

Fuente: Elaboración propia.

Entonces, una vez ejecutado el comando ping se puede observar en la terminal del controlador c0, el procesamiento del paquete por el controlador.

```

"Node: c0" (root)
packet in dpid:1 puerto_in[1] origen:00:00:00:01:01:01 -> destino:ff:ff:ff:ff:ff:ff
packet in dpid:3 puerto_in[4] origen:00:00:00:01:01:01 -> destino:ff:ff:ff:ff:ff:ff
packet in dpid:2 puerto_in[3] origen:00:00:00:01:01:01 -> destino:ff:ff:ff:ff:ff:ff
packet in dpid:5 puerto_in[3] origen:00:00:00:01:01:01 -> destino:ff:ff:ff:ff:ff:ff
packet in dpid:4 puerto_in[3] origen:00:00:00:01:01:01 -> destino:ff:ff:ff:ff:ff:ff
packet in dpid:5 puerto_in[1] origen:00:00:00:05:05:05 -> destino:00:00:00:01:01:01
packet in dpid:3 puerto_in[5] origen:00:00:00:05:05:05 -> destino:00:00:00:01:01:01
packet in dpid:1 puerto_in[3] origen:00:00:00:05:05:05 -> destino:00:00:00:01:01:01
packet in dpid:1 puerto_in[1] origen:00:00:00:01:01:01 -> destino:00:00:00:05:05:05
packet in dpid:3 puerto_in[4] origen:00:00:00:01:01:01 -> destino:00:00:00:05:05:05
packet in dpid:5 puerto_in[3] origen:00:00:00:01:01:01 -> destino:00:00:00:05:05:05

```

Figura 60. Procesando el paquete enviado desde h1 hacia h5.

Fuente: Elaboración propia.

A continuación, se muestran los cambios que se efectúan en el interior de los hosts mediante sus respectivas terminales que se encuentran en modo escucha.

```

"Node: h1"
23:16:12.145632 ARP, Request who-has 10.0.0.5 tell 10.0.0.1, length 28
23:16:12.170086 ARP, Reply 10.0.0.5 is-at 00:00:00:05:05:05 (oui Ethernet), length 28
23:16:12.170110 IP 10.0.0.1 > 10.0.0.5: ICMP echo request, id 22631, seq 1, length 64
23:16:12.179533 IP 10.0.0.5 > 10.0.0.1: ICMP echo reply, id 22631, seq 1, length 64

"Node: h2"
23:16:12.157766 ARP, Request who-has 10.0.0.5 tell 10.0.0.1, length 28
23:16:13.820818 STP 802.1d, Config, Flags [none], bridge-id 8000.3a:70:29:5c:2f:2f.8001,
length 43

"Node: h3"
23:16:12.152642 ARP, Request who-has 10.0.0.5 tell 10.0.0.1, length 28
23:16:13.799620 STP 802.1d, Config, Flags [none], bridge-id 8000.32:f9:ac:af:b4:39.8001,
length 43

"Node: h4"
23:16:12.162524 ARP, Request who-has 10.0.0.5 tell 10.0.0.1, length 28
23:16:13.807039 STP 802.1d, Config, Flags [none], bridge-id 8000.0e:e3:3b:11:b7:53.8001,
length 43

"Node: h5"
23:16:12.157629 ARP, Request who-has 10.0.0.5 tell 10.0.0.1, length 28
23:16:12.157663 ARP, Reply 10.0.0.5 is-at 00:00:00:05:05:05 (oui Ethernet), length 28
23:16:12.179076 IP 10.0.0.1 > 10.0.0.5: ICMP echo request, id 22631, seq 1, length 64
23:16:12.179112 IP 10.0.0.5 > 10.0.0.1: ICMP echo reply, id 22631, seq 1, length 64

```

Figura 61. Intercambio de mensajes en los hosts.

Fuente: Elaboración propia.

De la misma forma, se muestra el intercambio de paquetes que ocurre en el interior de los switches.

```

"Node: s1" (root)
23:16:12.149571 ARP, Request who-has 10.0.0.5 tell 10.0.0.1, length 28
23:16:12.166457 ARP, Reply 10.0.0.5 is-at 00:00:00:05:05:05 (oui Ethernet), length 28
23:16:12.173141 IP 10.0.0.1 > 10.0.0.5: ICMP echo request, id 22631, seq 1, length 64
23:16:12.179395 IP 10.0.0.5 > 10.0.0.1: ICMP echo reply, id 22631, seq 1, length 64

"Node: s2" (root)
23:16:12.152648 ARP, Request who-has 10.0.0.5 tell 10.0.0.1, length 28
23:16:13.801505 STP 802.1d, Config, Flags [none], bridge-id 8000.32:f9:ac:af:b4:39.8002,
length 43

"Node: s3" (root)
23:16:12.152653 ARP, Request who-has 10.0.0.5 tell 10.0.0.1, length 28
23:16:12.162708 ARP, Reply 10.0.0.5 is-at 00:00:00:05:05:05 (oui Ethernet), length 28
23:16:12.176144 IP 10.0.0.1 > 10.0.0.5: ICMP echo request, id 22631, seq 1, length 64
23:16:12.179246 IP 10.0.0.5 > 10.0.0.1: ICMP echo reply, id 22631, seq 1, length 64

"Node: s4" (root)
23:16:12.157633 ARP, Request who-has 10.0.0.5 tell 10.0.0.1, length 28
23:16:13.792815 STP 802.1d, Config, Flags [none], bridge-id 8000.06:52:1c:16:0b:58.8002,
length 43

"Node: s5" (root)
23:16:12.152665 ARP, Request who-has 10.0.0.5 tell 10.0.0.1, length 28
23:16:12.162698 ARP, Reply 10.0.0.5 is-at 00:00:00:05:05:05 (oui Ethernet), length 28
23:16:12.176152 IP 10.0.0.1 > 10.0.0.5: ICMP echo request, id 22631, seq 1, length 64
23:16:12.179237 IP 10.0.0.5 > 10.0.0.1: ICMP echo reply, id 22631, seq 1, length 64

```

Figura 62. Intercambio de mensajes en los switches.

Fuente: Elaboración propia.

Analizando a detalle las Figuras 60, 61 y 62, se puede observar que se cumple el intercambio de paquetes entre los hosts h1 y h5, mediante los mensajes:

ARP Request, ya que el host h1 no conoce la dirección MAC del host 5, se envía un paquete ARP Request destinado a todos los nodos en el segmento de red (con la dirección destino ff:ff:ff:ff:ff:ff, que es la dirección de difusión de la red), dicho paquete solicita al nodo que posee la dirección IP 10.0.0.5, responda esta solicitud.

ARP Reply, todos los hosts reciben el paquete de solicitud ARP Request, pero solo el host con dirección IP 10.0.0.5 responderá con el paquete de respuesta ARP Reply, proporcionando de esta forma su dirección MAC.

ICMP Echo request, luego que el host h1 ya conoce la dirección MAC del host h5, este envía un mensaje de solicitud de eco al host h5.

ICMP Echo reply, como el host h5 también ya conoce la dirección MAC del host h1, responde con un mensaje de réplica al mensaje eco solicitado.

Para comprobar el intercambio de paquetes entre estos dos hosts también se utilizó Wireshark como muestra la Figura 63, primeramente, el host h1 con dirección IP 10.0.0.1 envía el paquete ARP request con destino la dirección broadcast de la red buscando la dirección IP 10.0.0.5, luego el host h5 le responde con el paquete ARP reply, seguidamente h1 le envía a h5 una serie de datos con el paquete ICMP Echo request y h5 le responde con la misma serie de datos con el paquete ICMP Echo reply completando de esta forma el requerimiento de eco.

No.	Time	Source	Destination	Protocol	Length	Info
5378	304.721476216	00:00:00...	Broadcast	ARP	42	Who has 10.0.0.5? Tell 10.0.0.1
5379	304.745912842	00:00:00...	00:00:00...	ARP	42	10.0.0.5 is at 00:00:00:05:05:05
5382	304.733455716	00:00:00...	Broadcast	ARP	42	Who has 10.0.0.5? Tell 10.0.0.1

Frame 5378: 42 bytes on wire (336 bits), 42 bytes captured (336 bits) on interface 0 Ethernet II, Src: 00:00:00_01:01:01 (00:00:00:01:01:01), Dst: Broadcast (ff:ff:ff:ff:ff:ff) Address Resolution Protocol (request) Hardware type: Ethernet (1) Protocol type: IPv4 (0x0800) Hardware size: 6 Protocol size: 4 Opcode: request (1) Sender MAC address: 00:00:00_01:01:01 (00:00:00:01:01:01) Sender IP address: 10.0.0.1 (10.0.0.1) Target MAC address: 00:00:00_00:00:00 (00:00:00:00:00:00) Target IP address: 10.0.0.5 (10.0.0.5)						
--	--	--	--	--	--	--

No.	Time	Source	Destination	Protocol	Length	Info
5378	304.721476216	00:00:00...	Broadcast	ARP	42	Who has 10.0.0.5? Tell 10.0.0.1
5379	304.745912842	00:00:00...	00:00:00...	ARP	42	10.0.0.5 is at 00:00:00:05:05:05
5382	304.733455716	00:00:00...	Broadcast	ARP	42	Who has 10.0.0.5? Tell 10.0.0.1

Frame 5379: 42 bytes on wire (336 bits), 42 bytes captured (336 bits) on interface 0 Ethernet II, Src: 00:00:00_05:05:05 (00:00:00:05:05:05), Dst: 00:00:00_01:01:01 (00:00:00:01:01:01) Address Resolution Protocol (reply) Hardware type: Ethernet (1) Protocol type: IPv4 (0x0800) Hardware size: 6 Protocol size: 4 Opcode: reply (2) Sender MAC address: 00:00:00_05:05:05 (00:00:00:05:05:05) Sender IP address: 10.0.0.5 (10.0.0.5) Target MAC address: 00:00:00_01:01:01 (00:00:00:01:01:01) Target IP address: 10.0.0.1 (10.0.0.1)						
--	--	--	--	--	--	--

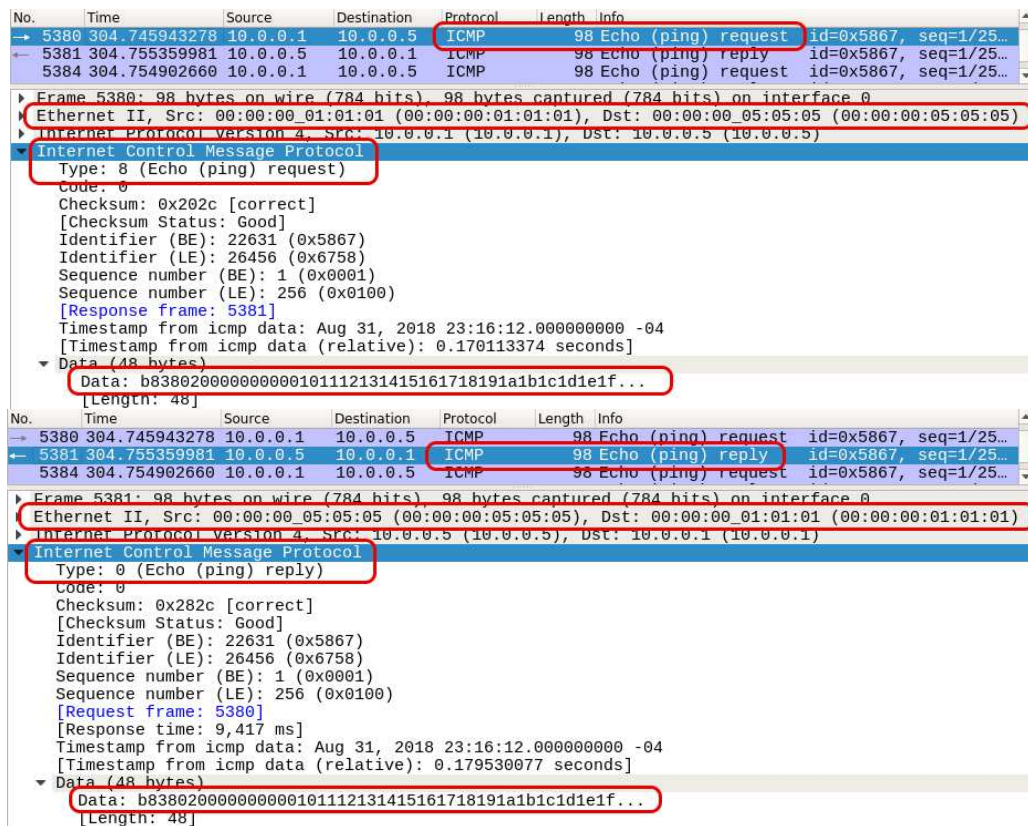


Figura 63. Intercambio de paquetes ARP e ICMP entre h1 y h5.

Fuente: Elaboración propia.

En la Figura 64, se muestra cómo quedaron las tablas de flujos de los switches luego del intercambio de paquetes mediante el comando ping, para ello ejecutamos el comando: `ovs-ofctl -O openflow13 dump-flows sX`, donde X representa al número de switch.

```

"Node: s1" (root)
root@va:~/Escritorio# ovs-ofctl -O openflow13 dump-flows s1
OFPST_FLOW reply (OF1.3) (xid=0x2):
 cookie=0x0, duration=863.700s, table=0, n_packets=313, n_bytes=18780, priority=65535,d1_dst=01:80:c2:00:00:00 actions=CONTROLLER:65535
 cookie=0x0, duration=833.439s, table=0, n_packets=2, n_bytes=245, priority=65534,in_port=2 actions=drop
 cookie=0x0, duration=586.987s, table=0, n_packets=2, n_bytes=140, priority=1,in_port=3,d1_dst=00:00:00:01:01:01 actions=output:1
 cookie=0x0, duration=586.984s, table=0, n_packets=1, n_bytes=42, priority=1,in_port=1,d1_dst=00:00:00:05:05:05 actions=output:3
 cookie=0x0, duration=863.724s, table=0, n_packets=4, n_bytes=385, priority=0 actions=CONTROLLER:65535
root@va:~/Escritorio#

"Node: s2" (root)
root@va:~/Escritorio# ovs-ofctl -O openflow13 dump-flows s2
OFPST_FLOW reply (OF1.3) (xid=0x2):
 cookie=0x0, duration=945.655s, table=0, n_packets=161, n_bytes=9660, priority=65535,d1_dst=01:80:c2:00:00:00 actions=CONTROLLER:65535
 cookie=0x0, duration=945.693s, table=0, n_packets=5, n_bytes=568, priority=0 actions=CONTROLLER:65535
root@va:~/Escritorio#

"Node: s3" (root)
root@va:~/Escritorio# ovs-ofctl -O openflow13 dump-flows s3
OFPST_FLOW reply (OF1.3) (xid=0x2):
 cookie=0x0, duration=1024.939s, table=0, n_packets=310, n_bytes=18600, priority=65535,d1_dst=01:80:c2:00:00:00 actions=CONTROLLER:65535
 cookie=0x0, duration=994.765s, table=0, n_packets=2, n_bytes=245, priority=65534,in_port=3 actions=drop
 cookie=0x0, duration=748.240s, table=0, n_packets=2, n_bytes=140, priority=1,in_port=5,d1_dst=00:00:00:01:01:01 actions=output:4
 cookie=0x0, duration=748.230s, table=0, n_packets=1, n_bytes=42, priority=1,in_port=4,d1_dst=00:00:00:05:05:05 actions=output:5
 cookie=0x0, duration=1024.973s, table=0, n_packets=9, n_bytes=971, priority=0 actions=CONTROLLER:65535
root@va:~/Escritorio#

"Node: s4" (root)
root@va:~/Escritorio# ovs-ofctl -O openflow13 dump-flows s4
OFPST_FLOW reply (OF1.3) (xid=0x2):
 cookie=0x0, duration=1091.780s, table=0, n_packets=158, n_bytes=9480, priority=65535,d1_dst=01:80:c2:00:00:00 actions=CONTROLLER:65535
 cookie=0x0, duration=1091.793s, table=0, n_packets=3, n_bytes=448, priority=0 actions=CONTROLLER:65535
root@va:~/Escritorio#

```

```
"Node: s5" (root)
root@vo:~/Escritorio# ovs-ofctl -O openflow13 dump-flows s5
DPST_FLOW reply (DP1.3) (xid=0x2):
cookie=0x0, duration=1144.633s, table=0, n_packets=5, n_bytes=300, priority=65535,d1_dst=01:80:c2:00:00:00 actions=CONTROLLER:65535
cookie=0x0, duration=867.922s, table=0, n_packets=2, n_bytes=140, priority=1,in_port=1,d1_dst=00:00:00:01:01:01 actions=output:3
cookie=0x0, duration=867.905s, table=0, n_packets=1, n_bytes=42, priority=1,in_port=3,d1_dst=00:00:00:05:05:05 actions=output:1
cookie=0x0, duration=1144.652s, table=0, n_packets=6, n_bytes=648, priority=0 actions=CONTROLLER:65535
root@vo:~/Escritorio#
```

Figura 64. Tablas de flujos de los switches.

Fuente: Elaboración propia.

En la tercera línea de cada una de las terminales mostradas en la Figura 64, se confirma que una entrada de flujo fue añadida a la tabla miss de cada switch, cuya acción o proceso estará determinada por el controlador para transferir datos con un tamaño máximo de 65535 bytes.

Luego se tiene el registro de las entradas de flujo en los switches s1, s3 y s5 con una prioridad de nivel 1, y en el caso de los switches s2 y s4 no se registran entradas de flujo ya que no están en la trayectoria de los hosts h1 y h5.

De esta forma se logró verificar el correcto funcionamiento del sistema realizando un análisis en los dispositivos virtuales, describiendo detalladamente el comportamiento en su interior y el intercambio de paquetes que ocurre en cada dispositivo, por medio de su terminal emulada y también por Wireshark.

Para analizar el funcionamiento de la aplicación se realizarán tres pruebas con diferentes variantes, la primera consistirá en utilizar la red con valores predeterminados proporcionados por el emulador, la segunda prueba se realizará con las caídas programadas de dos enlaces dentro de la red y para la tercera prueba se asignarán prioridades a cada switch antes de la emulación.

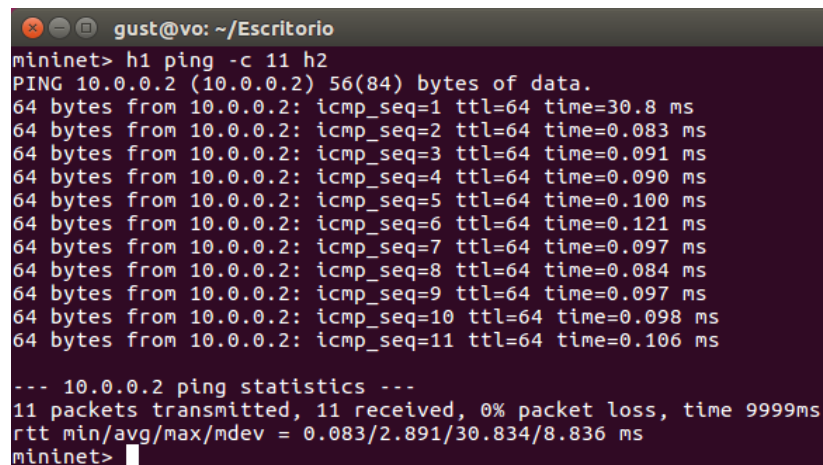
En cada prueba se medirá el tiempo de respuesta promedio de un paquete y el tiempo de respuesta total que tarda un conjunto de diez paquetes mediante la ejecución del comando ping después que el controlador haya establecido las reglas de salida en los switches, con el fin de evaluar y comparar los tiempos de comunicación que se tiene en cada caso.

4.3. Prueba 1: Red Definida por Software con Valores Predeterminados

El comando ping es un método de diagnóstico muy utilizado en operaciones de conectividad en redes de datos, ya que proporciona características básicas de una determinada conexión haciendo uso de los paquetes Echo Request y Echo Reply del protocolo ICMP, en las siguientes pruebas solo se utilizará el tiempo de respuesta que nos brinda dicho comando.

En la Figura 65, se ejecuta el comando ping en el CLI de Mininet enviando 11 paquetes de datos del host h1 al host h2, el primer paquete tiene un tiempo de respuesta elevado en comparación

con los tiempos de respuesta de los otros paquetes, debido a que este paquete es la primera entrada de flujo que ingresa al switch s1 y es enviado al controlador para ser procesado según las aplicaciones que se están ejecutando, luego el controlador se encarga de proporcionar las reglas o acciones de salida para esta entrada de flujo en todos los switches de la red. Este proceso solo ocurre cuando se tiene nuevas entradas de flujo, por lo que una vez que el switch ya conoce este tipo de paquetes el enrutamiento lo hace de manera directa sin intervención del controlador, y el tiempo de respuesta de los siguientes paquetes se reduce significativamente.



```

gust@vo: ~/Escritorio
mininet> h1 ping -c 11 h2
PING 10.0.0.2 (10.0.0.2) 56(84) bytes of data.
64 bytes from 10.0.0.2: icmp_seq=1 ttl=64 time=30.8 ms
64 bytes from 10.0.0.2: icmp_seq=2 ttl=64 time=0.083 ms
64 bytes from 10.0.0.2: icmp_seq=3 ttl=64 time=0.091 ms
64 bytes from 10.0.0.2: icmp_seq=4 ttl=64 time=0.090 ms
64 bytes from 10.0.0.2: icmp_seq=5 ttl=64 time=0.100 ms
64 bytes from 10.0.0.2: icmp_seq=6 ttl=64 time=0.121 ms
64 bytes from 10.0.0.2: icmp_seq=7 ttl=64 time=0.097 ms
64 bytes from 10.0.0.2: icmp_seq=8 ttl=64 time=0.084 ms
64 bytes from 10.0.0.2: icmp_seq=9 ttl=64 time=0.097 ms
64 bytes from 10.0.0.2: icmp_seq=10 ttl=64 time=0.098 ms
64 bytes from 10.0.0.2: icmp_seq=11 ttl=64 time=0.106 ms

--- 10.0.0.2 ping statistics ---
11 packets transmitted, 11 received, 0% packet loss, time 999ms
rtt min/avg/max/mdev = 0.083/2.891/30.834/8.836 ms
mininet>

```

Figura 65. Tiempo de conexión entre h1 y h2.

Fuente: Elaboración propia.

Como se muestra en la Figura 65, el tiempo de respuesta del primer paquete enviado desde el host h2 con dirección IP 10.0.0.2 es igual a 30.8 milisegundos, por ser un tiempo que también es consumido por el controlador es un tiempo muy elevado en comparación con el tiempo de respuesta de los otros paquetes.

Entonces, para el cálculo del tiempo de respuesta total y el tiempo de respuesta promedio (que son las variables que se evaluarán), solo se tomará en cuenta los tiempos de respuesta en los que el controlador no intervenga o cuando los switches ya tengan establecidas las reglas de salida.

Cálculo del tiempo de respuesta total del host h2 al host h1:

t_T = Tiempo de respuesta total de 10 paquetes

$$t_T = t_2 + t_3 + t_4 + t_5 + t_6 + t_7 + t_8 + t_9 + t_{10} + t_{11}$$

$$t_T = (0.083 + 0.091 + 0.090 + 0.100 + 0.121 + 0.097 + 0.084 + 0.097 + 0.098 + 0.106) \text{ [ms]}$$

$$t_T = 0.967 \text{ [ms]}$$

Entonces, el tiempo de respuesta total de 10 paquetes con un tamaño de 64 bytes cada uno, enviados desde el host h2 al host h1 es igual a 0.976 [ms], aproximadamente 1[ms].

Cálculo del tiempo de respuesta promedio entre los host h1 y h2:

t_p = Tiempo de respuesta promedio

$$t_p = \frac{t_T}{\text{Número de paquetes}}$$

$$t_p = \frac{0.967 \text{ [ms]}}{10}$$

$$t_p = 0.097 \text{ [ms]}$$

Entonces, el tiempo de respuesta promedio que se tiene en el envío de un paquete de 64 bytes entre los host h1 y h2 es de 0.097 [ms].

Las variables calculadas t_T y t_p se evaluarán y se compararán con los resultados que se tengan en las otras pruebas, realizando modificaciones programadas en la topología de la red.

A continuación, se presenta un resumen de los tiempos de respuesta de comunicación del host h1 con los demás hosts de la red realizando el mismo procedimiento, y para su respectiva consulta las capturas tomadas de la terminal del CLI de Mininet se encuentran en el Anexo C.

Tabla 15. *Tiempos de respuesta de los hosts de la red hacia h1.*

N° de ping	tiempo [ms]			
	h1 -> h2	h1 -> h3	h1 -> h4	h1 -> h5
1	30.8	16.8	32.8	33.9
2	0.083	2.60	3.03	5.75
3	0.091	2.91	2.93	6.43
4	0.090	3.08	2.97	5.76
5	0.100	2.90	2.93	5.99
6	0.121	3.37	2.90	6.67
7	0.097	3.69	2.98	5.38
8	0.084	2.47	2.80	5.95
9	0.097	2.78	3.27	5.04
10	0.098	2.97	3.26	6.10
11	0.106	3.48	2.90	5.61
t_T	0.967	30.25	29.97	58.68
t_p	0.097	3.02	3.00	5.87

Fuente: Elaboración propia.

De la Tabla 15, se puede concluir que todos los tiempos de respuesta después del primer ping mejoran significativamente, porque los switches ya tienen establecidas en sus tablas de flujos las

reglas o acciones de salida designadas por el controlador, y el enrutamiento de los paquetes lo realiza directamente sin intervención del controlador.

Con los datos de la Tabla 15, se elabora el gráfico que muestra los tiempos de respuesta de todos los hosts de la topología de red hacia el host h1.



Figura 66. Tiempos de respuesta para h1 con valores predeterminados.

Fuente: Elaboración propia.

Este análisis y verificación del tiempo de respuesta que se realizó para el host h1, se lo hizo también de la misma forma para los otros hosts de la red, donde se tuvo un comportamiento similar de los tiempos de respuesta.

4.4. Prueba 2: Red Definida por Software con la Inhabilitación Programada de Dos Enlaces

Para un análisis completo del sistema se verificará el recálculo del STP que realiza el controlador automáticamente, designando nuevamente funciones y estados a todos los puertos de los switches cuando se detecta un cambio en la arquitectura de red, en este caso específico se inhabilitará mediante instrucciones la caída de dos enlaces, el primero será la conexión directa entre el switch s1 y el switch s3, y la segunda la conexión directa entre el switch s3 y el switch s5.

Los tiempos de respuesta que se medirán en esta sección serán con la caída programada de los enlaces mencionados anteriormente, para lo cual se inhabilitara el puerto 3 del switch s1 cuya denominación es s1-eth3 y también se inhabilitara el puerto 3 del switch s5 cuya denominación es s5-eth3, como se muestra en la Figura 67.

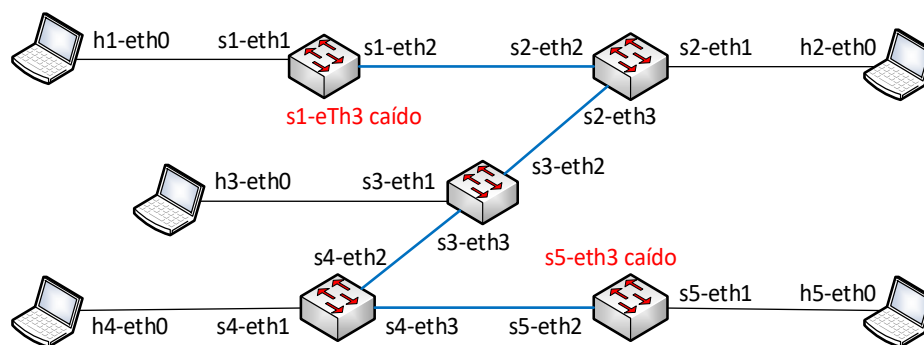


Figura 67. Topología de red con 2 puertos inhabilitados.

Fuente: Elaboración propia.

Para dicha acción se ingresa en las terminales de los switches en los que se realizarán las inhabilitaciones de los puertos y se ejecuta el comando: `ifconfig sX-sthY down`, donde X representa el número del switch y Y es el número del puerto, como se muestra a continuación:

```

"Node: s1" (root)
root@vo:~/Escritorio# ifconfig s1-eth3 down
root@vo:~/Escritorio#
    
```

```

"Node: s5" (root)
root@vo:~/Escritorio# ifconfig s5-eth3 down
root@vo:~/Escritorio#
    
```

Figura 68. Caídas de los puertos s1-eth3 y s5-eth3.

Fuente: Elaboración propia.

Luego de ejecutar esos comandos, se puede observar en la terminal del controlador como se registran las caídas de los enlaces y la inhabilitación de los puertos mencionados.

```

"Node: c0" (root)
[STP][INFO] dpid=0000000000000003: [puerto=4] PUERTO_DESIGNADO Estado: DEHABILITADO
[STP][INFO] dpid=0000000000000001: [puerto=3] Enlace caído.
[STP][INFO] dpid=0000000000000001: [puerto=3] PUERTO_DESIGNADO Estado: DEHABILITADO

"Node: c0" (root)
[STP][INFO] dpid=0000000000000003: [puerto=3] PUERTO_DESIGNADO Estado: ESCUCHA
[STP][INFO] dpid=0000000000000005: [puerto=3] Enlace caído.
[STP][INFO] dpid=0000000000000005: [puerto=3] PUERTO_DESIGNADO Estado: DEHABILITADO
    
```

Figura 69. Registro de los puertos inhabilitados.

Fuente: Elaboración propia.

Entonces la aplicación realiza nuevamente el cálculo del STP de forma automática, hasta designar nuevamente las funcionalidades y estados a los puertos afectados.

```

"Node: c0" (root)
[STP][INFO] dpid=0000000000000004: [puerto=1] PUERTO_DESIGNADO Estado: REENVIA
[STP][INFO] dpid=0000000000000004: [puerto=2] PUERTO_DESIGNADO Estado: REENVIA
[STP][INFO] dpid=0000000000000004: [puerto=3] PUERTO_ROOT Estado: REENVIA
[STP][INFO] dpid=0000000000000003: [puerto=1] PUERTO_DESIGNADO Estado: REENVIA
[STP][INFO] dpid=0000000000000003: [puerto=2] PUERTO_DESIGNADO Estado: REENVIA
[STP][INFO] dpid=0000000000000003: [puerto=3] PUERTO_ROOT Estado: REENVIA
[STP][INFO] dpid=0000000000000002: [puerto=1] PUERTO_DESIGNADO Estado: REENVIA
[STP][INFO] dpid=0000000000000002: [puerto=2] PUERTO_DESIGNADO Estado: REENVIA
[STP][INFO] dpid=0000000000000002: [puerto=3] PUERTO_ROOT Estado: REENVIA
[STP][INFO] dpid=0000000000000001: [puerto=1] PUERTO_DESIGNADO Estado: REENVIA
[STP][INFO] dpid=0000000000000001: [puerto=2] PUERTO_ROOT Estado: REENVIA
    
```

Figura 70. Reasignación de funciones y estados a los puertos.

Fuente: Elaboración propia.

Según los resultados obtenidos la arquitectura de red queda de la siguiente manera:

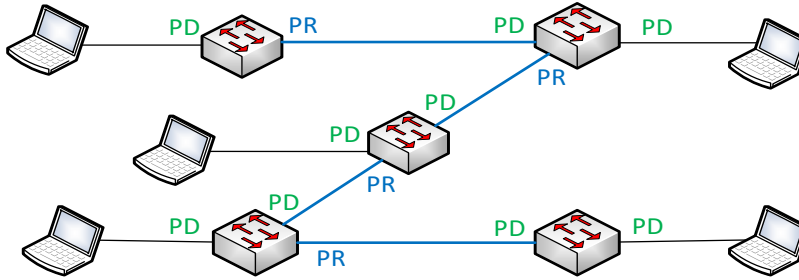


Figura 71. Esquema de la arquitectura de red con la caída de 2 puertos.

Fuente: Elaboración propia.

Teniendo esta topología de red se medirán los tiempos de respuesta hacia el host h1 de la misma forma que se lo hizo en el primer caso, enviando 11 paquetes con el comando ping a los demás hosts de la red, excluyendo el primer paquete en todos los casos por ser un tiempo de respuesta consumido también por el controlador.

En el Anexo C se encuentran las capturas correspondientes con las cuales se elabora la Tabla 16, y se hallan los valores del tiempo de respuesta total de 10 paquetes de 64 bytes cada uno, como también el tiempo de respuesta promedio en cada conexión.

Tabla 16. *Tiempos de respuesta de los hosts de la red hacia h1.*

N° de ping	tiempo [ms]			
	h1 -> h2	h1 -> h3	h1 -> h4	h1 -> h5
1	8.13	9.61	10.9	17.1
2	0.081	3.48	6.30	4.64
3	0.092	3.29	5.60	2.13
4	0.113	3.26	5.61	3.92
5	0.105	3.49	7.04	2.20
6	0.102	3.44	6.57	1.98
7	0.106	3.17	6.15	3.99
8	0.104	3.50	6.30	2.41
9	0.108	3.20	6.34	2.58
10	0.125	3.26	6.79	2.41
11	0.104	3.39	5.88	2.00
t_T	1.040	33.48	62.58	28.26
t_p	0.104	3.35	6.26	2.83

Fuente: Elaboración propia.

En la Tabla 16 se observa el tiempo de respuesta de 11 paquetes al igual que se lo hizo en la primera prueba, donde se confirma en los primeros paquetes nuevamente la intervención del controlador para asignar las reglas de salida en los switches ya que tienen un tiempo de respuesta

superior a los demás, y luego los tiempos de respuesta de los demás paquetes se estabilizan y se mantienen muy precisas entre ellas con respecto a su tiempo promedio. Con estos datos se ilustra la Figura 72 que muestra un comportamiento similar del sistema que se tuvo en la primera prueba con los valores predeterminados por el emulador.

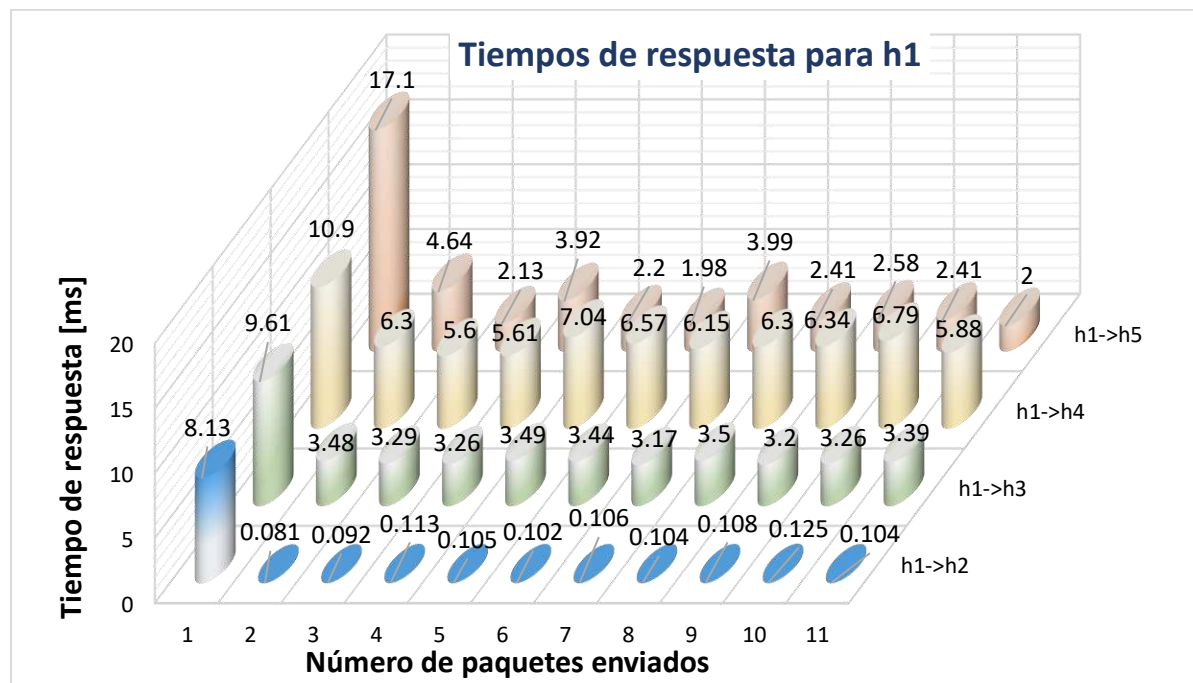


Figura 72. Tiempos de respuesta para h1 con la inhabilitación de dos enlaces.

Fuente: Elaboración propia.

Para volver al estado inicial de la topología de red se tiene que habilitar el puerto 3 de los switches s1 y s5 mediante la ejecución del comando: `ifconfig sX-sthY up`, donde X representa el número de switch y Y es el número del puerto, y luego en el controlador se puede observar cómo se detecta los puertos y enlaces activos de la red, como muestra la Figura 73.

```

"Node: c0" (root)
[STP][INFO] dpid=0000000000000001: [puerto=3] Enlace activado.
[STP][INFO] dpid=0000000000000001: [puerto=3] PUERTO_DESIGNADO Estado: ESCUCHA
[STP][INFO] dpid=0000000000000003: [puerto=4] Enlace activado.
[STP][INFO] dpid=0000000000000003: [puerto=4] PUERTO_DESIGNADO Estado: ESCUCHA

"Node: c0" (root)
[STP][INFO] dpid=0000000000000005: [puerto=3] Enlace activado.
[STP][INFO] dpid=0000000000000005: [puerto=3] PUERTO_DESIGNADO Estado: ESCUCHA
[STP][INFO] dpid=0000000000000003: [puerto=5] Enlace activado.
[STP][INFO] dpid=0000000000000003: [puerto=5] PUERTO_DESIGNADO Estado: ESCUCHA

```

Figura 73. Registro de los enlaces activados.

Fuente: Elaboración propia.

Una vez detectado los enlaces activos, el controlador realiza nuevamente el cálculo del STP siguiendo el mismo procedimiento inicial, pasando por los estados de ESCUCHA, APRENDE y

REENVIA en todos los switches hasta establecer la funcionalidad y el estado del puerto en cada switch, como muestra la Figura 74.

```

"Node: c0" (root)
[STP][INFO] dpid=0000000000000003: [puerto=1] PUERTO_DESIGNADO Estado: REENVIA
[STP][INFO] dpid=0000000000000003: [puerto=2] PUERTO_DESIGNADO Estado: REENVIA
[STP][INFO] dpid=0000000000000003: [puerto=3] PUERTO_DESIGNADO Estado: REENVIA
[STP][INFO] dpid=0000000000000003: [puerto=4] PUERTO_DESIGNADO Estado: REENVIA
[STP][INFO] dpid=0000000000000003: [puerto=5] PUERTO_ROOT Estado: REENVIA
[STP][INFO] dpid=0000000000000005: [puerto=1] PUERTO_DESIGNADO Estado: REENVIA
[STP][INFO] dpid=0000000000000005: [puerto=2] PUERTO_DESIGNADO Estado: REENVIA
[STP][INFO] dpid=0000000000000005: [puerto=3] PUERTO_DESIGNADO Estado: REENVIA
[STP][INFO] dpid=0000000000000004: [puerto=1] PUERTO_DESIGNADO Estado: REENVIA
[STP][INFO] dpid=0000000000000004: [puerto=2] PUERTO_NO_DESIGNADO Estado: BLOQUEADO
[STP][INFO] dpid=0000000000000004: [puerto=3] PUERTO_ROOT Estado: REENVIA
[STP][INFO] dpid=0000000000000001: [puerto=1] PUERTO_DESIGNADO Estado: REENVIA
[STP][INFO] dpid=0000000000000001: [puerto=2] PUERTO_NO_DESIGNADO Estado: BLOQUEADO
[STP][INFO] dpid=0000000000000001: [puerto=3] PUERTO_ROOT Estado: REENVIA
[STP][INFO] dpid=0000000000000002: [puerto=1] PUERTO_DESIGNADO Estado: REENVIA
[STP][INFO] dpid=0000000000000002: [puerto=2] PUERTO_DESIGNADO Estado: REENVIA
[STP][INFO] dpid=0000000000000002: [puerto=3] PUERTO_ROOT Estado: REENVIA

```

Figura 74. Designación de funciones y estados luego del recálculo del STP.
Fuente: Elaboración propia.

Con este procedimiento se midieron los tiempos de respuesta en la arquitectura de red de la Figura 67, que se utilizarán para compararlos con las otras pruebas más adelante, y por otro lado se comprobó el funcionamiento de la aplicación ante caídas de algún enlace dentro de la red al realizar el recálculo del STP de forma automática.

4.5. Prueba 3: Red Definida por Software con la Asignación de Prioridades a los Switches

Como tercera prueba de funcionamiento de la arquitectura de red se designará prioridades a cada uno de los switches según su datapath_id (dpid), como muestra la siguiente tabla:

Tabla 17. Asignación de prioridades al datapath_id.

datapath_id	Bridge_priority
0000000000000001	32768
0000000000000002	36864
0000000000000003	40960
0000000000000004	45056
0000000000000005	49152

Fuente: Elaboración propia.

Para el funcionamiento del sistema con esta designación de prioridades, se realizará pequeñas modificaciones en el código de los archivos redmininet.py y servivioSTP.py que se desarrollaron en el capítulo 3.

Al iniciar la emulación, el programa Mininet asigna de forma secuencial de menor a mayor el valor del dpid a cada uno de los switches, según el orden en el que se encuentran escritos en el archivo del programa si los dpids no están especificados como en los anteriores casos, para esta

prueba lo que se realizará es la adición de código en el archivo redmininet.py como se muestra en la Figura 75, en el cual se asigna un número de dpid a cada uno de los switches según los valores de la Tabla 17. Entonces, para este caso específico se modifica el archivo del programa para que el switch s3 tenga la mayor prioridad y sea asignado como Root Bridge.



```

info( '*** creando switches...\n')
s1 = net.addSwitch('s1', cls=OVSSwitch, protocols='OpenFlow13')
s2 = net.addSwitch('s2', cls=OVSSwitch, protocols='OpenFlow13')
s3 = net.addSwitch('s3', cls=OVSSwitch, protocols='OpenFlow13')
s4 = net.addSwitch('s4', cls=OVSSwitch, protocols='OpenFlow13')
s5 = net.addSwitch('s5', cls=OVSSwitch, protocols='OpenFlow13')

```

```

info( '*** creando switches...\n')
s1 = net.addSwitch('s1', cls=OVSSwitch, dpid='0000000000000005', protocols='OpenFlow13')
s2 = net.addSwitch('s2', cls=OVSSwitch, dpid='0000000000000002', protocols='OpenFlow13')
s3 = net.addSwitch('s3', cls=OVSSwitch, dpid='0000000000000001', protocols='OpenFlow13')
s4 = net.addSwitch('s4', cls=OVSSwitch, dpid='0000000000000003', protocols='OpenFlow13')
s5 = net.addSwitch('s5', cls=OVSSwitch, dpid='0000000000000004', protocols='OpenFlow13')

```

Figura 75. Designación preferencial del dpid a cada switch.

Fuente: Elaboración propia.

La segunda modificación consiste en editar las líneas de código del archivo servicioSTP.py como muestra la Figura 76, la edición en este caso solo corresponde en quitar los tres apostrofes al inicio y al final de la porción de código mostrado, con el fin de habilitar la posibilidad de asignar una determinada prioridad a cada switch según su número de dpid.



```

'''# configuracion de prioridades segun el dpid.
config = {dpid_lib.str_to_dpid('0000000000000001'):
    {'bridge': {'priority': 0x8000}},
    dpid_lib.str_to_dpid('0000000000000002'):
    {'bridge': {'priority': 0x9000}},
    dpid_lib.str_to_dpid('0000000000000003'):
    {'bridge': {'priority': 0xa000}},
    dpid_lib.str_to_dpid('0000000000000004'):
    {'bridge': {'priority': 0xb000}},
    dpid_lib.str_to_dpid('0000000000000005'):
    {'bridge': {'priority': 0xc000}}}
self.stp.set_config(config)'''

```

```

# configuracion de prioridades segun el dpid.
config = {dpid_lib.str_to_dpid('0000000000000001'):
    {'bridge': {'priority': 0x8000}},
    dpid_lib.str_to_dpid('0000000000000002'):
    {'bridge': {'priority': 0x9000}},
    dpid_lib.str_to_dpid('0000000000000003'):
    {'bridge': {'priority': 0xa000}},
    dpid_lib.str_to_dpid('0000000000000004'):
    {'bridge': {'priority': 0xb000}},
    dpid_lib.str_to_dpid('0000000000000005'):
    {'bridge': {'priority': 0xc000}}}
self.stp.set_config(config)

```

Figura 76. Habilitación de prioridades en el cálculo del STP.

Fuente: Elaboración propia.

La asignación de prioridades se lo realiza de manera ordenada según el valor del dpid de cada switch, y solo son habilitados en caso de ser necesarios como en la presente demostración.

La modificación del archivo servicioSTP.py es reconocido por el controlador Ryu mediante la recarga de sus aplicaciones, para lo cual se abre una terminal del sistema Ubuntu y se dirige al directorio donde se encuentra la carpeta del controlador Ryu, y desde allí se ejecutará el comando: `sudo python setup.py install`.

Para comprobar el correcto funcionamiento de la arquitectura de red definida por software con estas modificaciones se ejecutan las aplicaciones nuevamente en Mininet y en el controlador Ryu respectivamente, y se comprueba que se estableció la red virtual según los dpid's que se asignaron a cada switch como en la Figura 75.

```

"Node: c0" (root)
[STP][INFO] dpid=0000000000000001: [puerto=1] PUERTO_DESIGNADO Estado: REENVIA
[STP][INFO] dpid=0000000000000001: [puerto=2] PUERTO_DESIGNADO Estado: REENVIA
[STP][INFO] dpid=0000000000000001: [puerto=3] PUERTO_DESIGNADO Estado: REENVIA
[STP][INFO] dpid=0000000000000001: [puerto=4] PUERTO_DESIGNADO Estado: REENVIA
[STP][INFO] dpid=0000000000000001: [puerto=5] PUERTO_DESIGNADO Estado: REENVIA
[STP][INFO] dpid=0000000000000005: [puerto=1] PUERTO_DESIGNADO Estado: REENVIA
[STP][INFO] dpid=0000000000000005: [puerto=2] PUERTO_NO_DESIGNADO Estado: BLOQUEADO
[STP][INFO] dpid=0000000000000005: [puerto=3] PUERTO_ROOT Estado: REENVIA
[STP][INFO] dpid=0000000000000004: [puerto=1] PUERTO_DESIGNADO Estado: REENVIA
[STP][INFO] dpid=0000000000000004: [puerto=3] PUERTO_ROOT Estado: REENVIA
[STP][INFO] dpid=0000000000000004: [puerto=2] PUERTO_NO_DESIGNADO Estado: BLOQUEADO
[STP][INFO] dpid=0000000000000003: [puerto=1] PUERTO_DESIGNADO Estado: REENVIA
[STP][INFO] dpid=0000000000000003: [puerto=2] PUERTO_ROOT Estado: REENVIA
[STP][INFO] dpid=0000000000000003: [puerto=3] PUERTO_DESIGNADO Estado: REENVIA
[STP][INFO] dpid=0000000000000002: [puerto=1] PUERTO_DESIGNADO Estado: REENVIA
[STP][INFO] dpid=0000000000000002: [puerto=2] PUERTO_DESIGNADO Estado: REENVIA
[STP][INFO] dpid=0000000000000002: [puerto=3] PUERTO_ROOT Estado: REENVIA

```

Figura 77. Asignación del Root Bridge según la prioridad establecida.

Fuente: Elaboración propia.

En la siguiente figura se ilustra el establecimiento de la red para un mejor entendimiento.

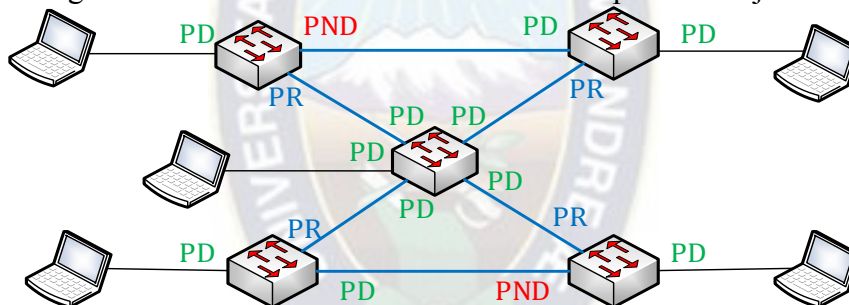


Figura 78. Establecimiento de las funciones de cada puerto con prioridad al switch s3.

Fuente: Elaboración propia.

Con este proceso se comprobó que también se puede asignar una determinada prioridad a los switches de la red mediante programación con el objetivo de tener una simulación más personalizada y de acuerdo a requerimientos específicos, y la aplicación se encarga de realizar el cálculo del STP de forma automática.

A continuación, se realiza el cálculo de los tiempos de respuesta para el host h1 siguiendo el mismo procedimiento de las anteriores pruebas, se enviará 11 paquetes mediante el comando ping en el CLI de Mininet y se tomaran las respectivas capturas de pantalla que se encuentran en el Anexo C, en base a estos datos se construirá la Tabla 18 y se realizará el cálculo del tiempo de respuesta total de 10 paquetes y el tiempo de respuesta promedio en cada conexión, sin tomar en cuenta el tiempo de respuesta del primer paquete enviado.

Tabla 18. *Tiempos de respuesta de los hosts de la red hacia h1.*

N° de ping	tiempo [ms]			
	h1 -> h2	h1 -> h3	h1 -> h4	h1 -> h5
1	31.6	15.2	20.9	20.5
2	0.096	2.34	2.25	2.14
3	0.083	2.06	2.31	2.94
4	0.093	2.04	2.21	2.33
5	0.090	2.11	2.69	2.26
6	0.091	2.17	2.12	2.11
7	0.087	2.26	2.55	2.51
8	0.090	2.40	2.26	2.50
9	0.087	1.98	2.20	2.10
10	0.092	2.62	2.63	2.29
11	0.093	2.66	2.54	2.14
t_T	0.902	22.64	23.76	23.32
t_p	0.090	2.26	2.38	2.33

Fuente: Elaboración propia.

Los tiempos de respuesta mostrados en la anterior tabla demuestran un comportamiento de la red muy similar a las anteriores dos pruebas, teniendo un primer tiempo de respuesta superior a los demás y luego la precisión de los siguientes 10 tiempos de respuesta con respecto al tiempo de respuesta promedio. Con los datos de la Tabla 18 se elabora la siguiente ilustración:

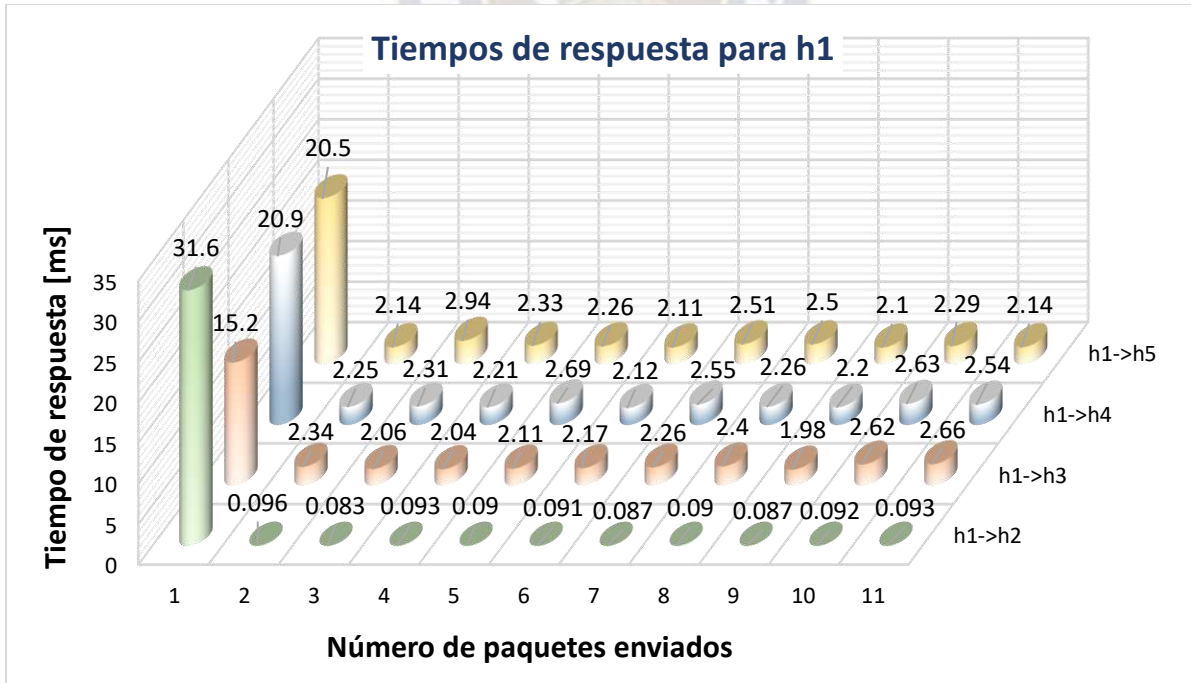


Figura 79. Tiempos de respuesta para h1 con asignación de prioridades.

Fuente: Elaboración propia.

Como última tarea, se agrupará en la Tabla 19 los tiempos de respuesta total y promedio de las tres pruebas realizadas con el fin de analizar sus diferencias y verificar si existe un cambio significativo al momento de hacer modificaciones en la topología de red.

A continuación, se extraerá los tiempos de respuesta total y promedio de las Tablas 15, 16 y 18 para analizar los resultados que se tuvo en cada prueba.

Tabla 19. *Tiempos de respuesta total y promedio de las 3 pruebas.*

		tiempo [ms]			
		h1 -> h2	h1 -> h3	h1 -> h4	h1 -> h5
Tiempo de respuesta total [t_T]	Prueba 1	0.967	30.25	29.97	58.68
	Prueba 2	1.040	33.48	62.58	28.26
	Prueba 3	0.902	22.64	23.76	23.32
Tiempo de respuesta promedio [t_p]	Prueba 1	0.097	3.02	3.00	5.87
	Prueba 2	0.104	3.35	6.26	2.83
	Prueba 3	0.090	2.26	2.38	2.33

Fuente: Elaboración propia.

En todos los casos se enviaron 11 paquetes de 64 bytes cada uno y al realizar estos cálculos no se tomó en cuenta el tiempo de respuesta del primer paquete, por ser un tiempo que transcurre en el establecimiento de las reglas en las tablas de flujos de todos los switches, por lo que solo se tomaron 10 paquetes para calcular los tiempos de respuesta total y promedio.

De los tiempos de respuesta total, se observa que se tienen valores muy próximos para cada conexión y en cada prueba, excepto en las conexiones de la prueba 1 entre el host h1 con el host h5 y de la prueba 2 del host h1 con el host h4, donde se tienen valores de los tiempos de respuesta total un poco superiores comparados con los datos de las otras pruebas.

De los tiempos de respuesta promedio, también se observa que presentan valores muy próximos en cada prueba exceptuando los valores de las conexiones comentadas en el anterior párrafo, y se nota claramente un tiempo de respuesta promedio superior o más rápido entre los host h1 y h2 comparado con los tiempos de respuesta promedio del host h1 con los demás hosts de la red.

Como se muestra en la Tabla 19, no existe una diferencia significativa en la comparación de los tiempos de respuesta en las tres pruebas realizadas, por lo que se puede afirmar que las modificaciones que se realicen en los enlaces o puertos y las configuraciones personalizadas de los switches no afectan el desempeño eficiente de la arquitectura de red definida por software.

Capítulo 5.- CONCLUSIONES Y RECOMENDACIONES

5.1. Conclusiones

En esta sección se describirán las conclusiones del presente trabajo indicando primordialmente que se cumplieron con los objetivos planteados del proyecto en base al desarrollo, funcionamiento y pruebas de la arquitectura de red definida por software.

- Se expusieron todos los conceptos básicos sobre esta nueva tecnología conocida como redes definidas por software o SDN, describiendo detalladamente conceptos abstractos y en algunos casos difíciles de entender, como todo nuevo concepto.
- Se describió de forma detallada y ordenada el procedimiento de instalación y puesta en funcionamiento de todas las herramientas necesarias para la simulación de la arquitectura de red definida por software, utilizando exclusivamente software libre.
- En el desarrollo del proyecto hubo muchos inconvenientes que se lograron superar de una u otra forma, pero se puede afirmar que el trabajo más complejo fue el diseño y desarrollo de los módulos que fueron ejecutados por el emulador Mininet y el controlador Ryu, que después de innumerables pruebas y mejoras en las líneas de código, al final se logró alcanzar un funcionamiento deseado de la red definida por software propuesta.
- Se utilizó el programa Wireshark para el análisis y comprobación de los resultados obtenidos, siendo de gran ayuda para comprender y verificar claramente los conceptos de intercambio de paquetes y protocolos que utiliza esta tecnología para su interconexión.
- Se realizaron tres pruebas dentro de la red con diferentes variantes, donde se pudo verificar el correcto funcionamiento de todo el software que conforma la red definida por software, evidenciándose el cálculo del STP de forma automática cuando se emula la red con valores determinados por Mininet, cuando se presentan enlaces caídos dentro de la red y cuando se asignan prioridades a los switches de la red.
- Los resultados obtenidos con las diferentes pruebas realizadas en la arquitectura de red definida por software, fueron satisfactorios y llegaron a cumplir con las expectativas esperadas, ya que los valores medidos y calculados de los tiempos de respuesta son mucho menores de los que se presenta en cualquier escenario de red tradicional.
- Este trabajo es una muestra pequeña de las virtudes que ofrece esta tecnología, en cuanto al paradigma de la programación en la administración de una red, y cómo se pueden personalizar

los servicios y automatizar las tareas, mediante el desarrollo de aplicaciones de acuerdo a las necesidades de cada lugar que pueden ser probadas y mejoradas mediante simulación, por lo que es una innovación tecnológica importante para los estudiantes de redes de datos y los encargados del área tecnológica de las empresas.

- Se evidenció que varios fabricantes de dispositivos de red reconocidos, empresas gigantes de contenido, en varios países del mundo y también en la región están implementando este tipo de tecnología de diferentes formas, por lo que este trabajo representa un aporte al avance tecnológico de las redes de datos en el país, motivando a implementar este tipo de conceptos desde las universidades para llevarlo a cualquier empresa que utiliza una red de datos.
- Por último, se pudo confirmar las ventajas que ofrecen las redes definidas por software frente a las redes tradicionales que tienden a desaparecer por su lógica de funcionamiento privativa y poco flexible, dando espacio de esta manera a la gestión de redes administradas exclusivamente por software o de libre personalización en los dispositivos de red como fue presentado en este trabajo.

5.2. Recomendaciones

Las siguientes recomendaciones son las más importantes al momento de iniciar una implementación de este tipo de tecnología.

- Tratándose de una tecnología relativamente nueva como en este caso, existen en internet una gran variedad de emuladores, simuladores y controladores que son actualizados constantemente y otros que se dejaron en el olvido, por lo que se tiene que hacer un estudio minucioso al momento de elegir un determinado software ya que, algunos son obsoletos y solo tienen soporte para OpenFlow v 1.0 y otros que relativamente son nuevos con muy poca documentación para su uso.
- Existen varios tipos de controladores como se observó en el presente trabajo, algunos libres y otros comerciales, se recomienda comenzar con un controlador de distribución libre y elegir el que más se adecue a las necesidades del proyecto, y también a los conocimientos sobre algún lenguaje de programación para no malgastar recursos económicos, ni tiempo.
- Se recomienda realizar varias pruebas y mejoras con todas las situaciones posibles con el fin de alcanzar la estabilidad deseada en el emulador y el controlador SDN, además de conocer la capacidad, la fiabilidad, el rendimiento de los programas y de los equipos con los cuales se

trabajar, ya que ningún software garantiza su funcionamiento al 100 % y menos aún si se trata de software libre.

- Se recomienda a la Facultad de Ingeniería y específicamente a la Carrera de Ingeniería Electrónica apoyar proyectos de investigación como éste, que tendría un impacto grande si se implementaría con equipos reales, en ambientes adecuados y equipados para su experimentación, ya que se trata de una mejora significativa en la administración de redes y su aplicación se la realiza en todos los lugares que tienen una red de datos.
- Es muy recomendable la implementación de un laboratorio exclusivamente de redes de datos en nuestra casa de estudios con la finalidad de comprobar esta clase de trabajos e incentivar a los estudiantes a obtener una certificación internacional como, por ejemplo, el CCNA (*Cisco Certified Network Associate*) de Cisco de muy buena reputación y de gran demanda laboral en nuestro medio.
- Por último, se recomienda a nuestras autoridades gubernamentales tomar en cuenta este tipo de proyectos ya que hacen uso estricto de software libre, siendo una alternativa al software de tipo privativo con el cual vienen la mayoría de los equipos de interconexión, y ser una opción más en el proceso de transición de Implementación de Software Libre y de Gobierno Electrónico en Bolivia.

5.3. Futuras Líneas de Trabajo

Por último, se expondrán tres futuras líneas de trabajo relacionadas con este trabajo:

- Para la complementación del proyecto, se aspira a implementarlo de forma física con equipos y tráfico de datos reales en ambientes de laboratorio ideales, ya que seguramente habrá que realizar algunas mejoras mediante la programabilidad del software y comparar los resultados que arrojan estos con los de la simulación.
- Se pretende trabajar en el desarrollo de aplicaciones similares para otros controladores con el objetivo de ver las diferentes ventajas y desventajas reales que se presentan frente al controlador Ryu empleado en el presente trabajo.
- Se pueden desarrollar otras aplicaciones o servicios con el fin de trabajar de manera conjunta con la aplicación del presente trabajo, y así verificar la flexibilidad y eficiencia de la red definida por software.

REFERENCIAS BIBLIOGRÁFICAS

- [1] Alejandro García, Carlos Rodríguez, Caridad Anías, Frank Casmartíño. (2014). “Controladores SDN, elementos para su elección y evaluación”.
- [2] Angelescu Silviu. (2010). “CCNA Certification All-in-One For Dummies”.
- [3] Blandón Gómez Daniel F. (2013). “OpenFlow: el protocolo del futuro”.
- [4] Carlos Alberto Contreras. (2014). “Implementación de un OpenFlow Controller para el manejo de OpenFlow Switches”.
- [5] Carpenter, B. (2002). "Middleboxes: Taxonomy and Issues". RFC 3234.
- [6] Casado M.; Michael J.; Luo J.; Freedman M.; Pettit J.; McKeown N.; S. Shenker. (2007). “Ethane: Taking Control of Enterprise”.
- [7] Christian Kuster, Joaquín Oldán. (2015). “Balanceo de carga en redes IPv4, un enfoque de Redes Definidas por Software”.
- [8] Cisco VNI. (2017). “Cisco Visual Networking Index: Forecast and Methodology, 2016-2021”.
- [9] Dan Savu, Stefan Stancu. (2014). “Software Defined Networking, technology details and openlab research overview”.
- [10] David Mejía, Iván Bernal. (2013). “Prototipos de redes definidas por software”.
- [11] Dayrene Frómata, Caridad Anías, Susana Ballester, Sergio León. (2016). “Desarrollo de aplicaciones SDN”.
- [12] Diego Kreutz, Fernando Ramos, Paulo Verissimo, Christian Esteve, Siamak A zodolmolky, Steve Uhlig. (2014). “Software-Defined Networking: A Comprehensive Survey”.
- [13] Feamster N.; Rexford J.; Zegura E. (2013). “The Road to SDN”.
- [14] IEEE Std 802.1D-2004. (2004). “802.1D IEEE Standard for Local and metropolitan area networks – Media Access Control (MAC) Bridges”.
- [15] José García Marcos. (2015). “Redes Definidas por Software. Evolución de las redes empresariales de datos, programabilidad e integración con sistemas distribuidos”.
- [16] José Leonardo Henao Ramírez. (2015). “Guía de implementación y uso del emulador de redes Mininet”.
- [17] McKeown N.; Anderson T.; Balakrishnan H.; Parulkar G.; Peterson L.; Rexford J.; Shenker S.; Turner J. (2008). “OpenFlow: Enabling Innovation in Campus Networks”.
- [18] ONF TR-521. (2016). “SDN architecture - Open Networking Foundation”.
- [19] ONF TR-535. (2016). “ONF SDN Evolution”.

- [20] ONF TS-007. (2012). “OpenFlow Switch Specification - Open Networking Foundation”.
- [21] ONF TS-016. (2014). “OpenFlow Management and Configuration Protocol”.
- [22] ONF White Paper. (2012). “Software-Defined Networking: The New Norm for Networks”.
- [23] Ramón Loera, Víctor Morales, Jesús Hernández. (diciembre 2015). “Redes Definidas por Software: beneficios y riesgos de su implementación en Universidades”.
- [24] Rudiger Fogelbach, Melvin García, Guillermo González. (2015). “Seguridad y Gestión del Riesgo en Redes Definidas por Software”.
- [25] Sebastián Castrillón Ospina. (2016). “Modelado y validación formal de una topología SDN/OpenFlow”
- [26] Siamak Azodolmolky. (October 2013). “Software Defined Network with OpenFlow”.
- [27] Vivek Tiwari. (2013). “SDN and OpenFlow for Beginners with hands on labs”.
- [28] Yan, H.; Maltz, D.; Eugene, Ng; Gogineni H.; Zhang H.; Cai Z. (2007). “Tesseract: A 4D Network Control Plane”.

Páginas web:

- [29] Investigación de redes definidas por software <https://principletechnologica.com/>
- [30] Open Networking Foundation (ONF). (2018). “Software-Defined Networking (SDN) Definition”. Extraído de: <https://www.opennetworking.org/sdn-definition/>
- [31] Página web de GitHub <https://github.com>
- [32] Página web de Open Networking Foundation (ONF) <https://opennetworking.org>
- [33] Página web de OpenFlow <http://archive.openflow.org>
- [34] Página web del emulador Mininet <http://mininet.org>
- [35] Página web del controlador Ryu <http://osrg.github.io/ryu/>
- [36] Página web del Sistema Operativo Ubuntu <http://releases.ubuntu.com/16.04/>

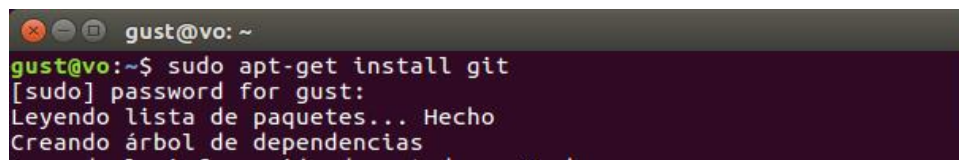
ANEXOS

ANEXO A: Programas necesarios previos a la instalación del emulador y controlador

En el presente anexo se muestra la instalación de los paquetes que son necesarios antes de instalar el emulador Mininet y el controlador Ryu, con el fin de evitar los inconvenientes que se presentaron en el proyecto por falta de librerías o ciertas dependencias.

1.- instalación de git

A continuación, se muestra el comando que se ejecuta para la instalación del controlador de versiones de software Git.

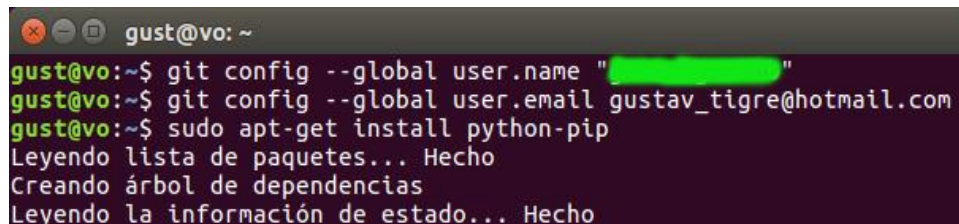


```
gust@vo: ~  
gust@vo:~$ sudo apt-get install git  
[sudo] password for gust:  
Leyendo lista de paquetes... Hecho  
Creando árbol de dependencias
```

Figura A1. Comando para la instalación de git.

Fuente: Elaboración propia.

Luego se lo configura, con el nombre de usuario y correo electrónico.



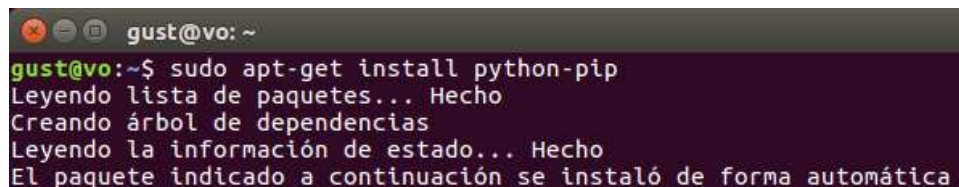
```
gust@vo: ~  
gust@vo:~$ git config --global user.name "[REDACTED]"  
gust@vo:~$ git config --global user.email gustav_tigre@hotmail.com  
gust@vo:~$ sudo apt-get install python-pip  
Leyendo lista de paquetes... Hecho  
Creando árbol de dependencias  
Leyendo la información de estado... Hecho
```

Figura A2. Comandos para la configuración de git.

Fuente: Elaboración propia.

2.- instalación de pip

A continuación, se muestra la instalación del sistema de gestión de paquetes Python.

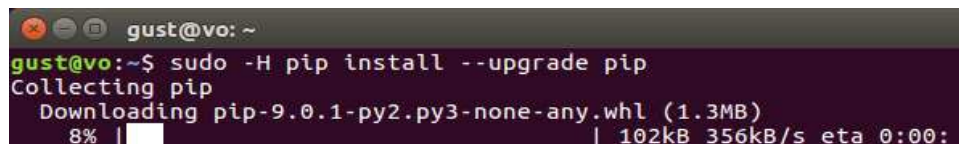


```
gust@vo: ~  
gust@vo:~$ sudo apt-get install python-pip  
Leyendo lista de paquetes... Hecho  
Creando árbol de dependencias  
Leyendo la información de estado... Hecho  
El paquete indicado a continuación se instaló de forma automática
```

Figura A3. Comando para la instalación de pip.

Fuente: Elaboración propia.

En caso de que solamente se necesite actualizarlo ejecutamos el siguiente comando.



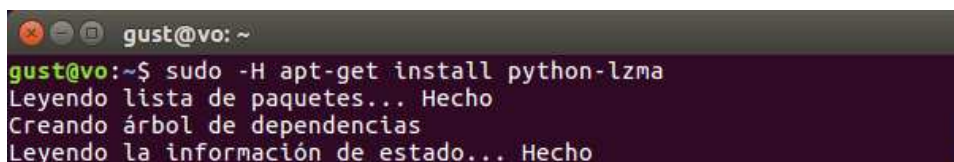
```
gust@vo: ~  
gust@vo:~$ sudo -H pip install --upgrade pip  
Collecting pip  
  Downloading pip-9.0.1-py2.py3-none-any.whl (1.3MB)  
    8% | 102kB 356kB/s eta 0:00:
```

Figura A4. Comando para la actualización de pip.

Fuente: Elaboración propia.

3.- instalación de lzma

La instalación de la librería de python lzma proporciona una interfaz para que se pueda leer y escribir datos en carpetas que fueron comprimidos.



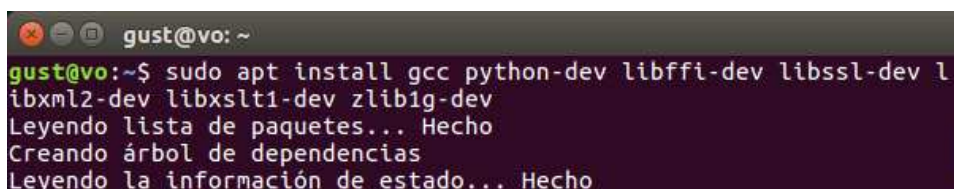
```
gust@vo: ~  
gust@vo:~$ sudo -H apt-get install python-lzma  
Leyendo lista de paquetes... Hecho  
Creando árbol de dependencias  
Leyendo la información de estado... Hecho
```

Figura A5. Comando para la instalación de python-lzma.

Fuente: Elaboración propia.

4.- Librerías para el funcionamiento del controlador Ryu

A continuación, se muestra el comando para instalar varias librerías que tiene que tener el sistema operativo antes de instalar el controlador Ryu.



```
gust@vo: ~  
gust@vo:~$ sudo apt install gcc python-dev libffi-dev libssl-dev l  
ibxml2-dev libxslt1-dev zlib1g-dev  
Leyendo lista de paquetes... Hecho  
Creando árbol de dependencias  
Leyendo la información de estado... Hecho
```

Figura A6. Instalación de librerías previas a la instalación del controlador Ryu.

Fuente: Elaboración propia.

Con la instalación previa de estos paquetes, se procede con la clonación e instalación del emulador Mininet y del controlador Ryu respectivamente (sin que se presente ningún problema), tal como se describe en el Capítulo 3 del presente proyecto.

ANEXO B: Comandos básicos del emulador Mininet

El emulador Mininet ofrece muchas bondades para experimentar virtualmente y tiene una curva de aprendizaje muy baja por su sencillez como se expuso en el Capítulo 2, en este anexo se presentan los comandos más básicos aplicados a topologías de redes comunes con el objetivo de familiarizarse con el manejo de este emulador.

Sobre la sintaxis de comandos que se usa en Mininet es preferible aclarar que:

\$ precede a comandos del sistema operativo Linux.

mininet> precede a los comandos que se ejecutarán con el CLI de Mininet.

precede a los comandos de Linux que se escriben en la terminal como usuario root.

Para la personalización de una red específica se debe seguir el siguiente formato:

sudo mn --[opción]=[parámetro],[argumento] --[opción]=[parámetro],[argumento]

La cual varía según el tipo de opción, parámetro y argumento utilizado, a continuación, se expondrán los más importantes:

- help** despliega en la terminal un listado de las posibles opciones que se tiene en Mininet.
- switch**=[**parametro**] con esta opción se invoca a un tipo específico de switch para emularlo, y entre sus parámetros se puede elegir: default, ivs, ovs, ovsk, ovsbr, ovsl, user.
- host** invoca a la creación de un host virtual.
- controller**=[**parametro**] permite invocar un tipo de controlador para la red, y entre sus parámetros se puede elegir: default, none, nox, ovsc, remote, ryu.
- link**=[**parametros**] permite configurar características del enlace, y entre sus parámetros se puede elegir: default, tc, bw, delay, loss.
- topo**=[**parametro**] permite configurar el tipo de topología a emular, y entre sus parámetros se puede elegir: minimal, single, reversed, torus, tree.
- clean** limpia los registros de emulación hechos luego de finalizar la emulación.
- custom**=[**ruta del archivo**] lee archivos escritos con extensión “.py” creados para emular una red personalizada, como en el presente proyecto.

Tabla B1. *Estructura de los comandos constructores de Mininet.*

Usuario	Mininet		Opción		Parámetro		Argumento
sudo	mn	--	help	=		,	
			switch		default		
					ovs		
					ovsk		
					ovsbr		stp=[1/0]
					ovsl		
					ivs		
					user		
			host		cfs		
					rt		
					default		
			controller		none		
					nox		
					ovsc		
					ryu		
			link		default		
					tc		bw=[BW], delay=[TIME], loss=[%]
			topo		minimal		
					linear		k=[SW], n=[HOST]
single	K=[HOST]						

Usuario	Mininet		Opción		Parámetro		Argumento
					reversed		K=[HOST]
					tree		depth=[ALTURA], fanout=[RAMAS]
					torus		x=[N], t=[N]
			clean				
			custom		<fichero.py>		
					cli		
					none		
					build		
					pingpair		
					pingall		
					iperf		
					iperfudp		
					all		
			xterms				
			ibase		[IP]/[mask]		
			mac				
			arp				
					error		
					warning		
					info		
					debug		
					output		
			inamespace				
			listenport		[port]		
			nat				
			version				

Fuente: Elaboración propia recopilado de varias fuentes.

Luego de ejecutar el comando `sudo mn` junto con alguno de las opciones y/o parámetros mencionados anteriormente, se inicia la emulación y la interfaz de línea de comandos de Mininet está a la espera para ejecutar instrucciones sobre la topología de red, a continuación, se describirán los comandos más importantes del CLI de Mininet.

>**help** muestra en pantalla los distintos formatos de uso de las instrucciones en el CLI, acompañada de la documentación necesaria para cada comando.

>**dump** muestra la información sobre los dispositivos como: interfaces de uso, direcciones IP e identificador de puerto.

>**net** muestra el nombre de las interfaces con las que se conectan los dispositivos de la topología.

>**intfs** lista las interfaces utilizadas por cada dispositivo.

- >**nodes** muestra la denominación de todos los nodos disponibles.
- >**ports** muestra de forma ordenada el número de puerto que usa cada interfaz.
- >**pingallfull** realiza una prueba de conectividad entre todos los hosts.
- >**iperf [hX] [hY]** realiza una prueba de rendimiento de ancho de banda TCP entre hosts específicos.
- >**iperfudp [bw] [hX] [hY]** realiza una prueba de rendimiento de ancho de banda UDP entre hosts específicos.
- >**xterm [nodoX] [nodoY]** abre una terminal para cada nodo específico.
- >**exit** finaliza la emulación y cierra el programa.

Tabla B2. *Lista de comandos para el CLI de Mininet.*

Comando	Argumento	descripción
help		muestra información documentada
dump		información detallada de la red
net		información sobre los enlaces
intfs		información sobre las interfaces
nodes		información sobre los nodos usados
ports		información sobre los puertos usados
time	[COMANDO]	tiempo de ejecución de un comando
switch	[SW] [start/stop]	inicia y finaliza un switch
links		información sobre los enlaces
link	[NODO1] [NODO2] [up/down]	habilita, deshabilita enlaces
noecho	[HOST] [CMD argumentos]	ejecuta comandos shell en hosts
sh	[CMD argumentos]	ejecuta comandos shell en anfitrión
source	<file>	lectura de ficheros
pingall		prueba de conexión de la red
pingallfull		prueba de conexión con resultados
pingpair		prueba de conexión entre h1 y h2
pingpairfull		prueba de conexión detallada entre h1 y h2
iperf	[HOST1] [HOST2]	prueba de ancho de banda TCP
iperfudp	[BW] [HOST1] [HOST2]	prueba de ancho de banda UDP
px	[PYTHON]	ejecución de declaraciones python
py	[OBJETO.FUNCION()]	ejecución de expresiones python
xterm	[HOSTn] ...	abre terminales independientes
x	[HOST] [CMD argumentos]	crea túneles X11
gterm	[HOSTn] ...	abre terminales GUI
dpctl	[COMANDO][argumentos]	ejecuta funciones dpctl
EOF		finaliza la emulación
exit		finaliza la emulación

Comando	Argumento	descripción
quit		finaliza la emulación

Fuente: Elaboración propia recopilado de varias fuentes.

Por último, se muestran varios comandos utilizados para la administración de los switches OpenFlow, los cuales permiten hacer uso de los switches virtuales utilizados estableciendo reglas puntuales para los flujos.

El formato para este tipo de comando es el siguiente:

dpctl [opciones] comando [switch] [argumento]

[opciones] no es un campo obligatorio y es usado con el fin de informar sobre las características generales del sistema y los cambios que ocurren en la emulación, algunas variables pueden ser: --strict, --timeout=[SEGUNDOS], --verbose, --log-file, --help y --version.

comando es el espacio en el que se escribe el comando que permite ejecutar funcionalidades que se desean hacer en el switch, entre los más importantes están: show [sX], status [sX], dump-desc [sX], dump-tables [sX], dump-flows [sX], benchmark [dispositivo] [n] [contador].

[switch] este campo asigna al switch que se le aplicará el comando.

[argumento] se incluye los argumentos según los comandos que se utilizan.

Tabla B3. Estructura para los comandos dpctl.

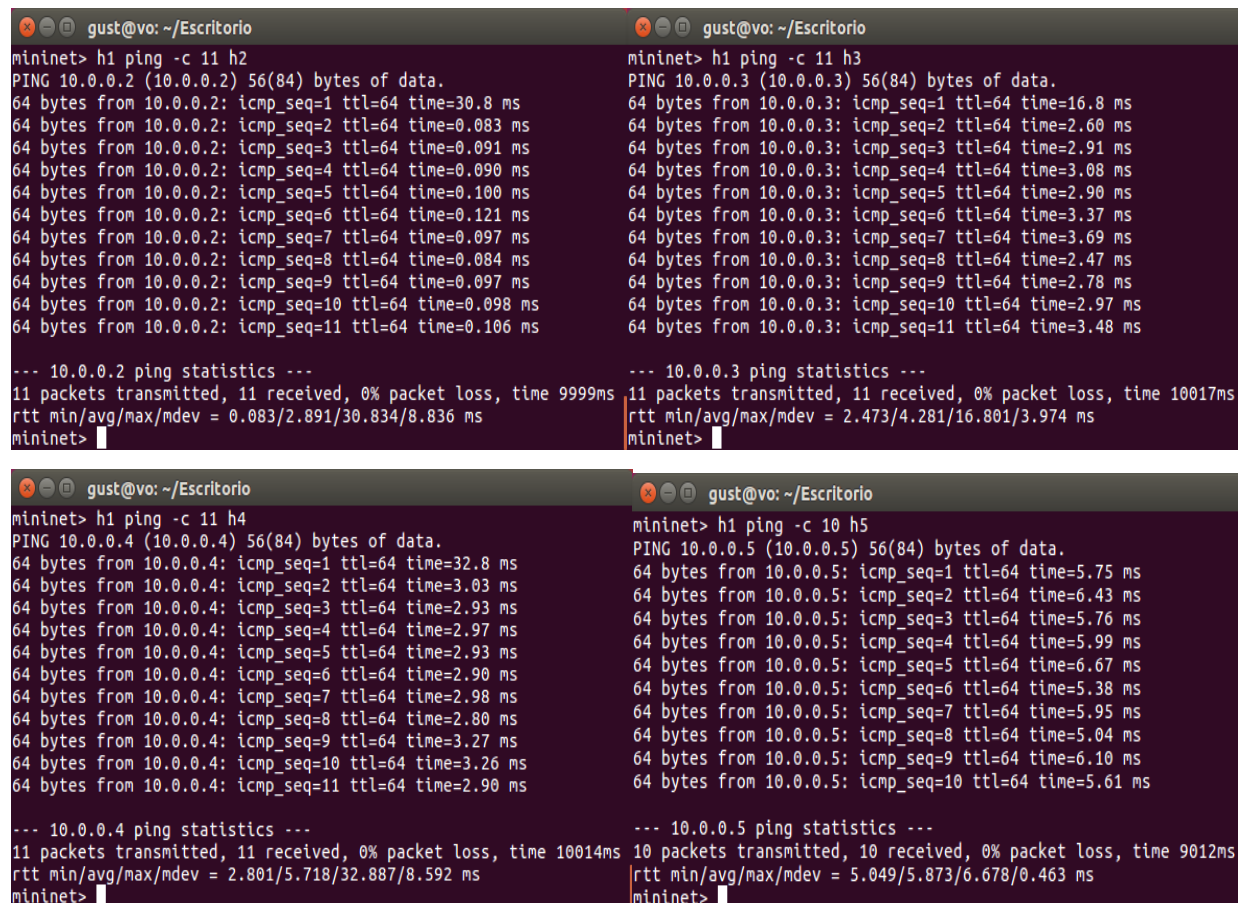
dpctl	Opciones		Comando	Dispositivo	Argumentos
dpctl	--	timeout	=[SEG]	show	
		verbose		status	
		log-file	<file>	show-protostat	
		help		dump-desc	
		version		dump-tables	
			mod-port	tcp:IP:puerto	[up/down/flood/noflood]
			dump-ports		[PUERTO]
			dump-flows		[FLUJO]
			dump-aggregate		[FLUJO]
			monitor		
			probe		
			ping		[N]
			benchmark		[N][CONTADOR]
			add-flow		[FLUJO]
			add-flows		<file>
			mod-flows		[FLUJO]
			del-flows		[FLUJO]

Fuente: Elaboración propia recopilado de varias fuentes.

ANEXO C: Captura de los tiempos de respuesta de las pruebas realizadas

Prueba 1: Red definida por software con valores predeterminados

Con los datos de estas capturas se realiza la Tabla 15.



```
gust@vo: ~/Escritorio
mininet> h1 ping -c 11 h2
PING 10.0.0.2 (10.0.0.2) 56(84) bytes of data.
64 bytes from 10.0.0.2: icmp_seq=1 ttl=64 time=30.8 ms
64 bytes from 10.0.0.2: icmp_seq=2 ttl=64 time=0.083 ms
64 bytes from 10.0.0.2: icmp_seq=3 ttl=64 time=0.091 ms
64 bytes from 10.0.0.2: icmp_seq=4 ttl=64 time=0.090 ms
64 bytes from 10.0.0.2: icmp_seq=5 ttl=64 time=0.100 ms
64 bytes from 10.0.0.2: icmp_seq=6 ttl=64 time=0.121 ms
64 bytes from 10.0.0.2: icmp_seq=7 ttl=64 time=0.097 ms
64 bytes from 10.0.0.2: icmp_seq=8 ttl=64 time=0.084 ms
64 bytes from 10.0.0.2: icmp_seq=9 ttl=64 time=0.097 ms
64 bytes from 10.0.0.2: icmp_seq=10 ttl=64 time=0.098 ms
64 bytes from 10.0.0.2: icmp_seq=11 ttl=64 time=0.106 ms

--- 10.0.0.2 ping statistics ---
11 packets transmitted, 11 received, 0% packet loss, time 9999ms
rtt min/avg/max/mdev = 0.083/2.891/30.834/8.836 ms
mininet>

gust@vo: ~/Escritorio
mininet> h1 ping -c 11 h3
PING 10.0.0.3 (10.0.0.3) 56(84) bytes of data.
64 bytes from 10.0.0.3: icmp_seq=1 ttl=64 time=16.8 ms
64 bytes from 10.0.0.3: icmp_seq=2 ttl=64 time=2.60 ms
64 bytes from 10.0.0.3: icmp_seq=3 ttl=64 time=2.91 ms
64 bytes from 10.0.0.3: icmp_seq=4 ttl=64 time=3.08 ms
64 bytes from 10.0.0.3: icmp_seq=5 ttl=64 time=2.90 ms
64 bytes from 10.0.0.3: icmp_seq=6 ttl=64 time=3.37 ms
64 bytes from 10.0.0.3: icmp_seq=7 ttl=64 time=3.69 ms
64 bytes from 10.0.0.3: icmp_seq=8 ttl=64 time=2.47 ms
64 bytes from 10.0.0.3: icmp_seq=9 ttl=64 time=2.78 ms
64 bytes from 10.0.0.3: icmp_seq=10 ttl=64 time=2.97 ms
64 bytes from 10.0.0.3: icmp_seq=11 ttl=64 time=3.48 ms

--- 10.0.0.3 ping statistics ---
11 packets transmitted, 11 received, 0% packet loss, time 10017ms
rtt min/avg/max/mdev = 2.473/4.281/16.801/3.974 ms
mininet>

gust@vo: ~/Escritorio
mininet> h1 ping -c 11 h4
PING 10.0.0.4 (10.0.0.4) 56(84) bytes of data.
64 bytes from 10.0.0.4: icmp_seq=1 ttl=64 time=32.8 ms
64 bytes from 10.0.0.4: icmp_seq=2 ttl=64 time=3.03 ms
64 bytes from 10.0.0.4: icmp_seq=3 ttl=64 time=2.93 ms
64 bytes from 10.0.0.4: icmp_seq=4 ttl=64 time=2.97 ms
64 bytes from 10.0.0.4: icmp_seq=5 ttl=64 time=2.93 ms
64 bytes from 10.0.0.4: icmp_seq=6 ttl=64 time=2.90 ms
64 bytes from 10.0.0.4: icmp_seq=7 ttl=64 time=2.98 ms
64 bytes from 10.0.0.4: icmp_seq=8 ttl=64 time=2.80 ms
64 bytes from 10.0.0.4: icmp_seq=9 ttl=64 time=3.27 ms
64 bytes from 10.0.0.4: icmp_seq=10 ttl=64 time=3.26 ms
64 bytes from 10.0.0.4: icmp_seq=11 ttl=64 time=2.90 ms

--- 10.0.0.4 ping statistics ---
11 packets transmitted, 11 received, 0% packet loss, time 10014ms
rtt min/avg/max/mdev = 2.801/5.718/32.887/8.592 ms
mininet>

gust@vo: ~/Escritorio
mininet> h1 ping -c 10 h5
PING 10.0.0.5 (10.0.0.5) 56(84) bytes of data.
64 bytes from 10.0.0.5: icmp_seq=1 ttl=64 time=5.75 ms
64 bytes from 10.0.0.5: icmp_seq=2 ttl=64 time=6.43 ms
64 bytes from 10.0.0.5: icmp_seq=3 ttl=64 time=5.76 ms
64 bytes from 10.0.0.5: icmp_seq=4 ttl=64 time=5.99 ms
64 bytes from 10.0.0.5: icmp_seq=5 ttl=64 time=6.67 ms
64 bytes from 10.0.0.5: icmp_seq=6 ttl=64 time=5.38 ms
64 bytes from 10.0.0.5: icmp_seq=7 ttl=64 time=5.95 ms
64 bytes from 10.0.0.5: icmp_seq=8 ttl=64 time=5.04 ms
64 bytes from 10.0.0.5: icmp_seq=9 ttl=64 time=6.10 ms
64 bytes from 10.0.0.5: icmp_seq=10 ttl=64 time=5.61 ms

--- 10.0.0.5 ping statistics ---
10 packets transmitted, 10 received, 0% packet loss, time 9012ms
rtt min/avg/max/mdev = 5.049/5.873/6.678/0.463 ms
mininet>
```

Figura C1. Capturas de pantalla para la prueba 1.

Fuente: Elaboración propia.

Cabe aclarar que en el momento de realizar las pruebas de comunicación entre los host h1 y h5 solo se envían 10 paquetes, ya que el primer paquete que es procesado por el controlador se lo envió en la sección 4.2 al realizar las pruebas de funcionamiento básicas de la red definida por software, como se muestra en la Figura 59.

Prueba 2: Red definida por software con la inhabilitación programada de dos enlaces

Con los datos de estas capturas se realiza la Tabla 16.


```

gust@vo: ~/Escritorio
mininet> h1 ping -c 11 h2
PING 10.0.0.2 (10.0.0.2) 56(84) bytes of data.
64 bytes from 10.0.0.2: icmp_seq=1 ttl=64 time=8.13 ms
64 bytes from 10.0.0.2: icmp_seq=2 ttl=64 time=0.081 ms
64 bytes from 10.0.0.2: icmp_seq=3 ttl=64 time=0.092 ms
64 bytes from 10.0.0.2: icmp_seq=4 ttl=64 time=0.113 ms
64 bytes from 10.0.0.2: icmp_seq=5 ttl=64 time=0.105 ms
64 bytes from 10.0.0.2: icmp_seq=6 ttl=64 time=0.102 ms
64 bytes from 10.0.0.2: icmp_seq=7 ttl=64 time=0.106 ms
64 bytes from 10.0.0.2: icmp_seq=8 ttl=64 time=0.104 ms
64 bytes from 10.0.0.2: icmp_seq=9 ttl=64 time=0.108 ms
64 bytes from 10.0.0.2: icmp_seq=10 ttl=64 time=0.125 ms
64 bytes from 10.0.0.2: icmp_seq=11 ttl=64 time=0.104 ms

--- 10.0.0.2 ping statistics ---
11 packets transmitted, 11 received, 0% packet loss, time 999ms
rtt min/avg/max/mdev = 0.081/0.833/8.131/2.307 ms
mininet>

gust@vo: ~/Escritorio
mininet> h1 ping -c 11 h3
PING 10.0.0.3 (10.0.0.3) 56(84) bytes of data.
64 bytes from 10.0.0.3: icmp_seq=1 ttl=64 time=9.61 ms
64 bytes from 10.0.0.3: icmp_seq=2 ttl=64 time=3.48 ms
64 bytes from 10.0.0.3: icmp_seq=3 ttl=64 time=3.29 ms
64 bytes from 10.0.0.3: icmp_seq=4 ttl=64 time=3.26 ms
64 bytes from 10.0.0.3: icmp_seq=5 ttl=64 time=3.49 ms
64 bytes from 10.0.0.3: icmp_seq=6 ttl=64 time=3.44 ms
64 bytes from 10.0.0.3: icmp_seq=7 ttl=64 time=3.17 ms
64 bytes from 10.0.0.3: icmp_seq=8 ttl=64 time=3.50 ms
64 bytes from 10.0.0.3: icmp_seq=9 ttl=64 time=3.20 ms
64 bytes from 10.0.0.3: icmp_seq=10 ttl=64 time=3.26 ms
64 bytes from 10.0.0.3: icmp_seq=11 ttl=64 time=3.39 ms

--- 10.0.0.3 ping statistics ---
11 packets transmitted, 11 received, 0% packet loss, time 10015ms
rtt min/avg/max/mdev = 3.170/3.921/9.616/1.805 ms
mininet>

gust@vo: ~/Escritorio
mininet> h1 ping -c 11 h4
PING 10.0.0.4 (10.0.0.4) 56(84) bytes of data.
64 bytes from 10.0.0.4: icmp_seq=1 ttl=64 time=10.9 ms
64 bytes from 10.0.0.4: icmp_seq=2 ttl=64 time=6.30 ms
64 bytes from 10.0.0.4: icmp_seq=3 ttl=64 time=5.60 ms
64 bytes from 10.0.0.4: icmp_seq=4 ttl=64 time=5.61 ms
64 bytes from 10.0.0.4: icmp_seq=5 ttl=64 time=7.04 ms
64 bytes from 10.0.0.4: icmp_seq=6 ttl=64 time=6.57 ms
64 bytes from 10.0.0.4: icmp_seq=7 ttl=64 time=6.15 ms
64 bytes from 10.0.0.4: icmp_seq=8 ttl=64 time=6.30 ms
64 bytes from 10.0.0.4: icmp_seq=9 ttl=64 time=6.34 ms
64 bytes from 10.0.0.4: icmp_seq=10 ttl=64 time=6.79 ms
64 bytes from 10.0.0.4: icmp_seq=11 ttl=64 time=5.88 ms

--- 10.0.0.4 ping statistics ---
11 packets transmitted, 11 received, 0% packet loss, time 10014ms
rtt min/avg/max/mdev = 5.601/6.687/10.955/1.418 ms
mininet>

gust@vo: ~/Escritorio
mininet> h1 ping -c 11 h5
PING 10.0.0.5 (10.0.0.5) 56(84) bytes of data.
64 bytes from 10.0.0.5: icmp_seq=1 ttl=64 time=17.1 ms
64 bytes from 10.0.0.5: icmp_seq=2 ttl=64 time=4.64 ms
64 bytes from 10.0.0.5: icmp_seq=3 ttl=64 time=2.13 ms
64 bytes from 10.0.0.5: icmp_seq=4 ttl=64 time=3.92 ms
64 bytes from 10.0.0.5: icmp_seq=5 ttl=64 time=2.20 ms
64 bytes from 10.0.0.5: icmp_seq=6 ttl=64 time=1.98 ms
64 bytes from 10.0.0.5: icmp_seq=7 ttl=64 time=3.99 ms
64 bytes from 10.0.0.5: icmp_seq=8 ttl=64 time=2.41 ms
64 bytes from 10.0.0.5: icmp_seq=9 ttl=64 time=2.58 ms
64 bytes from 10.0.0.5: icmp_seq=10 ttl=64 time=2.41 ms
64 bytes from 10.0.0.5: icmp_seq=11 ttl=64 time=2.00 ms

--- 10.0.0.5 ping statistics ---
11 packets transmitted, 11 received, 0% packet loss, time 10012ms
rtt min/avg/max/mdev = 1.981/4.131/17.132/4.204 ms
mininet>

```

Figura C2. Capturas de pantalla para la prueba 2.
Fuente: Elaboración propia.

Prueba 3: Red definida por software con la asignación de prioridades a los switches

Con los datos de estas capturas se realiza la Tabla 18.

```

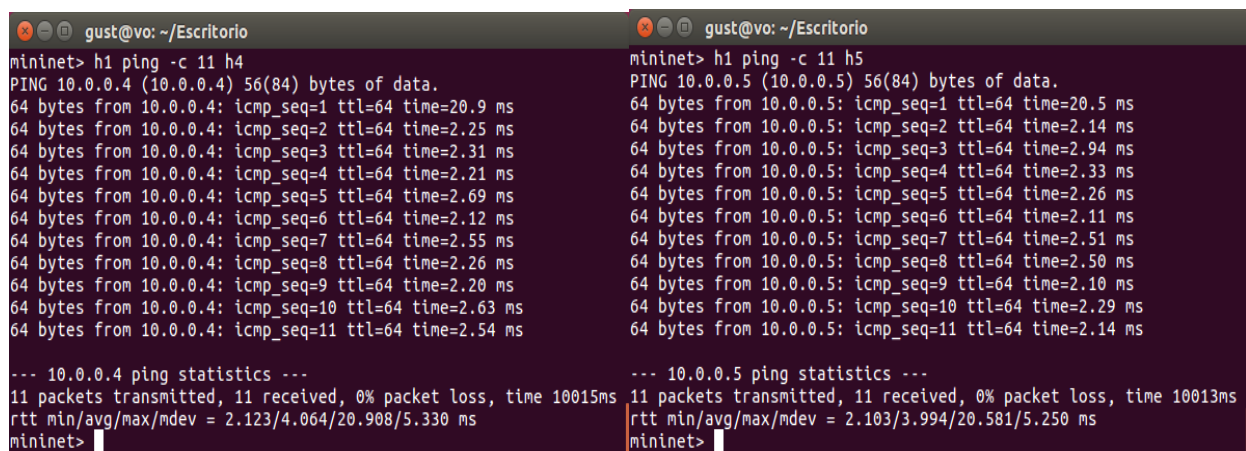
gust@vo: ~/Escritorio
mininet> h1 ping -c 11 h2
PING 10.0.0.2 (10.0.0.2) 56(84) bytes of data.
64 bytes from 10.0.0.2: icmp_seq=1 ttl=64 time=31.6 ms
64 bytes from 10.0.0.2: icmp_seq=2 ttl=64 time=0.096 ms
64 bytes from 10.0.0.2: icmp_seq=3 ttl=64 time=0.083 ms
64 bytes from 10.0.0.2: icmp_seq=4 ttl=64 time=0.093 ms
64 bytes from 10.0.0.2: icmp_seq=5 ttl=64 time=0.090 ms
64 bytes from 10.0.0.2: icmp_seq=6 ttl=64 time=0.091 ms
64 bytes from 10.0.0.2: icmp_seq=7 ttl=64 time=0.087 ms
64 bytes from 10.0.0.2: icmp_seq=8 ttl=64 time=0.090 ms
64 bytes from 10.0.0.2: icmp_seq=9 ttl=64 time=0.087 ms
64 bytes from 10.0.0.2: icmp_seq=10 ttl=64 time=0.092 ms
64 bytes from 10.0.0.2: icmp_seq=11 ttl=64 time=0.093 ms

--- 10.0.0.2 ping statistics ---
11 packets transmitted, 11 received, 0% packet loss, time 999ms
rtt min/avg/max/mdev = 0.083/2.961/31.673/9.079 ms
mininet>

gust@vo: ~/Escritorio
mininet> h1 ping -c 11 h3
PING 10.0.0.3 (10.0.0.3) 56(84) bytes of data.
64 bytes from 10.0.0.3: icmp_seq=1 ttl=64 time=15.2 ms
64 bytes from 10.0.0.3: icmp_seq=2 ttl=64 time=2.34 ms
64 bytes from 10.0.0.3: icmp_seq=3 ttl=64 time=2.06 ms
64 bytes from 10.0.0.3: icmp_seq=4 ttl=64 time=2.04 ms
64 bytes from 10.0.0.3: icmp_seq=5 ttl=64 time=2.11 ms
64 bytes from 10.0.0.3: icmp_seq=6 ttl=64 time=2.17 ms
64 bytes from 10.0.0.3: icmp_seq=7 ttl=64 time=2.26 ms
64 bytes from 10.0.0.3: icmp_seq=8 ttl=64 time=2.40 ms
64 bytes from 10.0.0.3: icmp_seq=9 ttl=64 time=1.98 ms
64 bytes from 10.0.0.3: icmp_seq=10 ttl=64 time=2.62 ms
64 bytes from 10.0.0.3: icmp_seq=11 ttl=64 time=2.66 ms

--- 10.0.0.3 ping statistics ---
11 packets transmitted, 11 received, 0% packet loss, time 10013ms
rtt min/avg/max/mdev = 1.982/3.445/15.209/3.726 ms
mininet>

```



The image shows two terminal windows side-by-side. The left window shows the output of a ping command from host h1 to host h4 (10.0.0.4). It displays 11 successful ping attempts with varying response times between 2.09 ms and 2.69 ms. The right window shows the output of a ping command from host h1 to host h5 (10.0.0.5). It also displays 11 successful ping attempts with varying response times between 2.10 ms and 2.51 ms. Both windows include a summary of the ping statistics at the bottom, showing 100% packet delivery and average response times.

```
gust@vo: ~/Escritorio
mininet> h1 ping -c 11 h4
PING 10.0.0.4 (10.0.0.4) 56(84) bytes of data.
64 bytes from 10.0.0.4: icmp_seq=1 ttl=64 time=20.9 ms
64 bytes from 10.0.0.4: icmp_seq=2 ttl=64 time=2.25 ms
64 bytes from 10.0.0.4: icmp_seq=3 ttl=64 time=2.31 ms
64 bytes from 10.0.0.4: icmp_seq=4 ttl=64 time=2.21 ms
64 bytes from 10.0.0.4: icmp_seq=5 ttl=64 time=2.69 ms
64 bytes from 10.0.0.4: icmp_seq=6 ttl=64 time=2.12 ms
64 bytes from 10.0.0.4: icmp_seq=7 ttl=64 time=2.55 ms
64 bytes from 10.0.0.4: icmp_seq=8 ttl=64 time=2.26 ms
64 bytes from 10.0.0.4: icmp_seq=9 ttl=64 time=2.20 ms
64 bytes from 10.0.0.4: icmp_seq=10 ttl=64 time=2.63 ms
64 bytes from 10.0.0.4: icmp_seq=11 ttl=64 time=2.54 ms

--- 10.0.0.4 ping statistics ---
11 packets transmitted, 11 received, 0% packet loss, time 10015ms
rtt min/avg/max/mdev = 2.123/4.064/20.908/5.330 ms
mininet>

gust@vo: ~/Escritorio
mininet> h1 ping -c 11 h5
PING 10.0.0.5 (10.0.0.5) 56(84) bytes of data.
64 bytes from 10.0.0.5: icmp_seq=1 ttl=64 time=20.5 ms
64 bytes from 10.0.0.5: icmp_seq=2 ttl=64 time=2.14 ms
64 bytes from 10.0.0.5: icmp_seq=3 ttl=64 time=2.94 ms
64 bytes from 10.0.0.5: icmp_seq=4 ttl=64 time=2.33 ms
64 bytes from 10.0.0.5: icmp_seq=5 ttl=64 time=2.26 ms
64 bytes from 10.0.0.5: icmp_seq=6 ttl=64 time=2.11 ms
64 bytes from 10.0.0.5: icmp_seq=7 ttl=64 time=2.51 ms
64 bytes from 10.0.0.5: icmp_seq=8 ttl=64 time=2.50 ms
64 bytes from 10.0.0.5: icmp_seq=9 ttl=64 time=2.10 ms
64 bytes from 10.0.0.5: icmp_seq=10 ttl=64 time=2.29 ms
64 bytes from 10.0.0.5: icmp_seq=11 ttl=64 time=2.14 ms

--- 10.0.0.5 ping statistics ---
11 packets transmitted, 11 received, 0% packet loss, time 10013ms
rtt min/avg/max/mdev = 2.103/3.994/20.581/5.250 ms
mininet>
```

Figura C3. Capturas de pantalla para la prueba 3.
Fuente: Elaboración propia.

ANEXO D: Analizador de protocolos Wireshark

En este anexo se realizará una breve descripción de lo que es el analizador de protocolos Wireshark utilizado para verificar y analizar los resultados obtenidos en las pruebas realizadas.

Se trata de una potente herramienta grafica heredero del software Ethereal que desde el 2006 es denominado Wireshark, catalogado como el mejor capturador, identificador y analizador de protocolos o paquetes que atraviesan un determinado nodo de red. El uso de este software libre es muy utilizado en el área de redes y presenta las siguientes características:

- Disponible para las plataformas más populares LINUX, Windows y Mac OS.
- La captura de paquetes se los realiza desde una o varias interfaces de red.
- Permite el análisis detallado de toda la información del protocolo en el paquete capturado.
- Cuenta con la capacidad de importar y exportar los paquetes capturados desde y hacia otros programas en archivos de extensión “.pcap”.
- Tiene la función de filtrar paquetes que cumplan con un criterio definido previamente.
- Permite realizar estadísticas de los paquetes capturados.
- Se puede identificar y configurar los paquetes de nuestra preferencia mediante el uso de colores.

A continuación, se describe concisamente las áreas que muestra por defecto el programa Wireshark.

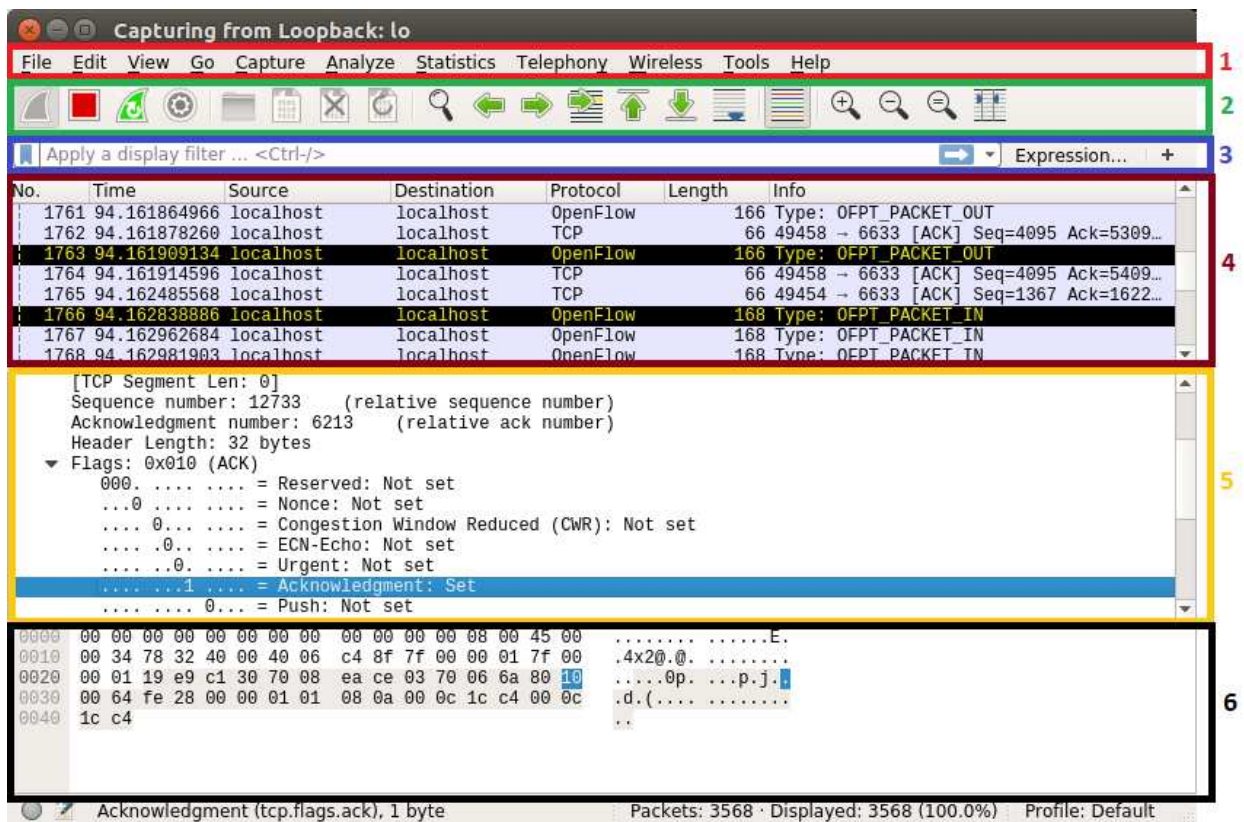


Figura D1. Áreas que contiene el programa Wireshark.

Fuente: Elaboración propia.

- 1.- Interfaz principal del programa Wireshark, es donde se tiene todas las opciones de uso mediante la distribución de varios menús.
- 2.- Barra de herramientas principal, permite el acceso rápido a las funciones más utilizadas y tiene la posibilidad de añadir o quitar otras herramientas.
- 3.- Barra de filtros, este espacio es para especificar el filtro que se desea aplicar a una determinada captura de paquetes.
- 4.- Ventana de paquetes capturados, en esta ventana se visualiza todos los paquetes capturados, mostrando su origen, destino, protocolo, tamaño e información importante sobre cada uno de los paquetes.
- 5.- Ventana de visualización del paquete, en esta zona se muestra a detalle la información que presenta el paquete seleccionado en la ventana anterior, y permite desglosar por capas cada uno de los campos de las cabeceras existentes.

6.- Ventana de información hexadecimal, muestra la información del paquete capturado por la tarjeta de red en formato hexadecimal y por otra parte también en caracteres ASCII.

Existen otros programas que realizan similares funciones como Iptraf, TCPtrack, Ibmonitor, PRGT Network Monitor, Capsa packet Sniffer, entre otros, algunos de distribución libre y otros comerciales, pero las ventajas de Wireshark frente a estos es que cuenta con bastante documentación sobre su uso, y sobre todo por el soporte que tiene para el protocolo OpenFlow que es esencial para el presente proyecto.